INTELLIGENT ROBOTICS LAB

智能机器人实验

基于视觉与多方式跟随的多机器人协同控制

第 5 大组共同实验报告

TonyPi 抱物穿行与标记路线控制

AR 直接跟随　地图定位与避障跟随

RGB-D 实时检测　多车 AR 队列跟随

第 1 组　杨开元 24345063、林楗棋 24363050
曹淇涵 24363003、刘小凡 24363059

第 2 组　陈凯华 24363156、刘彦廷 24363060
罗烝 24363064、许程奕 24363107

第 3 组　何奇涛 24363027、林子昂 24363052
李雨翰 24363043、陈韦诺 24363158

第 10 组　徐杰鸿 24363164、王振宇 24363096
林琢严 24363054、何爽 24363028

从动作录制、视觉识别到移动控制与队列联调，各组共同完成方案设计、场地搭建和实验整理。本报告汇总各组的实现、调试过程、问题分析与合作体会。

2026 年 10 月 20 日

摘要

本实验使用 TonyPi 人形机器人和 TurtleBot2 移动机器人，完成抱物穿行与多机跟随任务。TonyPi 需要抱起物块，沿围挡通道完成左转和右转，并在终点放下；后方车辆采用不同方法保持跟随，同时处理障碍、转角和目标暂时不可见等情况。

我们大组分工完成了人形机器人路线控制以及四类跟随方案。第 1 组以录制动作组和视觉反馈控制抱物行走，利用固定标记识别和 PnP 位姿解算判断路线阶段，并研究地图定位与避障跟随；第 2 组使用前车背贴 AR 标记进行相对位姿跟随；第 3 组采用 RGB-D 目标检测、连续关联和局部地图组织实时跟随；第 10 组将点云跟踪首车与 AR 跟随后车组成三车队列，结合扫描数据调整转向。

实验中，我们共同完成场地搭建、参数调整、程序联调和资料整理。调试结果说明，动作衔接、相机视野、标签尺寸、坐标变换和速度限制都直接影响协同效果。三车队列完成了直行、转弯与顺序停车，围栏和慢行前车引起的误避障仍需要改进；部分方法也还需要补充完整场景下的测试。报告结合各组的实现、现场现象和问题分析，总结了不同跟随方法的特点与不足。

关键词： TonyPi；TurtleBot2；视觉跟随；动作组；地图导航；多机协同

目录

摘要 I

1 实验任务与场地 1

2 小组分工与合作 3

3 系统架构与协同接口 6

4 TonyPi 抱物搬运与路线控制（第 1 组） 11

4.1 实验目标与总体方案 11

4.2 感知输入与目标定位 11

4.3 运动决策与控制方法 13

4.4 运行流程与参数设置 14

4.5 实验现象与结果分析 15

4.6 小组协作与改进总结 15

5 方式一：AR 标记识别与直接跟随（第 2 组） 16

5.1 实验目标与总体方案 16

5.2 感知输入与目标定位 16

5.3 运动决策与控制方法 17

5.4 运行流程与参数设置 18

5.5 实验现象与结果分析 18

5.6 小组协作与改进总结 19

6 方式二：地图定位与避障跟随（第 1 组） 20

6.1 实验目标与总体方案 20

6.2 感知输入与目标定位 20

6.3 运动决策与控制方法 21

6.4 运行流程与参数设置 23

6.5 实验现象与结果分析 23

6.6 小组协作与改进总结 24

7 方式三：RGB-D 目标识别与避障跟随（第 3 组） 25
7.1 实验目标与总体方案 25
7.2 感知输入与目标定位 25
7.3 运动决策与控制方法 25
7.4 运行流程与参数设置 26
7.5 实验现象与结果分析 27
7.6 小组协作与改进总结 27
8 方式四：AR 标记识别与队列跟随（第 10 组） 28
8.1 实验目标与总体方案 28
8.2 感知输入与目标定位 28
8.3 运动决策与控制方法 28
8.4 运行流程与参数设置 29
8.5 实验现象与结果分析 30
8.6 小组协作与改进总结 30
9 实验过程与结果分析 31
10 实验总结 38
参考资料 39
A 复现入口与协作清单 39
B 关键实现摘录与可执行契约 41
C 实际配置快照与参数复现 52
D 人形机器人感知、状态机与动作补充分析 58
E 地图跟随定位、规划与保护补充分析 74
F AR 队列跟随的完整启动命令 92

1 实验任务与场地

1.1 任务要求与验收口径

依据课程《第五–六章：基于视觉与 AI 定位的多机协同控制实验》[1]，系统同时包含人形机器人搬运和多车跟随两项任务。TonyPi 从指定位置抱起目标物体，在障碍通道中至少完成两个转角（左右转各至少一次），到达终点后放下物体；TurtleBot2 保持跟随并在终点区域停靠。课程总体评价由现场竞赛与方案、源码报告构成，权重分别为 80% 和 20%。手册中的任务平台数量写为 3–4 台 TurtleBot2，详细评分表按四台、四种方式设置；本报告为四种方式保留独立章节，实际参赛车辆与方式认定须由全队和教师确认。

表 1: 任务要求与对应记录

要求	系统对应内容	应提交的证据
抱持、转弯与放置	抓取动作、上肢锁定、A 左转/B 右转/C 停止	连续视频、动作文件版本、决策日志
不接触障碍物	人形路线约束；TurtleBot 代价地图与门控	接触次数、障碍位置、最小间距实测
保持跟随目标	标记身份、目标时效检查、失效处置	丢失次数、持续时间、重捕获记录
多种独立方式	四组分别说明观测来源和控制链	各组实现、接口及现场运行证据
协同到达终点	任务状态、各车停靠条件	全队同场运行记录与到达时间

表中的要求需要通过不同证据分别闭合：控制日志用于核对决策，连续视频用于确认持物、碰撞和最终停靠。只有软件状态结束而缺少现场记录时，不能据此判定整项比赛任务完成。

1.2 场地与感知约束

现场使用灰色地板及相对连续的灰色挡板构成通道，路线包含直行、左转、直行、右转和终点区域。地板接缝、胶带及其他地面标识均视为无关背景，不能作为路线识别标准。目标物块由早期蓝色调整为红色；抱持后物块可能遮挡画面下部，因此抓取确认与前方导航需要分别配置观察区域或头部姿态。

黄色雪糕筒可作为早期道路居中方案的终点候选；最新标记路线以固定标记 C 的距离触发停止。两种终点条件属于不同程序配置，不应在同一次实验记录中混写。图1仅表达路径拓扑，方向和长度以现场测量为准。

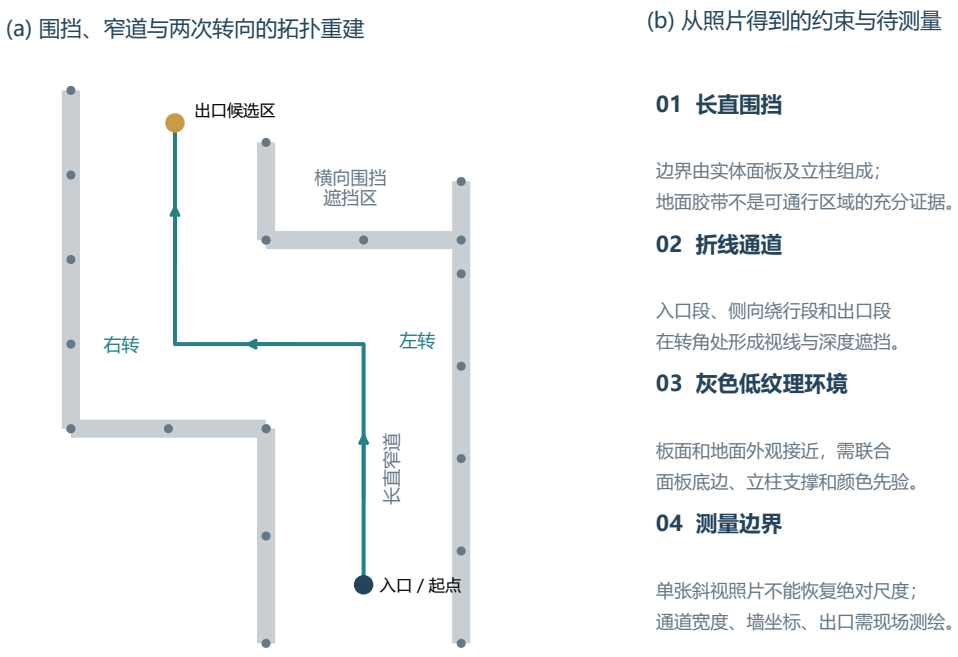


图 1: 路线拓扑示意；不按比例绘制，地面标识不参与定位

路线拓扑图用于说明 A/B/C 阶段与转向次序，不能代替场地测绘。拓扑上相邻的路段在相机视野中可能被围挡隔开，因此到达当前标记后仍需单独执行转角搜索。下面的实景照片补充展示围挡高度、通道边界与遮挡关系，使路线抽象与实际环境相互对应。



图 2: 灰色挡板通道实景（裁去无关前景，原图保留）。连续围挡形成包含转角与视线遮挡的狭窄通道，近端可见 TurtleBot2 平台。照片由实验小组提供，仅用于说明现场结构，不据此恢复绝对尺度。

实景中连续灰色挡板限制了跨路段观察，转角前后的目标可见性会明显变化。这解

释了人形控制采用“当前标记抵近—搜索下一标记”的分段逻辑，也说明移动底盘即使获得目标位置，仍需结合地图和实时扫描检查通行条件。照片中的地面胶带仅用于现场布置说明，不作为视觉导航输入。

2 小组分工与合作

我们大组由第 1、2、3、10 组共同完成实验。各组分别承担人形机器人控制和不同的 TurtleBot 跟随任务，在此基础上共同搭建赛道、讨论接口、完成调试并整理实验报告。组内采用主要任务分工与共同调试相结合的方式，四位成员承担的工作量大致相当；一个模块由一人牵头，遇到动作、感知或通信问题时，其他成员一起检查和调整。

2.1 大组任务分配

小组	主要任务	与其他组的配合
第 1 组	TonyPi 抓取、抱物行走、标记路线与转弯；方式二建图避障跟随	提供领行机器人的动作、目标外观和运行节奏，配合后车确认标签位置与跟随距离
第 2 组	方式一 AR 标记识别、相对位姿跟随与速度控制	与前车确认标签 ID 和尺寸，与其他组检查相机、TF 及速度话题
第 3 组	方式三 RGB-D 目标检测、身份关联、实时跟随与避障	根据 TonyPi 抱物后的外观调整识别条件，配合测试转弯、遮挡和终点停车
第 10 组	方式四前车 AR 跟随、三车队列、扫描避障与目标搜索	配置前后车 ID 对应关系，配合调整车距、转弯速度及队列停车顺序
大组共同	场地搭建、网络配置、多机联调、试验记录和报告整理	统一路线与车辆安排，交换调试现象，整理各组程序、图表和实验结果

表 2: 大组任务分配与相互配合

这里的小组编号与跟随方式编号不同，例如第 10 组负责方式四。联调时，我们还需要另外确认车辆编号、跟随目标和标签编号，避免把这些编号混用。

2.2 第 1 组分工

第 1 组围绕动作、视觉与状态机、场地与导航、数据与报告四个方向分工。动作效果会影响视觉判断，赛道宽度又会限制转弯方式，因此我们在现场一起观察和修改，而不是完成各自部分后才进行拼接。

表 3: 第 1 组成员分工

姓名/学号	主要负责内容	共同工作与配合
杨开元 24345063	路线状态机与标记转弯控制	负责串联识别、阶段判断和动作调用；与成员共同标定动作，检查左右转切换和异常处理
林楗棋 24363050	抱物动作组与行走、转向调整	负责动作组及姿态衔接优化，配合调试视觉纠偏；共同观察负载、步态偏移和转角效果
曹淇涵 24363003	赛道布置、动作辅助标定与建图调试	负责通道和转角布置，配合检查相机、扫描与地图；共同记录机器人姿态、障碍覆盖和运行现象
刘小凡 24363059	路线方案讨论、试验记录与报告整理	配合动作标定、导航和现场试验，整理程序说明、技术图表与视频；与成员一起分析问题并修改方案

第 1 组的工作从抓取动作开始，逐步扩展到抱物行走、转角判断和地图跟随。我们在调试中反复交换观察结果：动作偏移由控制和视觉部分一起处理，通道布局与相机视野问题则通过现场摆位和参数调整共同解决。

2.3 第 2 组分工

第 2 组从标签与相机、控制与门控、系统配置、试验与整理四个方向开展工作，重点把标记识别结果转换成稳定的跟随动作。

表 4: 第 2 组成员分工

姓名/学号	主要负责内容	共同工作与配合
陈凯华 24363156	AR 标签、Kinect 与坐标标定	配置标签尺寸和相机话题，检查内参与 TF；配合控制测试，核对实际距离与识别结果
刘彦廷 24363060	多机配置、目标 ID 与跟随场景	配置命名空间和目标编号，组织直行、转弯及障碍场景；配合检查前后车与标签对应关系
罗烝 24363064	跟随控制与速度门控	负责比例控制、速度限制与停止条件，配合整定参数；与成员一起检查目标丢失和扫描异常时的输出
许程奕 24363107	试验执行、数据分析与报告	整理实验步骤、运行记录及图表，配合多场景试验；与成员讨论跟随误差、摆动和停车现象

第 2 组在调试中将识别、控制和底盘响应放在同一条链路中检查。发现不跟随或转向异常时，成员分别查看标签、TF 和速度输出，再一起判断问题来自感知、参数还是执行。

2.4 第 3 组分工

第 3 组按感知、关联与控制、导航与保护、试验与整理四个方向安排任务。目标是让后车利用实时 RGB-D 信息识别并持续跟随 TonyPi，同时处理转角和障碍带来的影响。

表 5: 第 3 组成员分工

姓名/学号	主要负责内容	共同工作与配合
何奇涛 24363027	RGB-D 输入与目标候选检测	配置图像和深度输入，检查深度有效区域与候选筛选；配合采集不同角度和抱物状态的目标样本
林子昂 24363052	目标关联与跟随控制	负责连续观测、身份关联和跟随点计算，配合调整距离与转向参数，检查遮挡后的恢复过程
李雨翰 24363043	局部地图、导航与速度保护	配置点云、代价地图和运动限制，配合检查障碍覆盖与停车条件，并参与各模块联调
陈韦诺 24363158	试验场景、数据记录与报告	负责整理调试步骤、图像、日志及结果，配合目标样本采集、异常测试和报告撰写

第 3 组的感知与控制需要一起调整。候选检测不稳定时，我们先检查图像和深度，再核对目标关联；局部地图或跟随效果异常时，则结合点云、轨迹和实际画面一起分析。

2.5 第 10 组分工

第 10 组围绕三车纵列跟随开展工作，各成员分别牵头程序、标签配置、控制优化和试验记录，并共同完成现场调试。

表 6: 第 10 组成员分工

姓名/学号	主要负责内容	共同工作与配合
徐杰鸿 24363164	跟随程序与整体参数调试	负责动态跟踪程序及控制流程，配合队列运行；与成员共同分析转弯、避障和目标搜索现象

姓名/学号	主要负责内容	共同工作与配合
王振宇 24363096	标签配置、相机与车辆对应	负责标签安装、尺寸和 ID 检查，配合识别验证；共同调整相机视野、跟随距离和前后车位置
林琢严 24363054	控制优化与赛道布置	配合修改跟随和转向逻辑，负责场地布置与路线检查；参与直行、转弯和停车参数调整
何爽 24363028	障碍场景、运行记录与报告	配合代码优化、障碍响应测试和场地调整，整理现场问题、参数及实验结果，与成员共同完成报告

第 10 组通过前车和后车之间的配合检查队列行为。标签偏出视野、围栏触发避障或前车停车时，各成员结合车辆位置、扫描和程序状态一起调整，尽量减少单独调一台车对后续车辆造成的影响。

2.6 大组合作与共同收获

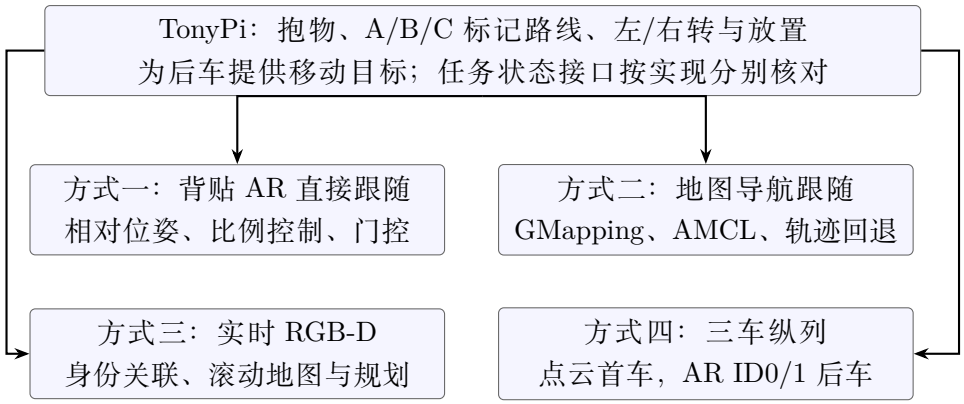
大组合作主要体现在三个方面。首先是共用赛道和设备，布置通道、贴标记和调整相机时，需要同时考虑人形机器人的摆动空间与后车视野。其次是交换调试信息，前车的速度、转弯幅度和停车时间都会影响后车，因此各组需要共同确定运行节奏。最后是整理实验结果，我们将各组原有报告、程序、照片和运行记录放在一起，统一符号、单位和图表，形成完整报告。

通过这次实验，我们体会到多机协同不仅是让每台机器人分别运动。只有目标编号、坐标系、速度输入和任务阶段能够对应起来，单机方法才能连成一个系统。各组承担不同模块，也在现场调试中相互补充：一组发现的问题往往需要另一组一起调整才能解决。

3 系统架构与协同接口

3.1 大组总体方案与各方式关系

系统按“领行搬运、目标观测、运动决策、执行与验证”组织。TonyPi 完成抱物路线，并作为移动跟随目标；各 TurtleBot 依据不同观测与环境表达完成跟随，不能将共用话题理解为共用控制器。图3概括方案关系；图中四类方式是四种方案，不表示这些场景都已完成同时运行。



连线表示方案关联，不表示同一控制器或同一已验收部署；
方式四队列为 TonyPi → 点云首车 → ID0 后车 → ID1 末车。

图 3: 第 5 大组任务与各跟随方案的关系；方式四内部还包含前后车链路

表 7: 四种跟随方式的共同口径对照

方式	目标观测与坐标	障碍响应	当前证据与主要缺口
一	前车背贴 ALVAR；相机或 TF 车体系	旧版停前进/转向；新版独立门控停车	已有 AR 实验说明；启动配置与定量测试待完善
二	TonyPi 标记转 map；AMCL 与历史轨迹	固定地图、实时扫描、move_base/DWA 与门控	部分软件/基础链验证；地图质量与全程跟随待测
三	RGB-D 身份目标转本车 odom	滚动点云代价地图、规划与制动门控	已有感知/控制代码；身份模板与现场验收待补
四	首车点云观测 TonyPi，后车 AR ID0/1 相对跟随	前/侧扫描净空、有界转向与搜索	三车联调完成直行/转弯/停车；专门障碍与重捕获未测

方式四的三车联调是一个子系统场景，不等于课程四台车采用四种独立方式的联合验收。方式一与方式四均含 AR 相对控制，独立性需根据完整控制链、目标选择和参赛安排说明，并由全队与教师确认。不得将“车辆数”“方式数”和“小组数”互换统计。

3.2 全队运行阶段与启动协同

联合试验按五个阶段组织：准备阶段固定围挡、地图、标签尺寸与车辆身份；就位阶段人工摆放物块、各车确认相机/定位及控制源；启动阶段 TonyPi 完成抱起，跟随车

在本方式必要输入满足后启动；运行阶段逐段通过 A 左转和 B 右转，分别记录领行与后车行为；收队阶段由各车独立核对进入终点及停稳后，再判定全队完成。

各组采用的启动条件不完全相同，联调时还需要分别检查。方式一新版、方式二、方式三依赖任务许可；当前人形`marker_route` 尚需任务桥适配。方式四新 C++ 节点直接使用标签和扫描，尚未接入`/competition/mission` 或同一个独立速度门控。若采用人工分时启动，应在记录中注明，不能将其称为自动任务联动。

3.3 速度发布权、故障传播与恢复

每台底盘只运行一个实际速度控制源；方式一旧节点输出 teleop，新门控方案输出 navi，而方式四默认直接发布`/mobile_base/commands/velocity`，绕开部分速度仲裁链。实际集成需登记唯一输出及停止机制，避免把终止一个上游节点当作停止所有底盘控制器。

领行相机掉线、掉物、目标丢失、扫描/TF 过期与导航失败分别记录。共享任务桥暂停不能默认停止未订阅该桥的方式四节点；全队停止需要覆盖每台车的实际命令入口。恢复前需重新确认目标身份、里程计/地图连续性和足够观察数据，不能让后车沿旧速度追赶。不同方式的原地等待、原地搜索或有界盲行策略存在差异，正文分别说明，不统一写成“目标丢失立即停车”。

3.4 终点收队与共同完成判据

TonyPi 放置只是领行任务的结束条件。全队完成还应同时满足参赛车辆进入终点区域、速度低于约定阈值并持续停稳、未发生未恢复的目标丢失或碰撞。可写为

$$G_{\text{team}} = G_{\text{carry}} \wedge G_{\text{place}} \wedge \bigwedge_{i \in \mathcal{R}} (G_{\text{arrival},i} \wedge G_{\text{stopped},i}) \wedge G_{\text{evidence}}. \tag{1}$$

$\mathcal{R}$ 为本次登记的参赛车辆集合，G_{evidence} 表示记录足以核对相应结论；该式用于说明整体完成条件，实际需要结合各车运行记录检查。终点区域和实际停稳阈值需现场明确，不能直接套用某一方式的目标距离。

3.5 感知、决策与执行

我们将系统分为感知、决策和执行三部分。相机或深度传感器提供目标与环境信息，控制程序选择动作或导航目标，舵机和底盘完成运动。人形机器人负责物块搬运与领行，各 TurtleBot 负责自身的跟随控制。调试时，我们先分别检查每一条链路，再检查它们之间的配合。

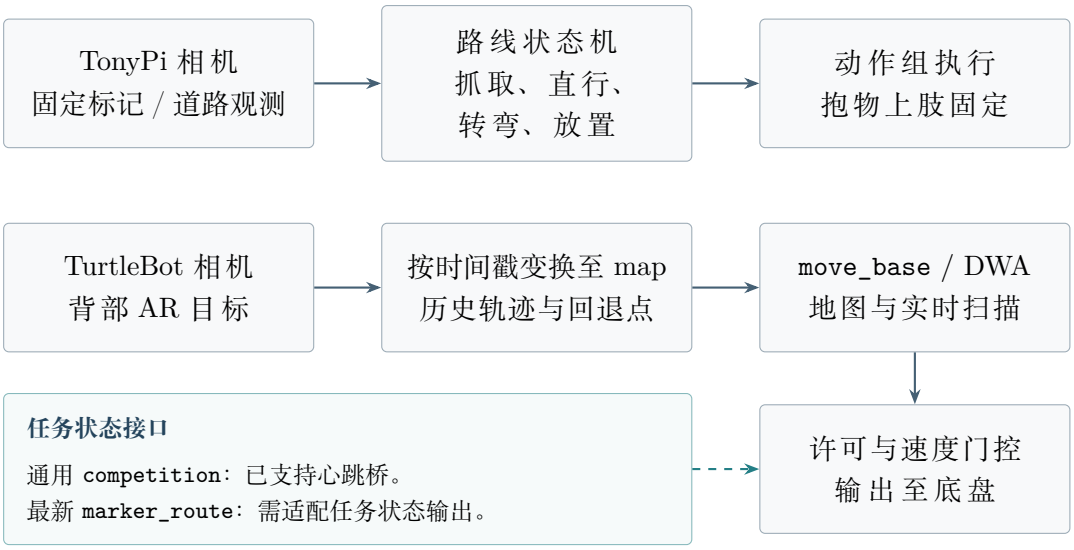


图 4: 人形机器人与地图跟随的数据链

图中的实线表示已经定义的感知和执行流程，虚线表示任务状态接口。后车能够看见前车，并不一定说明前车已经允许继续运行；例如 TonyPi 暂停或出现抓取问题时，跟随程序还需要相应停止。因此视觉观测、任务状态和运动输出应分别检查。

3.6 软硬件环境与功能包

TonyPi 使用 Python 3、OpenCV、NumPy、PyYAML 以及设备上的舵机 SDK。不同机器人的系统镜像和接口有所不同，我们在更换设备时需要检查串口、动作路径和相机输入。TurtleBot 采用课程虚拟机中的 Ubuntu 16.04、ROS Kinetic 和 Python 2.7 环境，各功能包放在`~/catkin_ws/src` 下编译。

AR 识别和速度门控放在`ar_marker_following` 中，地图跟随放在`multi_robot_following` 中，共用的轨迹与检查函数由`competition_core` 提供。这样可以复用功能，但也要求编译时同时包含相应依赖。各组调整程序后，还需要确认 launch 中填写的包名和实际脚本位置一致。

表 8: 多机联调中的主要接口

接口	内容	调试时检查的事项
/ar_pose_marker	带 ID 和时间戳的标记位姿	标签类型、实际尺寸、相机内参与坐标系
target_pose	转换后的目标位姿	目标来源与采样时刻，不能把旧位置当作新位置
/scan	障碍扫描	有效距离、视野范围、缺测及刷新情况
map->odom	地图定位变换	建图与 AMCL 定位分别使用，避免重复发布
/competition/mission	运行阶段	抱物、放置、完成和暂停状态的传递
/cmd_vel_mux/input/navi	导航速度	确认实际控制程序，避免与键盘遥控争用
/mobile_base/commands/velocity	方式四底盘速度	直接速度接口的停止与控制源检查

3.7 队列关系与通信配置

队列中的直接前车和采用的跟随方法是两个问题。方式二可以观察 TonyPi 的 AR 标记，也可以接收前车地图位姿；方式四则由首车跟踪 TonyPi，后续车辆识别直接前车的标签。每轮试验都应先确定这一关系，再设置目标 ID 和话题。

多台车需要使用不同的节点名称、命名空间和 TF 坐标系，同时保持地图原点与时间一致。虚拟机能够通过 SSH 访问，不代表机器人一定能反向连接 ROS；我们还需要检查局域网、ROS_MASTER_URI 和ROS_IP。联调时应关闭不使用的速度发布程序，保证每台底盘只有一个主要控制源。

4 TonyPi 抱物搬运与路线控制（第 1 组）

4.1 实验目标与总体方案

本部分的核心思路是保留人工录制且已经具备抱持效果的抓取动作，以动作组作为运动基本单元，将视觉观测作为动作选择与状态切换条件。机器人双足步态受负载、关节偏置及地面摩擦影响，固定动作次数容易累积航向误差，因此从开环演练逐步转向闭环控制。

表 9: 人形控制程序的用途与边界

入口	主要逻辑	用途与使用条件
`open_loop.py`	stand、grab、定次数前进/左右转、release	调试基线，不能补偿累计偏移
`competition.py`	识别/抓取确认、固定或视觉路线、放置确认；支持人工 stand 到 grab 起步	综合反馈框架，需现场 ROI 与任务接口配置
`lane_trial.py`	人工摆位后 stand 到 grab，有限次数道路居中/纠偏	局部闭环试验，不包含完整两次路口任务
`maze_route/marker_route.py`	抱起后依次靠近 A、左转找 B、右转找 C、停止放置	最新独立路线实现；局部实机验证，完整路线待验收

不同入口具有不同的感知与任务假设，后续状态机以最新标记路线为分析对象。复用旧入口的动作、持物确认或心跳功能时，应明确完成移植，不能仅因文件同处仓库就视为已经集成。

4.2 感知输入与目标定位

4.2.1 持物观察与头部姿态

早期通用视觉框架采用 HSV/LAB 分割、形态学去噪及轮廓筛选；物块改为红色后使用跨越 HSV 色相两端的双阈值。通用框架可在抓取后检查持物 ROI，失败时经复位再识别，并在放置后检查手中区域与地面区域。人工起步模式则从 stand 直接衔接 grab，省去搜索与接近阶段。

最新`marker_route` 默认原地播放`grab_final_v2` 后锁定上肢，**并未接入上述颜色持物确认、自动失败重抓和放置结果确认链**。因此“抓取动作完成”与“物块已可靠抓住”应分别记录，不能将通用框架的功能当作最新入口已验证的能力。比赛要求掉落后回到掉落点重新抱起；当前路线程序也没有完成该恢复闭环。

头部参数按程序和目标分别标定：早期道路/持物调试使用 PWM servo1=1200，

servo2 保持当前值；最新标记路线默认 pitch=1500、pan=1482，用来观察前方墙面标记。不同俯仰角会改变地面可见区域、物块遮挡与近距离码出画面的位置，报告应为每次运行登记真实参数，不能把两套姿态并写成同一配置。

4.2.2 道路边界与目标标记

地面与挡板都为灰色，仅靠颜色阈值难以区分。早期道路算法使用边缘、线段、挡板支撑立柱与底边的几何关系，排除孤立地缝和地面胶带。抱持物块所在区域单独遮罩，避免把红色物块边缘误判为道路边界。

在近、远两个图像截面分别得到左右边界 $x_L(y), x_R(y)$，可定义归一化横向误差与方向误差

$$e_c = \frac{x_L(y_n) + x_R(y_n)}{2W} - \frac{1}{2}, \tag{2}$$

$$e_h = \frac{x_L(y_f) + x_R(y_f) - x_L(y_n) - x_R(y_n)}{2W}. \tag{3}$$

其中 W 为图像宽度。组合误差超过阈值时选择左/右纠偏动作，误差处于容差内时执行一次前进，等待动作结束后重新观察。观测需连续稳定，不能在一张模糊帧上立即切换。挡板同色、遮挡、低对比和双足运动模糊仍可能导致无效道路判断；该方法不提供米制障碍距离或通用绕障规划。

4.2.3 固定标记识别与测距

最新程序不依赖地面标识，而是在墙面布置三张已知模板标记。每张标记由 9×9 格构成，外两格为黑边，中间 5×5 格为数据，黑色正方形边长 14 cm。检测过程为自适应阈值、四边形轮廓筛选、透视展开、单元格采样，以及四个旋转方向的模板匹配；允许的数据位误差为 1 格。它们是程序自定义模板，**不能当作 ArUco、AprilTag 或 TurtleBot 所用 ALVAR 标签互换**。

以标记四角为三维已知点，结合相机内参 K 与畸变参数，通过 PnP 求取旋转 R 和平移 t：

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} R \mid t \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}. \tag{4}$$

检测模块输出摄像头到标记中心的直线距离、方位角及墙面朝向。内参读取设备上的相机标定文件。码中心与镜头存在高度差时，PnP 直线距离不能直接等同于沿地面水平距离，转角距离需用真实摆位复核。

4.3 运动决策与控制方法

4.3.1 动作组与抱物步态

.d6a 文件实质是 SQLite 数据库，ActionGroup 表按Index 顺序保存动作帧。每帧由帧序号、持续时间（毫秒）及各ServoN 位置刻度组成，历史文件存在 16 通道和 18 通道两种宽度。总线舵机刻度与头部 PWM 脉宽不是同一种单位，不应将“刻度 500 中位”套用到头部 1200 或 1500 的 PWM 参数。角度与位置的关系可写为

$$q_i = k_i(p_i - p_{0,i}), \tag{5}$$

其中 p_i 为位置刻度，$p_{0,i}$ 为约定参考刻度，k_i 包含方向和角度比例。刻度 500 只是常用中位参考，不能单凭此值认定机械关节角为零；本方案主要复用实录动作，不依赖完整关节绝对角标定。

令抓取动作最后一帧的上肢集合为 $\mathcal{A} = \{6, 7, 8, 14, 15, 16\}$，步态第 k 帧各舵机位置为 $p_{k,i}^{\text{gait}}$，则抱物组合动作定义为

$$p_{k,i}^{\text{carry}} = \begin{cases} p_{\text{last},i}^{\text{grab}}, & i \in \mathcal{A}, \\ p_{k,i}^{\text{gait}}, & i \notin \mathcal{A}. \end{cases} \tag{6}$$

这一组合保持下肢帧序列、持续时间与其余通道不变，只替换六个手臂通道，减少转向步态破坏夹持姿态的可能性。历史版本分别组合过go_forward、go_forward_fast 及快速转向动作；最新标记路线使用grab_final_v2 和catch_ball_*，不能跨版本混用抓取末帧。

最新动作副本中的六个抱持刻度依次为 566、908、40、435、108、935。mkcopies.py 从设备原始动作生成独立副本，并逐帧核对未修改的腿部、索引和时间。放置动作仅在第一帧下蹲时保持抱紧手臂，后续降下、松开及站起帧保留原动作。运行时若首帧与当前姿态差值超过 25 刻度，将过渡时长延长到 300 ms；这一过渡不等于动态稳定性已获证明。

针对机体前倾，仓库另存两组大腿舵机相反方向的 20 刻度偏置候选，均带_offset 标记。该方案未作为当前机器人的默认步态，也没有完整实机稳定性结论，不能在报告中写成最终校正量。

4.3.2 路线状态机与纠偏

路线状态依次为准备、抱起、走向 A、左转寻找 B、走向 B、右转寻找 C、走向 C、停止与放置。距离阈值默认 40 cm；路口并非固定播放若干次就认定 90 度转弯，而是沿预定方向逐次转动，直到下一张目标码进入视野，再由直行阶段继续对中。每次转前后分别等待 300 ms，每次前进后等待 200 ms。

为降低步态大转角引起的左右摆动，程序对连续 3 帧观测取中位数，并根据近期单

次转向后的方位变化估计转角 $\hat{\theta}_{\text{step}}$。纠偏门限采用

$$|\beta| > \max\left(8^\circ, 0.6\hat{\theta}_{\text{step}}\right), \tag{7}$$

其中 β 为目标方位角。程序还从可见标记时的相邻步观测估计直行偏航，并在目标暂不可见时进行预测补偿。**标记不可见与相机无新帧是不同故障**：前者在计数约束下仍可能继续走，后者等待恢复。默认未见目标的盲行上限为 25 步，目标曾可见后连续丢失也有暂停阈值；这一实现与课程“丢失后原地重捕获”的严格要求存在差距，需在正式联调前收紧失效策略并验收。

当标记在近目标区域居中消失时，程序可使用最近观测判定已到达。这是处理近距离码从画面下缘离开的启发式，不能替代障碍测距。路口最多尝试 12 次转向；连续 20 步推进不足 5 cm、总步数过多或相机长时间无图像时触发暂停/退出。机器人没有深度或测距输入，对前方距离的判断仍有限，后续需要加强防碰撞测试。

4.4 运行流程与参数设置

我们先确认机器人摆位、抓取动作和前方标记可见，再启动相机观察窗口。准备完成后按 S 进入路线，程序依次执行抱起、走向 A、左转寻找 B、走向 B、右转寻找 C、走向 C 与放置；P 用于暂停，恢复前重新检查摆位与目标。路线距离阈值为 40 cm，头部朝向、动作停顿和纠偏门限随现场观测调整，并在每次运行记录中保存。

现场曾出现`/dev/video0` 消失，内核提示 USB 枚举错误和线缆异常，重新插接后恢复。早期通用框架通过本地 HTTP 快照共享网页相机，避免重复打开已被占用的设备；最新`robust_camera.py` 则按 USB 身份查找相机、处理网页相机释放和断线重连，只返回 0.5 秒内的新画面。两种相机管理模式应各自独立运行。

最新窗口显示运行阶段、目标标记框、标记 ID、距离与方位等信息；支持 S 启动/恢复、P 暂停、Q 或 Esc 退出。每次运行保存`route.log`、`decisions.csv` 及标注截图，用于核对状态转移的观测依据。识别框和数值可以帮助分析当时的判断，搬运效果还需要结合整个运行过程观察。比赛视频在 184 s 处记录了相机识别窗口与路线日志。下面保留原始屏摄画面，用于核对视觉检测和转角搜索分支；完整视频取证说明见第9.5节。

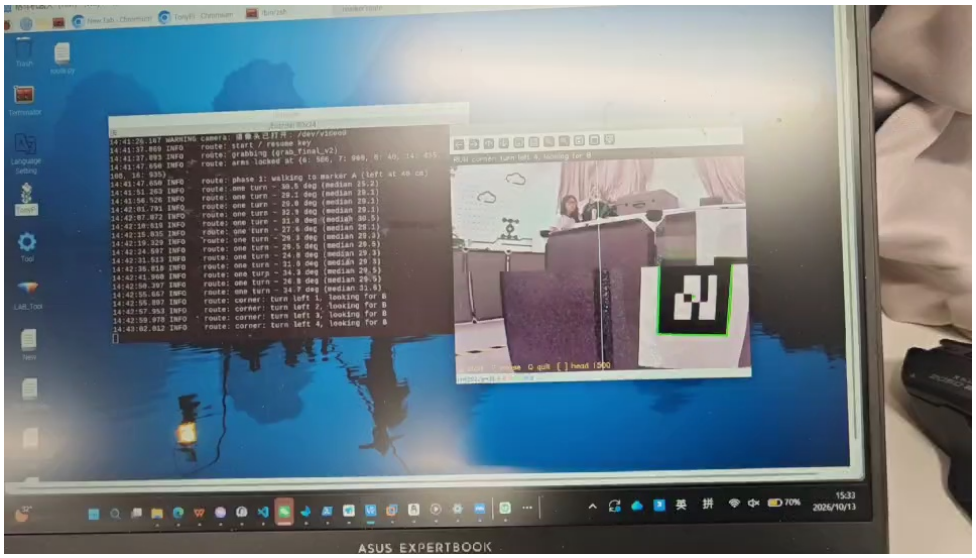


图 5: 比赛视频 184 s 处的标记检测与路线日志（V184）

画面中的标记检测框与寻找下一码日志提供了现场调试证据，但屏摄反光和透视限制读数可靠性，因此本图不用于计算距离误差或动作次数。

4.5 实验现象与结果分析

目前实机已跑通“抱起、走向 A、在约 39.6 cm 处开始左转”；完整 A/B/C 路线仍未实机验收 [3]。离线测试使用合成透视图验证标记识别与 PnP，并用给定步长、偏航和转角模型演练路线；合成实验精度与路线完成不能作为实地成功率。

后续应重点记录：满载单步距离与偏航、左右转角分布、两次路口完成率、物块掉落/接触次数、相机断线频次及完整路线时长。抓取成功确认、丢码原地重捕获、掉落恢复、终点放置确认以及任务心跳适配，仍需在最新标记入口中形成一致闭环。

4.6 小组协作与改进总结

第 1 组将动作录制、视觉识别、路线决策与现场测试分开推进，再把各模块放到同一条路线中联调。杨开元负责状态机与标记控制，林楗棋负责动作组合与步态，曹淇涵负责场地、视觉参数及建图配合，刘小凡负责运行记录、异常复测和报告整理。我们共同参与摆位、持物观察和参数调整，具体分工见第2节。

我们体会较深的是，单独完成抓取、识别或转弯，并不意味着连续搬运就一定顺利。后续应优先补齐持物确认、目标丢失和放置结果之间的反馈，再用同一组场地条件比较步态与纠偏参数。详细状态转换、测距和动作时序分析见附录D。

5 方式一：AR 标记识别与直接跟随（第 2 组）

5.1 实验目标与总体方案

本组使用 Kinect 识别前车背部的 ALVAR 标记，根据标签与后车的相对位置调整速度和方向。我们希望后车既能保持一定车距，又能在前车转弯时继续对准目标，并在扫描或目标信息异常时停止。[5]

实验中使用了两种直接跟随程序：`follow_ar.py` 直接在相机坐标下计算速度，便于检查识别与底盘响应；`marker_follower.py` 先将标记转换到车体坐标，再通过独立门控输出速度。两种程序的距离参数和保护条件不同，调试时分别设置，表中列出主要区别。

表 10: 两种 AR 直接跟随程序的比较

项目	相机坐标直接控制	车体坐标与独立门控
功能包	`ar_follower`	`ar_marker_following`
目标参数	`target_id`，默认 1	`marker_id`，默认 1
跟随距离	默认 0.4 m	默认 1.0 m
位置坐标	光学坐标 z 前向、x 横向	TF 到车体坐标 x 前向、y 左向
速度限制	线速度 0.3 m/s、角速度 1.0 rad/s	前进上限 0.12 m/s、角速度 0.45 rad/s
输出	直接发`teleop` 输入	通过门控后发导航输入
停止条件	空检测停车，时效保护不完整	检查目标、任务、扫描及指令时效
障碍响应	停止前进并慢速转向	门控停车，不负责路径绕障

课程还介绍了固定场地标签航路点的方法。我们这里选择把标签贴在前车上，利用相对位姿跟随，因此不需要后车预先知道前车的全局位置；相应地，标签的可见性对运行效果影响较大。

可令 TonyPi 贴 ID0、首车跟随 ID0，首车贴 ID1、次车跟随 ID1。每台车拥有唯一命名空间和 TF，但改变 ID 只改变目标，不改变直接跟随的控制律。该方案不提供地图路径规划，遇障会停车而非主动绕行；需要绕障时，应接入方式二导航或由对应组明确其他规划实现。

5.2 感知输入与目标定位

`ar_track_alvar` 接收 RGB 图像与 CameraInfo，输出`AlvarMarkers`。跟随节点只选取参数指定 ID，识别节点能看见其他 ID 也不代表控制节点会跟随。改变目标时，旧节点使用`_target_id:=1`，新版 launch 使用`marker_id:=1`，应核对实际入口后设置。

标签应为 ALVAR 格式，实际黑边尺寸须与`marker_size_cm` 一致。旧实验 launch 默认 14 cm，新包识别 launch 默认 9 cm，均不是现场尺寸已确定的结论。尺寸不一致会引入尺度误差；标记反光、视野边缘、遮挡、打印缩放和相机内参错误也会影响距离估计。多车应为直接前车分配明确标记 ID，并分别登记图像、内参话题与光学坐标系。

5.3 运动决策与控制方法

5.3.1 距离与方位闭环

早期节点直接使用相机光学坐标 z_m, x_m，控制思想为

$$v = \mathrm{sat}(K_v(z_m - d_0)), \qquad \omega = \mathrm{sat}(-K_\omega x_m). \tag{8}$$

光学坐标中横向向右为正，ROS 角速度向左为正，故转向项取负号。实机需核对坐标约定与安装方向，不能在 TF 或相机配置错误时仅用反号掩盖问题。旧节点扫描前方 60 度扇区，小于 0.25 m 时线速度归零、角速度设为 0.5 rad/s；目标太近时也停止前进，但扫描失联没有独立超时保护。

新`marker_follower.py` 将量测在原始时间戳变换到`base_frame`，保留有限且位于前方的目标 (x, y)。用 $\alpha = \mathrm{atan2}(y, x)$ 计算方位，源码控制律为

$$v = \begin{cases} \min\{0.12,\, 0.4(x - d_0)\}, & x > d_0 \wedge |\alpha| < 0.4, \\ 0, & \text{其他情况}, \end{cases} \tag{9}$$

$$\omega = \begin{cases} \mathrm{clamp}(1.2\alpha, 0.45), & x > 0.65, \\ 0, & \text{其他情况}. \end{cases} \tag{10}$$

距离以米、角度以弧度计。新版观测输入须在 0.5 s 内，控制阶段允许的最近目标年龄为 0.6 s，任务状态年龄为 1.5 s；只有 carrying、releasing、done 且未急停时发布有效许可。新版本不能依赖旧版“直接取相机 z”来解释车体 x 的数值。

5.3.2 速度门控与停止

新版原始速度发布到`cmd_vel_raw`，许可发布到`motion_allowed`，再由`velocity_gate.py` 检查并输出`cmd_vel_safe`。任务状态由上游控制器转换为许可，门控本身不直接订阅任务字符串。许可或速度超过 0.4 s 未更新、扫描超过 0.5 s 未刷新、急停有效、速度非有限或为倒车时输出零速度。

默认门控最大线速度 0.15 m/s、最大角速度 0.5 rad/s，前方停止距离 0.5 m；直接跟随控制器自身限制更低，不能用门控上限替代实际输出上限。直行扫描半扇区为 45 度，转动时扩大到 90 度，至少需要足够有效量测；NaN/Inf 缺测不能当作安全。壁钟线程用于处理 ROS 时钟暂停。目标丢失仍可能在目标时效窗口内沿用最近有效观测，不能写成瞬时硬件急停。

5.4 运行流程与参数设置

先确认底盘、相机、实际图像话题和标签 ID，再检查 TF 与/scan；只运行识别节点时不应期待底盘运动。任务状态、许可或扫描任一条件未满足，门控保持停止是预期行为。多车必须统一robot_ns，识别 launch 默认tb2、跟随 launch 默认tb1，不能同时依赖两套默认命名空间。

程序分成多个功能包后，启动文件中仍有部分节点保留了原包名。运行前需要把 launch 中的包名与脚本实际位置对应起来，再重新编译和检查节点。这个问题提醒我们，修改程序组织时不能只移动文件，还要一起检查启动配置和依赖。

表 11: AR 标签准备与目标编号设置示例

目标 ID	标签准备	控制入口参数
0	createMarker-s140	_target_id:=0 或marker_id:=0
1	createMarker-s141	_target_id:=1 或marker_id:=1
2	createMarker-s142	_target_id:=2 或marker_id:=2

表中 14 为示例标签边长，使用时应以实际打印尺寸为准。我们先在静止状态查看识别话题，再低速检查转向正负，最后逐步增加前车行走距离。临时调参后将有效值写回启动配置，避免下一次运行时忘记参数。

5.5 实验现象与结果分析

单标记识别时，我们观察到：标签进入相机视野后，/ar_pose_marker 中出现对应 ID；左右移动标签时横向坐标随之变化，靠近或远离时深度坐标变化。目标编号切换也要同时修改打印标签和启动参数，否则识别节点虽有输出，跟随节点仍可能不选中该目标。

跟随调试中的观察与控制规律一致：距离较大时向前跟进，接近期望间距后减速，目标偏向画面一侧时产生转向。前车急转或遮挡容易使标签离开视野，我们同时查看识别话题、运动许可和速度输出，区分检测丢失与门控停车。目前尚未完成完整多车路线的量化统计，后续需要在相同路线下重复运行并记录。

单标记调试主要检查标签移动与坐标变化的对应关系，以及距离和方位误差怎样影响速度。标签偏向一侧时，应产生相应转向；距离接近期望值时，应逐渐减速。目标 ID 切换和多车场景还需要分别运行，不能只改参数后就认为整个队列已经调好。

离线检查通过了 11 项控制与停止逻辑测试。它们有助于检查条件分支，但不能代替实际相机、TF 和底盘运行。后续还需要记录不同距离下的误差、转弯时的丢失时长，以及扫描或任务状态中断后的停车情况。

5.6 小组协作与改进总结

陈凯华负责标签、相机和坐标检查，刘彦廷负责多机配置与场景，罗恣负责控制和门控，许程奕负责试验记录与分析。我们共同进行参数调整和现场测试，具体分工见第2节。调试中最有帮助的方法是同时查看识别坐标和速度输出，这样能够区分“没有检测到目标”和“控制条件没有满足”。

直接 AR 跟随的优点是数据链短，便于观察相对误差与底盘响应；不足是对标签可见性较敏感，转角和近障时容易中断。我们把标签尺寸、ID 和命名空间与其他组逐项核对，使各组能够共用识别经验，同时分别检验各自的控制方式。

6 方式二：地图定位与避障跟随（第 1 组）

6.1 实验目标与总体方案

方式二将“看见目标”与“安全到达跟随位置”分开：相机观测 TonyPi 背部 AR 标记，在地图中恢复其位置并积累历史轨迹；随后由导航栈在已知地图和实时障碍约束下生成底盘速度。目标朝向不直接作为跟随方向，避免背贴标签朝向与实际行进方向不一致时生成错误目标。

独立 ROS 功能包`multi_robot_following` 位于`turtlebot/follow2/`，部署到虚拟机`~/catkin_ws/src`。包内包括建图入口、AMCL/move_base 配置、目标源适配、轨迹跟随与任务心跳桥。最新拆分将速度门控和 AR 识别 launch 移入`turtlebot/follow1/ar_marker_following`，并新增`turtlebot/follow2/competition_core` 共享包。因此方式二不再是仅复制一个目录就能完整启动的自包含包。

6.2 感知输入与目标定位

6.2.1 建图定位与障碍观测

底盘启动与键盘遥控已由小组确认可用：`turtlebot_bringupminimal.launch` 及`turtlebot_teleopkeyboard_teleop.launch`。建图优先沿用课程`turtlebot_navigation gmapping_demo.launch`，另用课程导航 RViz 配置查看地图、扫描和 TF。该入口已集成部分相机和导航节点，不与新包的建图入口重复启动。

建图时采集里程计和扫描，通过 GMapping 估计二维栅格地图；完成后使用`map_saver` 成对保存`.yaml` 与`.pgm`。比赛导航加载固定地图，AMCL 估计当前车位姿，move_base 进行规划。场地障碍布局改变需重新建图；从建图切换到定位时，必须停止 GMapping，避免与 AMCL 竞争`map->odom`。

现场曾报告约 35 cm 高障碍未在图中稳定出现。不能只按 URDF 假定镜头高度便认定扫描必然覆盖：应分别检查原始深度图、深度转扫描的采样行与范围、实际安装俯仰、TF、有效近距，以及 RViz 中`/scan` 是否已有对应点。若扫描本身漏检，调节地图膨胀层不能补回未观测障碍；若扫描有点而地图无障碍，再排查建图、TF 和清除配置。这个漏检问题还需要在相同障碍条件下继续复测。视频 8 s 处可见多台搭载电脑的 TurtleBot2 和现场标签布置，可用于说明实际平台外观。



图 6: 视频 8 s 处的 TurtleBot2 平台与现场布置（V008）

图中车辆与方式编号尚无法对应，故本图只作为平台实物记录，不把其中某台车认定为方式二实车，也不据画面推算镜头高度或标记尺寸。

6.2.2 目标观测与地图变换

方式二默认`target_source:=marker`，选取指定 ID 的 ALVAR 标记。`marker_target.py` 验证时间戳、有限数值与量测范围，在观测时刻 t_m 将目标变换到地图：

$$ {}^{m}T_{\text{target}}(t_m) = {}^{m}T_{c}(t_m)\,{}^{c}T_{\text{target}}(t_m). \tag{11} $$

其中 c 为相机观测坐标系，m 为地图坐标系。输出保留原始量测时间，TF 缺失、过期或外推失败时丢弃观测，不以当前时间重新包装旧数据。节点默认量测时效 0.5 s、最大量测范围 4 m，这些是代码限制，不是实际检测精度。

另一适配器可订阅前车地图定位位姿，但必须共享地图原点并检查时间戳、跳变和协方差。当前第 1 组方案以直接观察 TonyPi 为主；只有确认切换到前车定位模式时，才在实验表中登记该数据源。

6.3 运动决策与控制方法

6.3.1 沿历史路径生成跟随点

`Trail` 将有效目标位置记录为有界折线 $P = (p_0, \ldots, p_n)$。默认最小采样间距 0.08 m、最大保留路径长度 20 m、单次跳变上限 0.8 m。跟随距离默认 $d_f = 0.9\,\text{m}$，从最新点沿折线反向累计弧长；若回退点位于 p_{j-1} 与 p_j 之间，令该段长度 $\ell_j = \|p_j - p_{j-1}\|$、剩余距离 r，则

$$ p_g = p_j + \frac{r}{\ell_j}(p_{j-1} - p_j). \tag{12} $$

轨迹不足 d_f 时不生成有效目标。与沿目标当前朝向直接平移相比，这种方法保留已观测路径的转角信息。但 move_base 仍会在可通行区域平滑或重规划，后车的实际轨迹可能与前车不同，转角处仍可能出现切角。

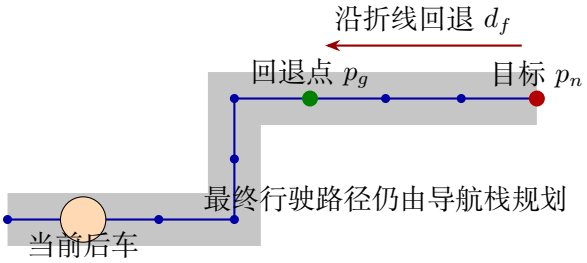


图 7: 沿目标历史折线回退生成导航目标；示意图不含真实场地尺度

历史折线回退沿目标已经走过的路径生成目标点，转角处不会简单沿当前视线把机器人拉向前车。图中回退距离由累计弧长定义，区别于两点直线间距。得到回退点后仍需执行足迹与地图检查，路径几何本身不保证底盘能够安全通过。

6.3.2 导航规划与速度保护

全局规划依托固定地图及目标点；全局与局部代价地图接收实时扫描，DWA 在速度与加速度约束下评估短时轨迹。候选目标在发送前执行圆形足迹栅格检查，越界、未知或高占用栅格均不可进入。足迹、膨胀半径及障碍更新参数必须按真实车体和通道标定，默认值不是经过测量的保证。

当前配置禁止倒车，并关闭自动原地旋转恢复，以降低拥挤通道中盲目恢复的风险。其代价是某些受阻情形需要重新规划或人工复位，而不能自行摆脱。视野之外、深度缺测或低于有效扫描面的物体仍可能漏检，导航栈不等于全空间避碰。

跟随控制层检查目标、定位、TF、地图/代价地图、任务状态与目标最小间距，满足条件才发导航目标与运动许可。独立`velocity_gate.py` 进一步要求许可、导航速度和扫描均新鲜，检查前方扇区有效量测并拒绝倒车、NaN/Inf 及非法速度。可概括为

$$u_{\text{out}} = \begin{cases} \text{limit}(u_{\text{nav}}), & G_{\text{mission}} \wedge G_{\text{target}} \wedge G_{\text{loc}} \wedge G_{\text{map}} \wedge G_{\text{TF}} \wedge G_{\text{scan}} \wedge G_{\text{cmd}}, \\ 0, & \text{其他情况.} \end{cases} \tag{13}$$

其中 G 表示相应许可；它们分别由上游与独立门控实现，并非一个节点完成所有检查。NaN/Inf 占满的扫描扇区被视为缺测，不当作空闲空间。门控具有壁钟检查线程，以避免 ROS 仿真时钟停顿后速度无法及时归零。

AMCL 在机器人静止时不一定持续发布位姿，程序定期请求无运动定位更新，并配置重采样频率，使新扫描可更新定位结果。请求本身不制造有效扫描或虚假新鲜度，定位故障仍需在 RViz 重新初始化或排查传感器。

6.4 运行流程与参数设置

地图跟随程序需要同时使用multi_robot_following、ar_marker_following 和competition_core。前者负责地图目标与导航，后两者提供识别、速度门控和共用计算。我们在工作空间中统一编译，并在每个终端加载环境后检查话题。

共用 Python 模块的安装路径也需要核对。程序能够从源码目录导入，并不一定说明 catkin 安装后的环境已经正确。调试时应检查competition_core 的安装目录及实际导入位置，避免运行到了同名的旧文件。

我们把软件检查与实机试验分开安排：先检查程序结构和输入缺失时的停止分支，再连接传感器与底盘，最后进行完整场景测试。这样可以逐步定位问题，但只有实际运行才能反映地图质量和避障效果。

表 12: 方式二部分默认参数（非最终实测值）

参数	默认值	作用
跟随路径间距	0.9 m	与目标沿历史路径保持滞后
最大前进速度	0.15 m/s	限制跟随底盘速度
最大角速度	0.5 rad/s	限制转向速度
线加速度限制	0.3 m/s^2	DWA 动态约束
控制/规划频率	10/1 Hz	局部控制与全局重规划
DWA 预测时长	1.5 s	轨迹评估窗口
平面目标容差	0.15 m	到达导航目标的容差

6.4.1 任务联动与终点收队

跟随器需要持续收到允许阶段的/competition/mission。通用competition.py 可通过 UDP 心跳桥提供状态；open_loop.py 不发送该心跳，最新marker_route.py 同样未实现该接口。因此“两个程序分别能启动”不等于“已经自动协同”：需增加适配器，将标记路线的搬运、放置、完成、暂停和故障映射到任务状态，明确频率、超时和停止行为后再联调。

单车导航调试时可以在终端持续发布carrying，但这仅是测试输入，不能写成全队自动联动已完成。终点还需核对后车是否真正进入终点区域并停稳，不能只用 TonyPi 的放置完成判定所有车辆到达。

6.5 实验现象与结果分析

已有部署记录表明功能包在 ROS Kinetic/Python 2.7 下完成构建、语法与 launch 解析检查；隔离 ROS master 中，在缺少目标、定位、任务与扫描时观测到零速度和禁

止运动。该检查未向真实底盘发送导航速度，证明的是部分失效路径的逻辑，不能推导实际绕障成功率。

本组已确认底盘和键盘遥控可用，地图质量、AMCL 稳定性、动态避障、目标丢失后原地恢复及全程跟随仍需按第9节采集真实数据。尤其应在 35 cm 障碍、转角目标遮挡和网络中断条件下逐项复核，而非直接进行多车混合测试。

6.6 小组协作与改进总结

第 1 组在完成人形控制的同时，将建图、目标坐标变换和导航分开调试。我们先用课程建图与遥控程序确认底盘、扫描和地图，再检查 AR 目标能否正确落在地图中，最后核对跟随器与速度门控。曹淇涵主要负责场地与建图配合，杨开元负责目标轨迹和任务联动，林楗棋配合动作速度与车距调整，刘小凡整理异常记录和复测结果。具体任务与人形部分共同安排，见第2节。

与第 2 组交流标签识别和相机配置，可以减少相同问题的重复排查；与第 3 组对照局部深度观测，可以帮助区分地图问题和传感器漏检。方式二更依赖地图、定位和坐标链一致，后续要先把这些条件稳定下来，再评价动态跟随。定位、规划、参数耦合与收队细节见附录E。

7 方式三：RGB-D 目标识别与避障跟随（第 3 组）

7.1 实验目标与总体方案

本组采用 RGB-D 信息实时寻找并跟随 TonyPi。程序包括颜色与深度候选检测、多帧关联、外观模板、历史轨迹目标、滚动代价地图和速度门控。目标与环境信息主要来自当前传感器和本车里程计，不要求提前建好全局地图。[6]

现有说明将检测阈值作为示例，要求登记真实身份模板并在独立样本上评估。区域深度有效不等于识别到了 TonyPi；颜色候选也不能替代身份连续性验证。源码和合成演练已提供，真实目标识别、导航与全队终点验收仍需对应组确认。

7.2 感知输入与目标定位

感知节点将 RGB 与配准深度按时间同步，以颜色候选或可选的深度人体轮廓提出目标区域。对候选执行面积、有效深度比例、深度离散程度和物理尺寸筛选；颜色区域深度采用有限样本统计，不将无效深度视为远处自由空间。设候选像素 (u, v) 深度为 z，相机内参为 f_x, f_y, c_x, c_y，其光学坐标近似为

$$X = (u - c_x)z/f_x, \qquad Y = (v - c_y)z/f_y, \qquad Z = z. \tag{14}$$

点云经采样及 TF 变换用于目标与局部环境表达；该几何过程要求深度尺度、配准和相机模型正确，不能仅凭有彩色框判断几何链可靠。

身份关联结合位置连续性和已登记外观模板，在多个候选接近时进行歧义判断。默认模板为空，只允许影子诊断；真机执行需登记 TonyPi 正面、背面、侧面及抱物等实际视角，并用独立负样本检查相似物体误识别。跟踪确认与重识别分别需要连续观测，不能把一次颜色匹配直接作为新身份。目标中断后不将未知位置直接连为已观察轨迹。

7.3 运动决策与控制方法

7.3.1 历史轨迹与局部规划

有效目标变换至本车`odom`，以历史路径回退点作为导航目标。与方式二相比，方式三不依赖预先保存的全局地图和 AMCL，但更依赖里程计连续性、局部深度覆盖及身份稳定。滚动代价地图依据实时点云更新，未知区域保守处理；深度缺测不能被错误转为空闲清除射线。

源码为目标丢失、任务过期、TF/里程计/代价地图无效设置停止与取消导航分支，并对导航失败、里程计跳变设置需复位的故障状态。重捕获确认后重建轨迹，避免沿遮挡期间的虚构路径前进。视野约束用于判断运动后目标是否仍可能处于相机可观察范围，不能据此保证遮挡不会发生。

7.3.2 运动许可与完成判据

独立门控结合命令和许可时效、观测覆盖、障碍点云以及速度制动包络限制运动。配置中的最大线速度 0.12 m/s、角速度 0.4 rad/s、回退间距 1.0 m 与最小欧氏距离 0.65 m 均为默认起点。制动模型需要实测减速度和反应时间，不能用导航栈加速度限制直接宣称物理停车距离。

完成逻辑同时考虑任务 done、目标稳定、回退点容差、车辆足迹进入标定的终点多边形以及低速持续保持。默认`finish_polygon` 为空，`hardware_calibrated`、`identity_calibrated` 和`depth_coverage_calibrated` 均为 false，表明当前共享配置并非已完成硬件标定的比赛配置。静止判断阈值和保持 2 s 为程序条件，实际到达仍需现场记录支持。

7.4 运行流程与参数设置

程序按输入检查、目标登记、跟踪确认、导航跟随和停稳判断依次运行。`rgbd_probe` 只用于检查选定区域的深度质量；`rgbd_identity_preview` 用于观察外观模板匹配；确认观测正确后，才进入跟随节点与速度门控的联调。静止观察时同时显示彩色图、候选区域和深度状态，检查配准、尺度与时间戳。

表 13: RGB-D 跟随的主要配置起点（待现场标定）

项目	配置值	调试时的含义
图像与深度同步容差	0.05 s	检查两路图像是否对应同一时刻
有效深度范围	0.4–4.0 m	过近、过远及缺测区域不作为可靠目标
候选深度有效比例	0.6	防止少量有效点代表整个区域
跟踪/重捕获连续帧数	4/6	连续确认后再使用目标身份
路径回退间距	1.0 m	由前车历史轨迹生成目标
最小两点距离	0.65 m	约束后车与当前目标的间隔
线/角速度上限	0.12 m/s、0.4 rad/s	限制导航输出
终点保持时间	2.0 s	到达后连续低速才判定完成

以上数值来自程序配置，作用是提供调试起点，不代表现场精度。目标模板为空时先进行不驱动底盘的观察；采集真实 TonyPi 的正面、背面、侧面和抱物图像后，再用其他物体作为负样本检验误识别。硬件、身份和深度覆盖标定通过后，才启用实际跟随。终点多边形要在本车里程计坐标中登记，运行期间不能随意重置里程计。

7.5 实验现象与结果分析

本组需要与第 1 组一起确认 TonyPi 不同方向、不同抱物姿态下的外观，并结合其行走速度调整跟随参数。与队列组交流时，还要明确首车实际使用的程序和目标信息，不能只根据“点云跟踪”的名称判断两者实现相同。

我们将合成输入和只显示结果的调试模式用于检查数据链，再通过实际图像登记目标特征。完整测试还应包括静止识别、直行、转角遮挡、相似物体和动态障碍，记录目标误切换、丢失时间与停车情况。当前现场测试还不充分，这部分需要继续补充。

表 14: RGB-D 方式分阶段检查与现有结果范围

阶段	检查内容	现有材料反映的范围
输入诊断	同步、深度有效比例、坐标系	已提供诊断与合成输入程序
候选与身份	多视角模板、相似物体、连续确认	已有算法，真实模板和独立样本待补齐
导航与保护	目标丢失、障碍缺测、里程计跳变	已有停止及取消分支，实车效果待复测
完整路线	连续跟随、转角遮挡、终点停稳	尚未形成完整现场统计

我们把“看到了候选区域”和“持续跟对了机器人”分开检查。前者主要看图像和深度，后者还要看身份关联与运动连续性。合成输入能够帮助排查分支和数据链，但全程表现仍应以实际路线和连续录像为依据。

7.6 小组协作与改进总结

何奇涛牵头 RGB-D 输入和检测，林子昂牵头目标关联与跟随，李雨翰牵头局部地图和速度保护，陈韦诺负责试验记录与整理。四人共同参与样本采集和联调，具体分工见第2节。我们需要把感知画面、深度和机器人运动放在一起检查，避免只在单一模块中反复调参。

与其他组联调时，我们共用 TonyPi 的行走与抱物场景采集样本，并交流转角遮挡和近障停车现象。下一步应优先补齐实际身份样本和深度覆盖测量，再逐步延长跟随路线，避免同时调整识别、导航和速度而难以判断问题来源。

8 方式四：AR 标记识别与队列跟随（第 10 组）

8.1 实验目标与总体方案

本组使用三台 TurtleBot2 纵列跟随：一号车以点云视觉跟踪 TonyPi，二号车跟随一号车，三号车跟随二号车。二、三号车通过 Kinect 识别前车背部 AR 标签，根据相对位置控制底盘；深度图转换得到的扫描用于近障响应。该方案不使用预建地图和全局路径规划。

二、三号车使用`team_turtlebot_follow_plus` 中的 C++ 节点。它们直接根据标签和扫描产生速度，区别于通过前车 AMCL 位姿和地图规划进行跟随的方案。

一号车和二号车尾部分别粘贴 ID 0 和 ID 1 标签，三号车尾部未贴标签。二号车识别一号车的 ID 0 标签，三号车识别二号车的 ID 1 标签。一号车通过点云数据识别并跟踪 TonyPi，不使用 AR 标签。以下功能包和启动参数说明适用于二、三号车。

8.2 感知输入与目标定位

功能包`team_turtlebot_follow_plus` 使用 ROS Kinetic/C++ 实现，位于`turtlebot/follow4/team_turtlebot_follow_plus/`。`ar_track_alvar` 根据 Kinect RGB 图像和相机内参发布标签位姿；`depthimage_to_laserscan` 将深度图转换为`/scan`。跟随节点订阅`/ar_pose_marker` 与`/scan`，向`/mobile_base/commands/velocity` 发布速度指令。

相机输入、内参和标签尺寸应成套核对。二、三号车分别只选择 ID 0 和 ID 1；其他标签即使进入画面，也不应改变直接前车的身份。深度扫描负责近障判断，标签位姿负责跟随方向，两种观测承担不同任务。

8.3 运动决策与控制方法

设相机光学坐标系中的标签位置为 (x, z)，其中 x 向图像右侧为正、z 指向相机前方，则标签距离与方位角为

$$r = \sqrt{x^2 + z^2}, \qquad \theta = \operatorname{atan2}(x, z). \tag{15}$$

控制节点以 10 Hz 运行，按距离误差比例调节前进速度，按方位角调节转向速度；线速度限制在零至设定上限之间，目标距离不足时不倒车，标签偏离车头方向较大时暂停前进。

避障逻辑将扫描数据划分为前方及左右侧扇区。前方出现近障时，节点选择有足够扫描净空的一侧并保持该转向选择；距离达到停车阈值或所选侧净空不足时停止前进。扫描数据超时则发布零速度。目标首次出现前车辆保持停止；已识别目标丢失后，车辆低速原地搜索，超过设定时限仍未重获目标则停止。该方法依赖标签保持在相机视野内，不能沿被遮挡路线推断前车位置。

8.4 运行流程与参数设置

二、三号车使用相同的跟随和避障参数，目标标签 ID 分别设为 0 和 1。表15列出实际启动值及参数用途。

表 15: 二、三号车运行参数及含义

参数	单位	二号车 / 三号车	含义
`marker_id`	–	0 / 1	目标标签编号。节点只跟随该 ID 对应的标签。
`marker_size`	cm	14.0 / 14.0	标签黑色方框的实际边长，用于 AR 距离估计，应与打印尺寸一致。
`desired_distance`	m	0.40 / 0.40	Kinect 到标签的目标距离，不等同于两车外壳间距。程序不倒车；标签近于设定距离时停止前进。
`max_linear_speed`	m/s	0.35 / 0.35	最大前进速度。实际速度随距离误差及避障控制变化。初次低速测试设为 0.05–0.10 m/s。
`max_angular_speed`	rad/s	0.25 / 0.25	正常跟随及近障转向的角速度上限。
`avoid_distance`	m	0.38 / 0.38	前方障碍进入该距离后，开始减速并向扫描较空的一侧偏转。
`stop_distance`	m	0.20 / 0.20	前方障碍达到该距离后，线速度置零；侧向净空足够时向较空的一侧转向。
`side_clearance`	m	0.40 / 0.40	转向侧所需的最小扫描净空。两侧均不足时车辆停止。
`avoid_release_margin`	m	0.20 / 0.20	障碍距离超过`avoid_distance` 加该余量后，退出避障状态并恢复跟随。
`obstacle_turn_speed`	rad/s	0.20 / 0.20	达到停车距离后，车辆向选定空旷侧转向时使用的角速度上限。
`marker_timeout`	s	0.8 / 0.8	标签超过该时长没有新检测结果后，节点开始目标搜索。
`lost_search_timeout`	s	16 / 16	原地搜索目标的最长时间；超时仍未重获目标时停止。
`search_angular_speed`	rad/s	0.20 / 0.20	目标丢失后原地搜索时的角速度，且不超过`max_angular_speed`。

运行时先编译并加载工作空间，再分终端启动底盘、Kinect 和深度转扫描节点；确认扫描连续更新后，分别以 ID 0 和 ID 1 启动二、三号车。初次测试先以 0.05–0.10 m/s 观察响应，再结合场地调整；表中的 0.35 m/s 是启动配置的速度上限。完整命令见附录F，便于调试时查阅。

8.5 实验现象与结果分析

三车联调结果见表16。本次完成了直行、转弯和依次停车，没有专门布置障碍物，也没有出现目标丢失，因此避障与重捕获还需要增加相应场景测试。

表 16: 方式四实验记录

测试项目	实际情况	结果
直行跟随	队列间距保持稳定，速度适中，车辆间未碰撞	完成
转弯跟随	采用圆弧轨迹转弯，保持后车对前车标签的观测	完成
障碍响应	最终场地未专门布置障碍；未碰撞围栏，但围栏及长时间停留的前车曾触发误避障	有待优化
目标保持	本次联调未发生目标丢失，重捕获逻辑未实际触发	未覆盖
终点停车	三车依次到达终点并平稳停靠	完成

赛道围栏未发生碰撞，但当前车低速行驶时，后车曾把围栏识别为障碍并触发避障；当前车长时间停留在近处时，后车也曾将其判断为障碍，继而原地转向。后续需结合赛道宽度和前后车间距调整避障距离及状态切换条件，并在围栏邻近、前车慢行和停车场景下复测。测试期间目标未丢失，目标重捕获功能尚无实际触发记录。

8.6 小组协作与改进总结

徐杰鸿负责跟随程序与参数，王振宇负责标签和车辆配置，林琢严配合控制优化与场地布置，何爽负责障碍场景、运行记录和报告。我们共同完成三车联调，遇到围栏和停车前车引起的误避障时，一起观察扫描和车辆位置再调整参数。详细分工见第2节。

本组以点云视觉跟踪和 AR 码标签跟随组成三车队列：一号车跟踪 TonyPi，二号车识别 ID 0，三号车识别 ID 1；标签边长为 14 cm。联调中三车完成直行、圆弧转弯和依次停车，车辆未发生碰撞或目标丢失。围栏和慢行前车引发的误避障仍需通过参数调整和复测处理；本次未进行专门障碍布置，不能据此判定复杂障碍下的避障性能。

9 实验过程与结果分析

9.1 实验安排与检查结果

我们先进行程序和单机调试，再逐步连接相机、底盘及多车任务。表中汇总了各部分完成的检查和仍需补充的试验。程序能够运行只是第一步，实际效果还需要结合现场画面和重复测试来评价。

表 17: 各部分实验检查情况

项目	检查内容与结果	进一步测试
动作组	核对组合动作的帧序、时长和手臂位置	满载步态、抓取及放置稳定性
标记路线	完成标记识别与局部路线调试，走向 A 并开始左转	A/B/C 完整路线重复运行
相机输入	重新接插后恢复取流，连续取得 640×480 画面	舵机负载变化时的断线情况
基础底盘	启动与键盘遥控可用	相机、扫描和导航联合运行
控制逻辑	11 项离线控制与停止检查通过	实际通信、底盘响应和停止延迟
三车队列	完成直行、圆弧转弯和顺序停车	专门障碍、丢失与重捕获场景
大组联调	结合现场视频检查前后车及转角现象	各方式同轮运行与完整终点记录

9.2 分阶段实验与异常注入

1. **单动作与视觉**。人工监督下分别测试站立、抓取、抱物前进、左右转和放置；固定相机姿态后测量标记距离/方位，统计多次误差与失效距离。

2. **人形单段与完整路线**。先测直行偏移，再测 A 左转/B 右转，最后无人工干预完成 A/B/C 全程。分别记录动作次数、转角偏差、接触、掉落及相机断线。

3. **地图和单车导航**。确认 35 cm 障碍进入原始深度与扫描，再检查地图。先静态目标导航、定位丢失和规划失败，再接入移动 AR 目标。

4. **目标与任务失效**。遮挡标签、停止目标消息、停止任务心跳、断开扫描或使 TF 过期，逐项测量停止延迟。人形最新程序的盲行与近距丢码启发式应单独评估。

5. **多车联调**。固定队列拓扑、共享地图与时间基准，避免速度控制源冲突；记录各车到达终点条件，不只记录领行机器人结束。

9.3 指标定义

距离标定用均方根误差 $\mathrm{RMSE}_d = \sqrt{N^{-1}\sum_i (\hat{d}_i - d_i)^2}$；$d_i$ 必须是与 PnP 输出同一定义的真实距离。跟随间距误差定义为 $e_f(t) = d(t) - d_0$，报告平均绝对误差与最大偏差，同时说明 d 采用欧氏距离还是路径弧长。完整任务成功率定义为 $N_{\mathrm{success}}/N_{\mathrm{trials}}$，每次试验均需记录，不能剔除失败试跑而不说明。

对接触次数、掉落次数、目标丢失持续时间、最长无新帧时间、停止延迟和各车到达时间分别统计。转角是否达到路线要求应通过外部测量或可复核视频判定，不能把“看见下一张码”直接记为精确 90 度。

表 18: 后续量化试验安排

项目	测量方法	用途
标记距离与方位	固定摆位、多距离和多角度比较	检查视觉判断与实际距离的关系
满载步态与转向	重复执行单步及左右转动作	统计步长、偏航与转角分布
完整路线	连续运行并记录失败轮次	比较路线完成率与运行时间
35 cm 障碍	同时查看深度、扫描及实际运动	排查漏检与避让效果
故障停车	分别中断目标、任务和扫描输入	测量发现异常到实际停稳的时间
全队终点	连续拍摄各车进入并停稳	检查队列到达与停车顺序

目前的记录以调试现象为主，量化比较还不充分。后续按表中方法重复试验，记录运行参数、日志和视频，并同时保留成功与失败的情况，才能判断一次调整是否有效。

9.4 数据归档与照片使用

运行记录模板保存在`latex/data/experiment_records.csv`。每行对应一次试验，保存代码版本、设备、运行参数、结果及证据路径；原始视频和 bag 较大时放在团队共享存储中，报告记录可访问的证据位置。不要将不同机器人的动作参数或不同版本的相机设置合并统计。

我们拍摄了通道、机器人和调试窗口，并从比赛视频中选取了运行画面，用来说明场地和程序判断之间的关系。下一节按视频时间介绍这些现象；单帧画面只反映某个时刻，完整过程还需要结合连续视频。

9.5 比赛视频证据与源码对应

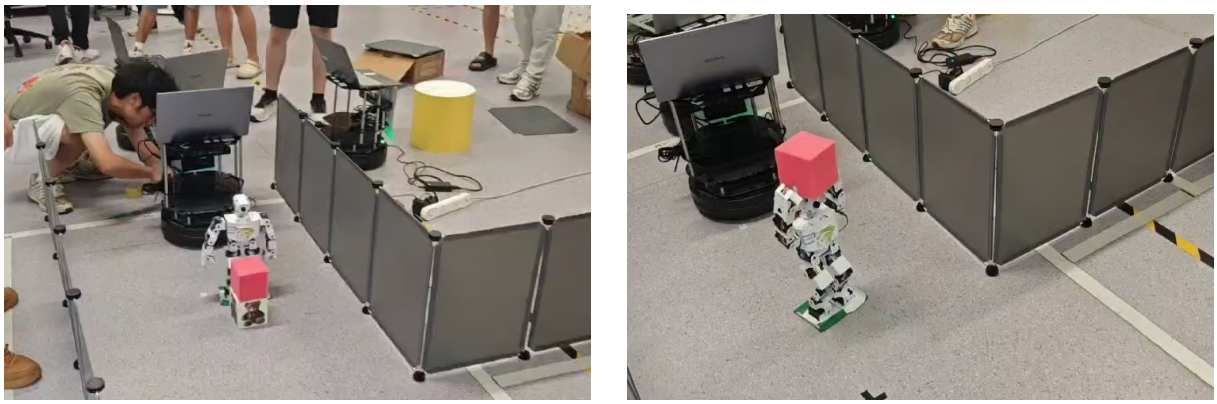
9.5.1 材料范围与取证方法

本次新增原始材料为“比赛视频.mp4”，文件名由提供者给定；其视频流包含 6830 帧，分辨率 960×544，名义帧率约 29 fps，总时长 235.52 s。本文时间均为视频起点后的相对时间，不是 ROS 时间，也不是任务起止计时。原片在机器人、操作者电脑与其他现场位置间多次切换，不能以视频总时长计算比赛耗时。截图从原片解码保存，仅裁去无关边缘以突出机器人和屏幕，原始帧完整保留，不生成或补绘实验内容；帧号、时间及原文件 SHA-256 摘要记录于`data/video_evidence.json`。

由于提供者无法确认车辆与方式编号的对应关系，本文使用“近端跟随车”和“后续车辆”作为画面描述。带有某种标签的外观并不唯一确定 ROS 节点或输入源；图像中的排队顺序也不等于`tb1` 至`tb4` 命名空间顺序。因此，下面的图像用于验证现场现象，算法归因以代码和运行配置为准。

9.5.2 起始准备与持物状态

图8将准备阶段与后续抱物阶段并列，目的是区分“机器人出现在场内”和“已经处于持物运行状态”。76 s 画面中物块位于机器人身前的低位，人员仍在邻近底盘处操作；133 s 画面中可见物块抬起，TonyPi 与移动底盘处于同一围挡通道。两帧不覆盖抓取全动作，不能由此认定抓取期间完全没有人工干预。



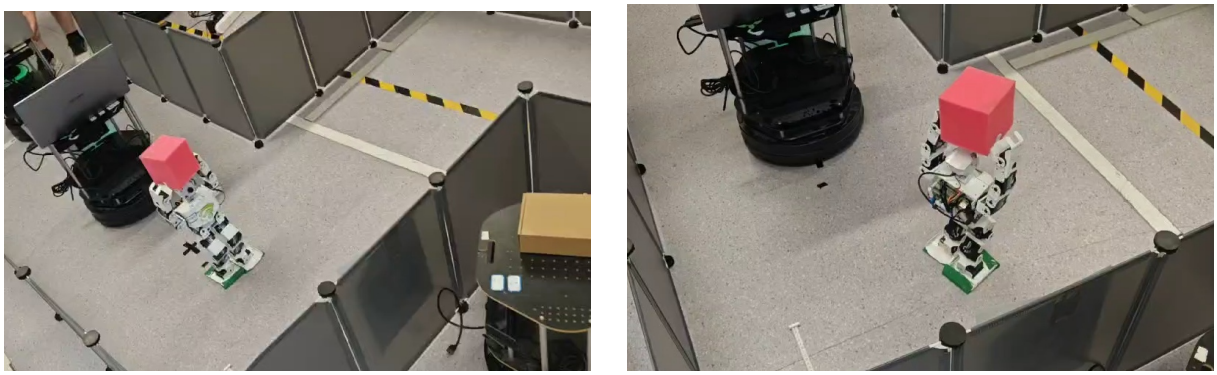
(a) V076：76 s，起始准备与近端底盘　　**(b)** V133：133 s，物块抬起后的现场

图 8: 准备与抱物画面对照，均为原始视频帧。

这组对照支持在报告中加入真实持物场景，但抓取可靠性仍不能只用一帧终态评价。源码的抓取动作结束与物块实际被可靠夹持是两个事实：前者可由程序日志确认，后者需要动作全过程和物块状态的连续观察。人员出现在机器人附近也只按画面记录，不据此推断其是否操作遥控。

9.5.3 通道推进与遮挡约束

图9选择 165 s 和 196 s 两个视角，用于展示抱物状态下的人形机器人、近端移动底盘及折线围挡的相对关系。与前一组准备图相比，这一组重点在于运行空间：通道限制横向余量，转角使后续车辆与目标间的直接视线变化，拍摄视角也随之改变。



(a) V165：165 s，围挡内的抱物运行画面 **(b)** V196：196 s，后续通道位置

图 9: 通道内的异构平台运行场景；不据透视图估计米制间距。

两帧说明真实围挡与机器人共同构成了遮挡和通行约束，但镜头移动使像素位移无法直接换算机器人路程。没有平面标定、外部位姿或可复核尺度时，本文不从画面测量 0.9 m 跟随间距，也不计算转向角误差。近端底盘与 TonyPi 同场出现不等于已证明其使用方式二或方式四，更不能由这些时刻的画面推出四车始终同步。

9.5.4 标记识别与状态转换的调试证据

184 s 处镜头转向操作电脑，画面同时包含相机标记识别窗口与路线终端日志。图10的用途与前两组实景不同：它提供视觉目标与状态机分支同时存在的现场记录，可与`marker_route.py` 中的路线观测、单次转向和`corner` 搜索分支对应。

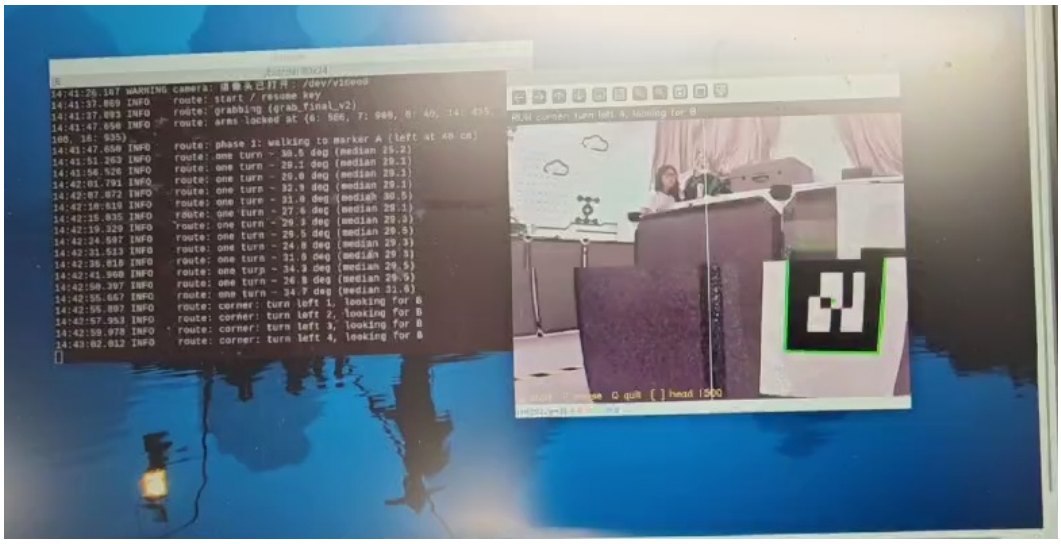


图 10: V184：184 s，电脑上的标记检测窗口与路线调试日志。保留屏幕反光与拍摄透视，不把不可读字符补写成测量值。

画面可辨认到标记检测框及寻找下一标记的路线日志，与“到达当前目标后进入转角搜索”的控制结构相符。这是源码状态转换具有现场调试对应物的证据，不能替代原始`decisions.csv`，也不据屏摄数字计算距离误差或控制周期。电脑显示的系统日期、终端时刻与视频相对时间尚未同步，本文只使用后者定位证据。

9.5.5 后段画面与完成条件的界限

图11取 218 s 处后段画面，目的是记录视频后段仍可见的人形持物状态和邻近底盘位置。该图不能标为“终点放置成功”：当前视角没有完整展示物块由手臂释放、稳定留在目标区域以及末车持续停稳的全过程。因此，本次可将证据等级从“仅有源码和照片”更新为“有现场视频支持的部分运行过程”，同时保留完整任务成功率、末车到达率和终点保持时间的测量空缺。未观察到某事件不等于事件未发生；本文只说明提交材料未形成足够的完整验收证据，不据此否定实验的其他未拍摄阶段。

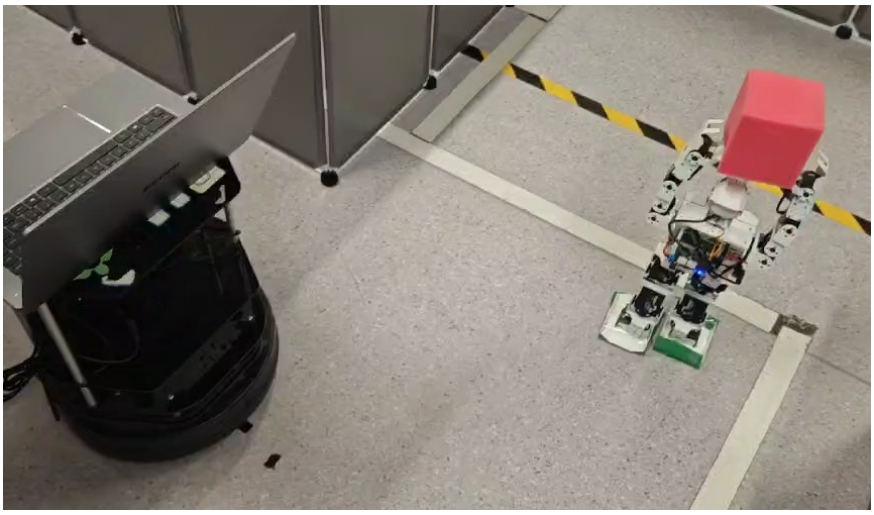


图 11: V218：218 s，视频后段的机器人与物块状态。该帧不构成终点放置或末车停靠验收。

该帧保留的是这一时刻的可见物体与机器人位置。终点验收还需在同一连续时间段内确认释放、物块稳定放置、机器人进入规定区域及末车保持停止，不能由单帧外观代替。

9.5.6 证据归档及后续补充方式

下表把已经获得的材料与仍需补齐的内容分开，使后续补充有明确对象。完整视频不重复装入 Overleaf ZIP；项目中保留被实际引用的原始帧和可复核索引，原片继续使用本地原路径归档。

表 19: 视频与代码共同支持的结论及剩余证据

问题	本次已获得	仍需材料
TonyPi 抱物运行	V133、V165、V196 等现场帧	全过程日志及掉落、碰撞核验
标记状态机	V184 检测窗口和转角搜索日志；对应代码	原始决策日志与同步时间基准
底盘与队列	多车布置及近端底盘同场画面	车辆编号、启动参数、话题连接
方式四控制机制	前车位姿模式、轨迹与守卫代码	对应实车日志和输入实际频率
终点完整验收	目前后段帧不能闭合放置与停稳链	放置全过程、末车停靠、全队结果

后续可以直接提供起点至终点的连续录像或运行日志目录，无需先整理成报告。若能再提供每台车对应的命名空间和启动命令，便可把“近端跟随车”替换为实际方式编号，并把视频事件与位姿、许可和速度时间线对齐。在这些信息缺失时，保持画面事实与算法机制分别论证。

9.6 大组联调与结果比较

各组先完成各自的方法调试，再在共同赛道中检查前后车之间的配合。联调时需要同时记录领行机器人和后车，尤其关注转角遮挡、前车停车、车距变化以及终点顺序。不同时间完成的单车试验可以帮助分析问题，但评价整体效果时仍需要一次连续运行。

表 20: 各部分实验情况与后续测试

项目	主要检查内容	已有实验材料	后续需要完善的内容
人形搬运	抓取、抱持、左右转与放置	动作组、现场照片和视频	全程重复运行、掉落与接触统计
方式一	标签识别、车距和转向	AR 实验步骤与控制说明	距离误差、转角丢失与停车时延
方式二	地图定位、绕障和路径保持	导航配置、地图与程序检查	矮障碍覆盖、完整路线跟随
方式三	身份保持、深度和局部导航	RGB-D 感知与控制程序	实际样本、遮挡和相似目标测试
方式四	车距、转弯和顺序停车	三车联调及问题记录	专门障碍与目标丢失场景
全队配合	同轮任务中各车到达与停车	多车现场画面与各组记录	连续运行记录和重复试验统计

9.7 共同发现的问题

各组在同一赛道调试时遇到了相互关联的问题。TonyPi 的转弯和步态会影响后车标签的可见性；围栏较近时，局部避障容易与跟随方向冲突；前车长时间停车时，后车可能把它当成近障。方式二还需要检查深度扫描能否覆盖约 35 cm 高的障碍，这一问题不能只通过调整地图参数解决。

我们将这些现象放在一起讨论，再分别修改动作、相机位置、跟随距离和速度限制。方式四的联调没有发生目标丢失，因此还不能评价实际重捕获效果；没有专门布置障碍的试验，也只能说明当时通道中的运行情况。后续需要增加相应场景，并保留成功和失败的记录。

9.8 报告整理与小组配合

报告整理时，我们尽量沿用各组原来的实验目的、步骤、控制公式和结果分析，只统一了符号、单位、图表编号与人员分工。第 1 组整理人形和地图导航部分，第 2 组整理 AR 实验，第 3 组整理 RGB-D 方案，第 10 组整理三车队列。各组相互检查前后接口，并共同完成总体设计、场地说明和实验总结。

这次合作让我们看到，同一个现场现象可能由多个环节共同造成。后车转向异常，不一定只是增益问题，也可能与标签朝向、前车步态或扫描触发有关。把各组观察到的情况放在一起，比各自只调单一参数更容易找出原因。

10 实验总结

这次实验把前面课程中的动作录制、视觉识别、坐标变换、建图导航和移动控制结合在了一起。我们大组分别完成了 TonyPi 抱物路线、AR 直接跟随、地图导航跟随、RGB-D 实时跟随和三车队列方案，再通过共用场地和相互调试检查各部分之间的关系。

对第 1 组而言，难点是抱物后步态容易偏移，单纯重复动作很难保持路线。我们保留抓取结束时的手臂姿态，调整下肢动作，并用标记距离和方位决定前进、纠偏及转弯。地图跟随部分则让我们进一步理解了目标位置、机器人定位和可通行路径并不是同一件事，需要把视觉观测、TF 和导航规划衔接起来。

第 2 组的 AR 实验说明，标签编号、实际尺寸和相机坐标约定虽然容易被忽略，却直接决定后车能否跟随。第 3 组的 RGB-D 方案进一步考虑了目标身份、深度有效性和局部环境；这部分仍需要更充分的实际样本和场景测试。第 10 组通过三车联调检查了前后车间距、圆弧转弯和顺序停车，同时发现围栏及长时间停留的前车会引发误避障，说明仅有距离阈值还不足以区分跟随对象与其他障碍。

各组的方法有不同的适用条件。AR 跟随实现直接，但受可见性影响较大；地图方法能够利用环境规划路径，却依赖地图、定位和传感器覆盖；RGB-D 方法减少了对固定标签的依赖，但身份保持和深度质量更难处理；队列方式便于扩展车辆，也会把前车转弯和停车的影响继续传给后车。

我们还需要继续完善抓取与放置确认、目标丢失后的恢复、任务状态联动、矮障碍检测及全队终点停车测试。现有局部试验不能代替一次完整比赛，因此后续应在相同场地重复运行，记录每台车的轨迹、接触、丢失和到达情况，再评价整体效果。

小组合作是本次实验的重要收获。各成员虽然有主要负责的方向，但动作、感知和执行问题往往需要一起判断；各组之间也需要交换前车速度、标签位置和故障现象。通过共同搭建、调试和整理报告，我们更清楚地认识到，多机系统的稳定运行来自各模块之间持续配合，而不只是某一个控制算法。

参考文献

[1] 课程实验手册. 第五–六章：基于视觉与 AI 定位的多机协同控制实验.

[2] 实验程序与小组资料. https://github.com/yky666/intelligent_robot_system.

[3] 第 1 组. TonyPi 动作组、标记识别与路线控制程序. humanoid/maze_route/.

[4] 第 1 组. 建图避障跟随程序与实验记录. turtlebot/follow2/.

[5] 第 2 组. 基于 AR 标记的 TurtleBot2 实时跟随实验报告与程序. turtlebot/follow1/.

[6] 第 3 组. RGB-D 实时检测与跟随方案. turtlebot/follow3/rgbd_following/.

[7] Grisetti G, Stachniss C, Burgard W. Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters. IEEE Transactions on Robotics, 2007, 23(1):34–46.

[8] Fox D, Burgard W, Thrun S. The Dynamic Window Approach to Collision Avoidance. IEEE Robotics and Automation Magazine, 1997, 4(1):23–33.

[9] 大组实验资料. 路线控制、地图导航、速度门控及参数配置文件.

[10] 第 10 组. TurtleBot2 视觉跟随与队列行驶实验报告、team_turtlebot_follow_plus 功能包.

[11] 第 1 组. 动作标定、赛道布置与多机调试记录.

A 复现入口与协作清单

A.1 源码入口索引

内容	仓库路径
最新人形路线	humanoid/maze_route/marker_route.py
检测与相机	humanoid/maze_route/markers.py, robust_camera.py
动作副本	humanoid/maze_route/actions/；生成规则见同目录mkcopies.py
通用反馈与道路	humanoid/scripts/competition.py, lane_control.py, lane_trial.py

内容	仓库路径
方式二入口	turtlebot/follow2/multi_robot_following/launch/vm_follow2.launch
目标与导航控制	该包scripts/marker_target.py 与map_follower.py
AR 控制与共享门控	turtlebot/follow1/ar_marker_following/scripts/marker_follower.py 与velocity_gate.py
轨迹与几何核心	新独立包turtlebot/follow2/competition_core/src/planning.py；旧包仍保留src/competition_core/planning.py，安装布局待核对
导航默认参数	该包config/move_base.yaml 及costmap_common.yaml

方式三入口为turtlebot/follow3/rgbd_following；方式四当前运行实现为turtlebot/follow4/team_turtlebot_follow_plus，含 C++ 控制节点和follow_ar.launch。方式四历史 README 中的地图位姿模式属于可选旧方案，不作为最新三车子系统的复现入口。

A.2 只查看最新标记检测

在具备相机、标定文件及显示环境的 TonyPi 上，从完整仓库运行：

```
cd ~/intelligent_robot_system
python3 humanoid/maze_route/marker_route.py --no-servos
```

--no-servos 不访问舵机串口并隐含 dry-run；单独--dry-run 仍可能设置头部，二者不能混淆。真实运动入口会关闭厂家上位机并停止相关服务，运行前核对同目录 README 与设备配置；本报告构建不会运行机器人控制脚本。

A.3 方式二现场启动索引

底盘、相机、扫描与地图请按docs/turtlebot_debug_commands.md 逐终端启动，避免重复驱动。最新结构还需同时部署方式一共享包和独立核心包，并解决正文记录的安装问题。完成迁移后构建、地图及 AMCL 初始位姿检查，主要跟随入口为：

```
source /opt/ros/kinetic/setup.bash
source ~/catkin_ws/devel/setup.bash
roslaunch multi_robot_following vm_follow2.launch \
  map_file:=$HOME/catkin_ws/maps/competition.yaml \
  marker_id:=1 marker_size_cm:=9.0
```

标记尺寸 9.0 cm 是该 launch 默认值，必须按实际 ALVAR 黑边尺寸替换；与人形航路 14 cm 自定义标记无关。正式跟随还需要持续任务状态和各项许可，单独启动此命令不会自动形成全队协同。

上述是入口索引，不是最新拆包后的实机成功记录；本次未在 ROS Kinetic 设备重新部署或运行这些命令。

A.4 报告提交前核对

1. 各组核对技术章节、实现入口、图片与实验记录；公共格式统一维护。
2. 补齐照片原件、地图/参数版本、每次试验记录与连续运行证据。
3. 将合成测试与实测结果分开，填入样本数、失败次数及实际验收结论。
4. 核对四种方式的独立性、任务状态适配及终点停靠定义。
5. 编译并检查 PDF 目录、公式、表格和图片；删除或补齐所有待填写项目。

B 关键实现摘录与可执行契约

B.1 最新路线的执行资源与首帧衔接

目标刻度、锁臂字典与停止事件作为输入；只在动作帧边界检查 stop_ev。首帧时长按姿态跳变延长，后续帧沿用数据库。

该函数更新的是已发送目标 pose，不是每帧编码器回传。首帧平滑不等于动力学稳定。放置前解除锁臂是调用方责任。

源码依据： humanoid/maze_route/marker_route.py，起始行 130。摘录中长行明确省略，不替代完整源码。

```
130 |     def play(self, name, lock, stop_ev):
131 |         with closing(sqlite3.connect('file:%s?mode=ro' % (ACTIONS / (name   ...
132 |             frames = db.execute('select * from ActionGroup order by "Index" ...
133 |         for k, f in enumerate(frames):
134 |             if stop_ev.is_set():
135 |                 log.warning('%s interrupted after frame %d/%d', name, k, le ...
136 |                 return False
137 |             target = {i: lock.get(i, f[1 + i]) for i in SERVOS}
138 |             ms = f[1]
139 |             if k == 0 and (self.pose is None or max(abs(target[i] - self.p  ...
140 |                 ms = max(ms, BRIDGE_MS)
141 |             if not self.fake:
142 |                 self.board.bus_servo_set_position(ms / 1000, [[i, target[i  ...
143 |             self.pose = target
144 |             time.sleep(ms / 1000)
145 |         return True
146 |
147 |
148 | # ------------------------------------------------------------- vision
149 | class Eyes:
150 |     """Camera thread: latest frame + marker detections, with a sequence num ...
151 |
```

```
152 |     def __init__(self):
153 |         import robust_camera
154 |         p = np.load(CALIB)
155 |         self.det = markers.MarkerDetector(p['mtx_array'], p['dist_array'])
156 |         self.cam = robust_camera.Camera()
157 |         self.cam.camera_open()
158 |         self.lock = threading.Lock()
159 |         self.frame, self.found, self.seq = None, {}, 0
```

B.2 新帧契约与多数可见

look 取得新 seq 后才能累计帧数，目标至少在多数帧出现。输入相机本身新鲜度由 robust_camera 保证，当前函数不伪造时间戳。

no-camera、None、dict 是三种不同返回类型。调用者先区分类型再访问距离，不能用 if obs 把 no-camera 字符串误当成有效观测。

源码依据： humanoid/maze_route/marker_route.py，起始行 180。摘录中长行明确省略，不替代完整源码。

```
180 |     def look(self, target, n=LOOK_FRAMES, timeout=2.0):
181 |         """Median measurement of `target` over n fresh frames, or None."""
182 |         _, _, seq0 = self.latest()
183 |         seen, frames, end = [], 0, time.monotonic() + timeout
184 |         while frames < n and time.monotonic() < end:
185 |             _, found, seq = self.latest()
186 |             if seq > seq0:
187 |                 seq0, frames = seq, frames + 1
188 |                 if target in found:
189 |                     seen.append(found[target])
190 |             time.sleep(0.01)
191 |         self.last_stats = (len(seen), frames)
192 |         if frames < n:
193 |             return 'no-camera'
194 |         if len(seen) * 2 <= n:          # need a majority of the frames
195 |             return None
196 |         return dict(dist_cm=float(np.median([m['dist_cm'] for m in seen])),
197 |                     bearing_deg=float(np.median([m['bearing_deg'] for m in  ...
198 |                     yaw_deg=float(np.median([m['yaw_deg'] for m in seen if  ...
199 |                     seen=len(seen), of=frames)
200 |
201 |     def close(self):
202 |         self.quit.set()
203 |         self.cam.camera_close()
204 |
205 |
```

B.3 路线初始化与阶段计数复位

等待启动事件，按可选 grab 动作建立锁臂，再遍历 ROUTE。每一段的目标、后续转向、步数和丢失记录独立初始化。

跳过 grab 仍读取 grab_final_v2 尾帧，因此需物理就位；不是允许任意姿态起跑。phase 只在转角成功返回后推进。

源码依据： humanoid/maze_route/marker_route.py，起始行 283。摘录中长行明确省略，不替代完整源码。

```
283 |     def _run(self):
284 |         a = self.a
285 |         if not self.wait_running():
```

```
286 |             return
287 |         if a.grab:
288 |             self.set_status('grabbing (grab_final_v2)')
289 |             if not self.act('grab_final_v2'):
290 |                 return
291 |         # hands as at the end of grab_final_v2: same values the action copi ...
292 |         with closing(sqlite3.connect('file:%s?mode=ro' % (ACTIONS / 'grab_  ...
293 |             last = db.execute('select * from ActionGroup order by "Index"   ...
294 |         self.lock_arms = {i: last[1 + i] for i in ARMS}
295 |         log.info('arms locked at %s', self.lock_arms)
296 |
297 |         for self.phase, (target, then) in enumerate(ROUTE):
298 |             steps = turns = misses = blind = 0
299 |             progress = []            # distance to the target before each w ...
300 |             seen_once = False
301 |             prev_walk_obs = None     # marker seen right before the last wa ...
302 |             last_seen = None         # latest measurement of the target
303 |             creep = 0.0              # predicted heading error while walkin ...
304 |             limit = a.turn_cm if then != 'stop' else a.stop_cm
305 |             self.set_status('phase %d: walking to marker %s (%s at %.0f cm  ...
306 |             while True:
307 |                 if self.stop_ev.is_set():
308 |                     return
309 |                 obs = self.eyes.look(target)
310 |                 if obs == 'no-camera':
311 |                     # The USB camera drops out and comes back by itself: stand
312 |                     # still and wait for it rather than walking blind.
313 |                     self.record(target, None, 'wait', 'no camera')
314 |                     t0 = time.monotonic()
315 |                     while self.eyes.look(target, n=1, timeout=1) == 'no-cam ...
316 |                         if self.stop_ev.is_set():
317 |                             return
318 |                         if int(time.monotonic() - t0) % 5 == 0:
319 |                             self.set_status('no camera frames for %.0f s;   ...
```

B.4 目标不可见、近距推断与真实故障

当前输入缺目标而最近观测足够近且居中时，构造 lost_close 并将距离截到 limit。它把一次失锁事件解释为可能抵近。

这不是新测距。图像遮挡也可能满足同样条件，必须记录 lost_close，不与正常观测到达混合统计。无相机新帧在此前另行处理。

源码依据： humanoid/maze_route/marker_route.py，起始行 331。摘录中长行明确省略，不替代完整源码。

```
331 |                 arrived_blind = (obs is None and last_seen is not None and
332 |                                  last_seen['dist_cm'] <= limit + a.lost_nea ...
333 |                                  abs(last_seen['bearing_deg']) <= max(a.cen ...
334 |                 if arrived_blind:
335 |                     log.info('%s lost at the goal (last seen %.0f cm, %+.0  ...
336 |                              target, last_seen['dist_cm'], last_seen['beari ...
337 |                     obs = dict(last_seen, dist_cm=min(last_seen['dist_cm']  ...
338 |                 if obs:
339 |                     seen_once, misses = True, 0
340 |                     if not obs.get('lost_close'):
341 |                         last_seen = obs
342 |                     if obs['dist_cm'] <= limit:
343 |                         if then == 'stop':
344 |                             self.record(target, obs, 'stop', 'reached')
345 |                             self.set_status('reached marker C at %.0f cm:   ...
346 |                             if a.release:
347 |                                 time.sleep(CORNER_PAUSE_S)
348 |                                 self.set_status('putting the cube down (put ...
349 |                                 self.lock_arms = {}          # put_down mo  ...
350 |                                 if not self.act('put_down'):
```

```
                                    return
                                self.record(target, None, 'put_down', 'done')
                                self.set_status('reached marker C, cube put ...
                            return
                        self.record(target, obs, 'corner', 'turn %s' % then)
                        if not self.corner(then, ROUTE[self.phase + 1][0],  ...
                            return
                        break
                    step = float(np.median(self.turn_steps[-5:]))
                    need = max(a.centre_deg, 0.6 * step)
                    if abs(obs['bearing_deg']) > need:
                        side = 'right' if obs['bearing_deg'] > 0 else 'left'
                        turns += 1
                        self.record(target, obs, 'turn_' + side,
                                    'centre %.1f deg (> %.1f, step ~%.0f)'  ...
                        if turns > a.max_centre_turns:
                            turns = 0
                            if not self.pause('too many centring turns for  ...
                                return
```

B.5 转角循环的可见性终止条件

按 side 固定方向逐次转动并观察 nxt。相机断流时等待，得到有效下一码观测即返回 True；次数耗尽进入 pause。

看到下一码不要求中心误差已经很小，下一段继续纠偏。暂停恢复后重新试一轮；无头失败会退出而不是永远等待无人可按的 S。

源码依据： humanoid/maze_route/marker_route.py，起始行 428。摘录中长行明确省略，不替代完整源码。

```
    def corner(self, side, nxt, at=None):
        """Keep turning (300 ms buffers) until the next marker is in view; only
        then walk on. Centring continues in the walk loop."""
        while True:
            for k in range(1, self.a.max_corner_turns + 1):
                self.set_status('corner: turn %s %d, looking for %s' % (sid ...
                time.sleep(CORNER_PAUSE_S)            # buffer before each  ...
                if not self.act('catch_ball_turn_' + side):
                    return False
                time.sleep(CORNER_PAUSE_S)            # and after it, befor ...
                obs = self.eyes.look(nxt)
                t0 = time.monotonic()
                while obs == 'no-camera':             # do not keep turning ...
                    if self.stop_ev.is_set():
                        return False
                    self.set_status('corner: no camera frames for %.0f s;   ...
                    if time.monotonic() - t0 > self.a.camera_wait:
                        break
                    obs = self.eyes.look(nxt)
                if isinstance(obs, dict):
                    self.record(nxt, obs, 'found', 'after %d turn(s)' % k)
                    return True
                self.record(nxt, None, 'turn_' + side, 'corner turn %d, %s  ...
            self.record(nxt, None, 'pause', 'next marker not found')
            if not self.pause('marker %s not found after %d turns' % (nxt,  ...
                return False      # resumed: keep turning and looking


# ------------------------------------------------------------- display
def overlay(img, found, target, status):
    h, w = img.shape[:2]
    markers.draw(img, found, target)
    cv2.line(img, (w // 2, 0), (w // 2, h), (255, 255, 255), 1)
    cv2.rectangle(img, (0, 0), (w, 24), (0, 0, 0), -1)
    cv2.putText(img, status[:80], (6, 17), cv2.FONT_HERSHEY_SIMPLEX, .5, (2 ...
```

```
463 |     return img
```

B.6 暂停事件的交互与无头差异

interactive=True 时清除 go 并等待恢复；否则设置 stop_ev 结束。wait_running 在等待 go 同时检查停止事件，避免退出死锁。

P 暂停只影响后续动作许可，已经发送的一帧或动作可能完成。Q 置 go 用于释放等待，不代表继续行走。

源码依据： humanoid/maze_route/marker_route.py，起始行 246。摘录中长行明确省略，不替代完整源码。

```
246 |     def pause(self, why):
247 |         """Window: wait for S. Headless: nobody can resume, so end the run."""
248 |         if self.interactive:
249 |             self.set_status(why + '; paused, press S to retry')
250 |             self.go.clear()
251 |             return self.wait_running()
252 |         self.set_status(why + '; headless, stopping')
253 |         self.stop_ev.set()
254 |         return False
255 |
256 |     def wait_running(self):
257 |         while not self.go.is_set():
258 |             if self.stop_ev.wait(0.1):
259 |                 return False
260 |         return not self.stop_ev.is_set()
261 |
262 |     def act(self, name, count=1):
263 |         for _ in range(count):
264 |             if not self.wait_running():
265 |                 return False
266 |             if self.a.dry_run:
267 |                 log.info('[dry-run] would play %s', name)
268 |                 time.sleep(0.5)
269 |             elif not self.servos.play(name, self.lock_arms, self.stop_ev):
270 |                 return False
271 |         time.sleep(SETTLE_S)
```

B.7 模板采样与黑边合法性

透视展开固定尺寸后 Otsu 二值化，只采每个格子的内部区域，输出 9 乘 9 数据和边框白格计数。

黑边和数据位误差是两类门限。角点顺序与透视矩阵必须一致，不能只比较 25 位内容而跳过外边框条件。

源码依据： humanoid/maze_route/markers.py，起始行 41。摘录中长行明确省略，不替代完整源码。

```
41 | def _read(gray, quad):
42 |     n = GRID * CELL_PX
43 |     M = cv2.getPerspectiveTransform(quad.astype(np.float32),
44 |                                     np.float32([[0, 0], [n, 0], [n, n], [0, ...
45 |     w = cv2.warpPerspective(gray, M, (n, n))
46 |     _, b = cv2.threshold(w, 0, 1, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
47 |     m = CELL_PX // 4
48 |     cells = np.array([[b[r*CELL_PX + m:(r+1)*CELL_PX - m, c*CELL_PX + m:(c  ...
49 |                        for c in range(GRID)] for r in range(GRID)], np.uint8)
50 |     ring = np.ones((GRID, GRID), bool)
```

```
    ring[BORDER:GRID - BORDER, BORDER:GRID - BORDER] = False
    return cells, int(cells[ring].sum())


class MarkerDetector:
```

B.8 候选筛选、模板匹配与重复检测

从灰度图、轮廓到凸四边形，检查尺度和拉伸，然后遍历模板四方向最少差异。重复同 ID 优先低错误、较大边长。

此处是局部摘录；候选几何与数据码匹配共同决定识别。程序不属于通用二维码识别库，替换标签字典需改变完整契约。

源码依据： humanoid/maze_route/markers.py，起始行 62。摘录中长行明确省略，不替代完整源码。

```
    def detect(self, frame):
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) if frame.ndim == 3 e ...
        g = cv2.GaussianBlur(gray, (3, 3), 0)
        bw = cv2.adaptiveThreshold(g, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2  ...
        contours, _ = cv2.findContours(bw, cv2.RETR_LIST, cv2.CHAIN_APPROX_ ...
        found = {}
        for c in contours:
            peri = cv2.arcLength(c, True)
            if peri < 4 * self.min_side:
                continue
            ap = cv2.approxPolyDP(c, 0.04 * peri, True)
            if len(ap) != 4 or not cv2.isContourConvex(ap):
                continue
            q = _order(ap.reshape(4, 2).astype(np.float32))
            sides = [np.linalg.norm(q[i] - q[(i + 1) % 4]) for i in range(4)]
            if min(sides) < self.min_side or max(sides) > 4 * min(sides):
                continue
            cells, border_white = _read(gray, q)
            if border_white > MAX_BORDER_WHITE:
                continue
            data = cells[BORDER:GRID - BORDER, BORDER:GRID - BORDER]
            best = None
            for mid, rots in _ROT.items():
                for r, t in enumerate(rots):
                    err = int((data != t).sum())
                    if best is None or err < best[0]:
                        best = (err, mid, r)
            err, mid, rot = best
            if err > MAX_DATA_ERR:
                continue
            m = self._measure(q)
            m.update(id=mid, err=err, rot=rot * 90, corners=q, side_px=flo  ...
            # several contours (inner/outer edge) can hit one marker: keep  ...
            if mid not in found or (err, -m['side_px']) < (found[mid]['err  ...
                found[mid] = m
        return found

    def _measure(self, q):
        ok, rvec, tvec = cv2.solvePnP(self.obj, q.reshape(-1, 1, 2), self.K ...
                                      flags=cv2.SOLVEPNP_IPPE_SQUARE)
        cx, cy = q.mean(0)
        # bearing from the undistorted image position of the marker centre
        u = cv2.undistortPoints(np.float32([[[cx, cy]]]), self.K, self.dist ...
        bearing = math.degrees(math.atan(u[0]))
        t = tvec.ravel()
        R, _ = cv2.Rodrigues(rvec)
        normal = R[:, 2]                        # marker normal (towards the  ...
```

B.9 PnP 输出的量纲与使用边界

已知四角、内参和畸变进入 IPPE 正方形 PnP；距离为 t 范数，方位由去畸变中心计算，yaw 由法向计算。

PnP 失败时 dist/yaw 可能为空；主线假设有效检测。斜距与水平距需同定义标定，不把图像方位误差当真实转角。

源码依据： humanoid/maze_route/markers.py，起始行 99。摘录中长行明确省略，不替代完整源码。

```
 99 |     def _measure(self, q):
100 |         ok, rvec, tvec = cv2.solvePnP(self.obj, q.reshape(-1, 1, 2), self.K ...
101 |                                       flags=cv2.SOLVEPNP_IPPE_SQUARE)
102 |         cx, cy = q.mean(0)
103 |         # bearing from the undistorted image position of the marker centre
104 |         u = cv2.undistortPoints(np.float32([[[cx, cy]]]), self.K, self.dist ...
105 |         bearing = math.degrees(math.atan(u[0]))
106 |         t = tvec.ravel()
107 |         R, _ = cv2.Rodrigues(rvec)
108 |         normal = R[:, 2]                        # marker normal (towards the  ...
109 |         # Robot heading against the wall the marker hangs on: 0 = square to it,
110 |         # + = robot turned RIGHT of square. Uses only the normal's horizont ...
111 |         # in camera terms so the head pitch does not inflate it.
112 |         yaw = -math.degrees(math.atan2(-normal[0], math.hypot(normal[1], n  ...
113 |         return dict(dist_cm=float(np.linalg.norm(t)) if ok else None, x_cm  ...
114 |                     bearing_deg=bearing, yaw_deg=yaw, centre=(float(cx), fl ...
115 |
116 |
117 | def draw(frame, found, target=None):
118 |     for mid, m in found.items():
119 |         colour = (0, 255, 0) if mid == target else (0, 200, 255)
120 |         q = m['corners'].astype(int)
121 |         cv2.polylines(frame, [q], True, colour, 3)
```

B.10 轨迹点接受、去噪与跳变拒绝

有限点进入轨迹，小于 spacing 不追加，超过 jump_limit 拒绝，长度超 max_length 删除最早点。默认 0.08/0.8/20 的单位均为米。

返回 True 可能仅表示小位移无需追加；调用者仍可能更新目标时效。拒绝跳变不会产生新的测量点，不能用插值掩盖定位突变。

源码依据： turtlebot/follow2/multi_robot_following/src/competition_core/planning.py，起始行 29。摘录中长行明确省略，不替代完整源码。

```
 29 |     def add(self, x, y):
 30 |         p = (float(x), float(y))
 31 |         if not all(finite(v) for v in p):
 32 |             return False
 33 |         if self.points:
 34 |             step = distance(self.points[-1], p)
 35 |             if step > self.jump_limit:
 36 |                 return False
 37 |             if step < self.spacing:
 38 |                 return True
 39 |         self.points.append(p)
 40 |         length = sum(distance(a, b) for a, b in zip(self.points, self.point ...
 41 |         while len(self.points) > 2 and length > self.max_length:
 42 |             length -= distance(self.points[0], self.points[1])
 43 |             self.points.pop(0)
 44 |         return True
```

```
45 |
46 |     def behind(self, gap):
47 |         if gap < 0:
48 |             raise ValueError('gap must be nonnegative')
49 |         remaining = gap
50 |         for i in range(len(self.points) - 1, 0, -1):
51 |             a, b = self.points[i - 1], self.points[i]
```

B.11 沿弧长回退与方向生成

从最新边向前遍历，落在一段内时按剩余弧长插值，并返回该段正向切线 atan2。历史不足返回 None。

该函数不是路径规划器，只生成目标点。move_base 仍可重规划；后车实际轨迹是否切角必须测量。

源码依据： turtlebot/follow2/multi_robot_following/src/competition_core/planning.py，起始行 46。摘录中长行明确省略，不替代完整源码。

```
46 |     def behind(self, gap):
47 |         if gap < 0:
48 |             raise ValueError('gap must be nonnegative')
49 |         remaining = gap
50 |         for i in range(len(self.points) - 1, 0, -1):
51 |             a, b = self.points[i - 1], self.points[i]
52 |             length = distance(a, b)
53 |             if length >= remaining and length > 0:
54 |                 ratio = remaining / length
55 |                 return (b[0] + ratio * (a[0] - b[0]),
56 |                         b[1] + ratio * (a[1] - b[1]),
57 |                         math.atan2(b[1] - a[1], b[0] - a[0]))
58 |             remaining -= length
59 |         return None  # do not drive toward a target before enough trail exists
60 |
61 |
62 | def grid_clear(data, width, height, resolution, origin_x, origin_y,
```

B.12 栅格足迹与未知区域拒绝

先验证地图结构，按原点旋转转换到网格，补偿半格对角线后检查足迹。未知、越界、高占据均阻塞。

输入是 OccupancyGrid 的-1 和 0 至 100，不能与内部 costmap 的 0 至 255 混用。目标可行不代表全路径连通。

源码依据： turtlebot/follow2/multi_robot_following/src/competition_core/planning.py，起始行 62。摘录中长行明确省略，不替代完整源码。

```
62 | def grid_clear(data, width, height, resolution, origin_x, origin_y,
63 |                origin_yaw, x, y, radius, threshold=65):
64 |     """Conservative circular footprint check including cell area, unknown = ...
65 |
66 |     OccupancyGrid values are 0..100 (not costmap's internal 0..255).
67 |     """
68 |     if resolution <= 0 or width <= 0 or height <= 0 or len(data) != width * ...
69 |         return False
70 |     dx, dy = x - origin_x, y - origin_y
71 |     c, s = math.cos(origin_yaw), math.sin(origin_yaw)
72 |     gx, gy = c * dx + s * dy, -s * dx + c * dy
```

```
    padding = radius + resolution * math.sqrt(2) / 2
    ix0, ix1 = int(math.floor((gx - padding) / resolution)), int(math.floo  ...
    iy0, iy1 = int(math.floor((gy - padding) / resolution)), int(math.floo  ...
    if ix0 < 0 or iy0 < 0 or ix1 >= width or iy1 >= height:
        return False
    for iy in range(iy0, iy1 + 1):
        for ix in range(ix0, ix1 + 1):
            cx, cy = (ix + 0.5) * resolution, (iy + 0.5) * resolution
            if math.hypot(cx - gx, cy - gy) <= padding:
                value = data[iy * width + ix]
                if value < 0 or value >= threshold:
                    return False
    return True


def localization_ok(covariance, xy_limit, yaw_limit):
    return len(covariance) == 36 and all(
        finite(covariance[i]) and 0 <= covariance[i] <= limit
        for i, limit in ((0, xy_limit), (7, xy_limit), (35, yaw_limit)))
```

B.13 地图跟随器的守卫执行顺序

以任务、目标、定位、地图和 TF 守卫过滤输入，再检查距离、轨迹和目标足迹；block 会撤销许可并取消导航。

每次只显示首个失败原因；并行故障应从原始遥测复核。ABORTED 等锁定与目标暂失的可恢复等待不同。

源码依据： turtlebot/follow2/multi_robot_following/scripts/map_follower.py，起始行 108。摘录中长行明确省略，不替代完整源码。

```
    def step(self):
        now = rospy.Time.now().to_sec()
        # Publisher emits this every cycle; velocity gate has a separate wa ...
        if self.finished or self.failed:
            return self.block('finished' if self.finished else 'navigation  ...
        if self.estop or self.mission not in ('carrying', 'releasing', 'do  ...
            return self.block('mission_stopped')
        if not fresh(self.target_stamp, now, self.target_timeout):
            return self.block('target_lost_hold_position')
        if not self.loc_good or not fresh(self.loc_stamp, now, 2.0):
            return self.block('localization_invalid')
        if self.grid is None or not fresh(self.grid_stamp, now, 3.0):
            return self.block('costmap_missing_or_stale')
        try:
            transform = self.buffer.lookup_transform(self.frame, self.base  ...
            if not fresh(transform.header.stamp.to_sec(), now, self.tf_time ...
                return self.block('base_tf_stale')
            t = transform.transform.translation
            robot = (t.x, t.y)
            if not all(finite(v) for v in robot):
                return self.block('base_tf_invalid')
        except (tf2_ros.LookupException, tf2_ros.ConnectivityException, tf  ...
            return self.block('base_tf_missing')
        if distance(robot, self.target) < self.min_gap:
            return self.block('leader_too_close')
        point = self.trail.behind(self.gap)
        if point is None:
            return self.block('waiting_for_leader_trail')
        if self.mission == 'done' and distance(robot, point) <= self.finish ...
            self.finished = True
            return self.block('finished')
```

```
139 |         grid = self.grid
140 |         origin = grid.info.origin
141 |         if grid.header.frame_id.lstrip('/') != self.frame.lstrip('/') or no ...
142 |                 grid.data, grid.info.width, grid.info.height, grid.info.res ...
143 |                 origin.position.x, origin.position.y, yaw(origin.orientation),
144 |                 point[0], point[1], self.radius):
145 |             return self.block('follow_goal_blocked')
146 |         # Do not send repeated goals near the current goal (avoids preempti ...
147 |         state = self.client.get_state() if self.goal is not None else None
148 |         if state in (4, 5, 9):  # ABORTED, REJECTED, LOST
149 |             self.failed = True
150 |             return self.block('navigation_failed_reset_required')
151 |         if not self.client.wait_for_server(rospy.Duration(0.01)):
152 |             return self.block('move_base_unavailable')
153 |         if self.goal is None or (now - self.goal_time >= self.goal_interva  ...
154 |             goal = MoveBaseGoal()
```

B.14 目标更新与导航失败状态

首次目标立即发，之后默认间隔至少 1 秒且位移至少 0.25 米。导航状态 4、5、9 锁定失败。

普通目标更新阈值可能遗漏终点小变化；不能从 RGB-D 版本的 need_goal 代码推断地图版本也有相同最终目标修正。

源码依据： turtlebot/follow2/multi_robot_following/scripts/map_follower.py，起始行 146。摘录中长行明确省略，不替代完整源码。

```
146 |         # Do not send repeated goals near the current goal (avoids preempti ...
147 |         state = self.client.get_state() if self.goal is not None else None
148 |         if state in (4, 5, 9):  # ABORTED, REJECTED, LOST
149 |             self.failed = True
150 |             return self.block('navigation_failed_reset_required')
151 |         if not self.client.wait_for_server(rospy.Duration(0.01)):
152 |             return self.block('move_base_unavailable')
153 |         if self.goal is None or (now - self.goal_time >= self.goal_interva  ...
154 |             goal = MoveBaseGoal()
155 |             goal.target_pose.header.frame_id = self.frame
156 |             goal.target_pose.header.stamp = rospy.Time.now()
157 |             goal.target_pose.pose.position.x = point[0]
158 |             goal.target_pose.pose.position.y = point[1]
159 |             goal.target_pose.pose.orientation.z = math.sin(point[2] / 2)
160 |             goal.target_pose.pose.orientation.w = math.cos(point[2] / 2)
161 |             self.client.send_goal(goal)
162 |             self.goal = point
163 |             self.goal_time = now
164 |         self.allowed.publish(True)
165 |         self.status.publish('following')
166 |
167 |
168 | if __name__ == '__main__':
169 |     rospy.init_node('map_follower')
170 |     Follower()
171 |     rospy.spin()
```

B.15 独立门控与真实扫描有效性

命令、许可、扫描过期或急停即返回零 Twist；有限非负线速度才继续检查，转向时扇区扩大。

墙钟线程持续发布而不依赖 ROS 时钟前进。其他直接发布者会绕过门控，系统必须确保唯一导航速度源。

源码依据： turtlebot/follow2/multi_robot_following/scripts/velocity_gate.py，起始行 58。摘录中长行明确省略，不替代完整源码。

```
58 |     def output(self):
59 |         now = self.clock()
60 |         out = Twist()
61 |         v, w = self.command.linear.x, self.command.angular.z
62 |         if self.estop or not self.allowed or not fresh(self.allowed_stamp,  ...
63 |             return out
64 |         if not finite(v) or not finite(w) or v < 0:  # no blind reverse wi  ...
65 |             return out
66 |         if self.require_scan:
67 |             scan = self.scan
68 |             # Wider sector during rotation; the full footprint stays in cos ...
69 |             sector = math.pi / 2 if abs(w) > 0.05 else math.pi / 4
70 |             if scan is None or not fresh(self.scan_stamp, now, 0.5) or not  ...
71 |                     scan.ranges, scan.angle_min, scan.angle_increment, scan ...
72 |                     scan.range_max, self.clearance, sector):
73 |                 return out
74 |         out.linear.x = clamp(v, self.max_v)
75 |         out.angular.z = clamp(w, self.max_w)
76 |         return out
77 |
78 |     def run(self):
79 |         while not rospy.is_shutdown():
80 |             with self.lock:
81 |                 self.pub.publish(self.output())
82 |             time.sleep(0.05)
83 |
84 |
85 | if __name__ == '__main__':
86 |     rospy.init_node('velocity_gate')
87 |     Gate()
88 |     rospy.spin()
```

B.16 任务桥的会话与序号契约

桥接 UDP 状态，检查来源 IP、session、run_id、sequence 和状态白名单，超过 1 秒无有效数据发布 stopped。

该摘录不意味着最新 marker_route 已发送心跳。适配器仍是集成缺口；测试终端 carrying 是测试输入而非自主协同证据。

源码依据： turtlebot/follow2/multi_robot_following/scripts/mission_bridge.py，起始行 16。摘录中长行明确省略，不替代完整源码。

```
16 | def main():
17 |     rospy.init_node('mission_bridge')
18 |     pub = rospy.Publisher('/competition/mission', String, queue_size=1)
19 |     address = rospy.get_param('~bind', '0.0.0.0')
20 |     port = rospy.get_param('~port', 15000)
21 |     allowed_ip = rospy.get_param('~sender_ip', '')
22 |     session = rospy.get_param('~session', 'team1')
23 |     clock = getattr(time, 'monotonic', time.time)
24 |     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
25 |     sock.bind((address, port))
26 |     sock.settimeout(0.1)
27 |     last = None
28 |     state = 'idle'
29 |     run_id = None
```

```
    sequence = -1
    valid = ('idle', 'searching', 'grasping', 'verifying_grasp', 'grasp_suc ...
             'grasp_failed', 'retrying', 'carrying', 'checking_hold',
             'checking_placement', 'releasing', 'verifying_release', 'done  ...
    try:
        while not rospy.is_shutdown():
            try:
                raw, sender = sock.recvfrom(2048)
                data = json.loads(raw.decode('utf-8'))
                if not allowed_ip or sender[0] != allowed_ip or data.get('  ...
                    continue
                new_run = data.get('run_id')
                seq = data.get('sequence')
                if not isinstance(new_run, string_types) or not isinstance( ...
                    continue
                # A new run is accepted only after the previous run timed out.
                if run_id is not None and run_id != new_run and last is no  ...
                    continue
                if new_run != run_id:
                    run_id, sequence = new_run, -1
                if seq <= sequence or data.get('state') not in valid:
                    continue
                state, last, sequence = data['state'], clock(), seq
            except socket.timeout:
                pass
            except (ValueError, TypeError, UnicodeError, AttributeError):
                pass
            pub.publish(state if last is not None and 0 <= clock() - last   ...
    finally:
        pub.publish('stopped')
        sock.close()


if __name__ == '__main__':
```

C 实际配置快照与参数复现

C.1 导航与 DWA 配置

这些参数进入 ROS 导航栈，控制频率、采样和动力学范围共同决定短时可行轨迹。最大速度是指令上限，不是实测匀速；加速度限制是规划假设，不等于满载实际制动下界。Kinetic 版本的字段名应与部署发行版匹配。

recovery_behavior_enabled 与 clearing_rotation_allowed 关闭，使导航失败不会后台原地旋转。代价是部分卡滞需要排故重启。改参数后应保留旧配置和同场对照，记录控制时延、规划失败与障碍余量。

源码依据： turtlebot/follow2/multi_robot_following/config/move_base.yaml；以下为调试配置示例，现场参数需结合实际设备调整。

```
base_local_planner: dwa_local_planner/DWAPlannerROS
controller_frequency: 10.0
planner_frequency: 1.0
planner_patience: 3.0
controller_patience: 3.0
oscillation_timeout: 8.0
oscillation_distance: 0.1
recovery_behavior_enabled: false
clearing_rotation_allowed: false
shutdown_costmaps: false
```

```
DWAPlannerROS:
  max_vel_x: 0.15
  min_vel_x: 0.0
  max_vel_y: 0.0
  min_vel_y: 0.0
  max_vel_trans: 0.15
  min_vel_trans: 0.03
  max_vel_theta: 0.5
  min_vel_theta: 0.1
  acc_lim_x: 0.3
  acc_lim_y: 0.0
  acc_lim_theta: 0.6
  xy_goal_tolerance: 0.15
  yaw_goal_tolerance: 0.4
  latch_xy_goal_tolerance: true
  sim_time: 1.5
  vx_samples: 10
  vy_samples: 1
  vth_samples: 20
  path_distance_bias: 32.0
  goal_distance_bias: 24.0
  occdist_scale: 0.05
  stop_time_buffer: 0.3
  publish_traj_pc: true
  publish_cost_grid_pc: true

```

C.2 固定地图与扫描代价层

静态地图负责场地围挡，实时扫描负责补充动态占据。robot_radius 与 inflation_radius 对应不同几何作用；不能只为让窄道规划成功而随意缩小足迹，必须测量相机架、底盘与线缆外包络。

扫描来源可能是 Kinect 深度转扫描，而不是独立激光雷达。此配置不证明低障碍已被传感器覆盖，必须逐层检查深度和 scan。raytrace 清除和障碍保持也要在障碍移入、移出场景验证。

源码依据： turtlebot/follow2/multi_robot_following/config/costmap_common.yaml；以下为调试配置示例，现场参数需结合实际设备调整。

```
robot_radius: 0.23
footprint_padding: 0.03
transform_tolerance: 0.5
obstacle_range: 3.0
raytrace_range: 3.5
observation_sources: laser
laser:
  data_type: LaserScan
  topic: scan
  marking: true
  clearing: true
  obstacle_range: 3.0
  raytrace_range: 3.5
  observation_persistence: 0.0
  expected_update_rate: 0.5
  min_obstacle_height: 0.0
  max_obstacle_height: 2.0
inflation_radius: 0.60
cost_scaling_factor: 3.0
always_send_full_costmap: true

```

C.3 地图导航启动与坐标约定

launch 声明 robot_ns、map_file、base_frame、odom_frame 与 scan 等接口，实际运行前核对课程驱动是否仍发布全局话题。公共 map 不代表所有 frame 都应相同，各车局部框架需独立。

以下保留启动文件的主要结构。若原文件超过摘录行数，完整文件随 code_snapshot 提供。报告中主机 IP 和设备路径应在本机参数中配置，不将登录凭据放入文稿。

源码依据： turtlebot/follow2/multi_robot_following/launch/navigation.launch；以下为调试配置示例，现场参数需结合实际设备调整。

```
<launch>
  <!-- Robot drivers must already provide ns/odom -> ns/base_footprint ...
  <arg name="robot_ns" default="tb2" />
  <arg name="map_file" />
  <arg name="map_frame" default="map" />
  <arg name="base_frame" default="$(arg robot_ns)/base_footprint" />
  <arg name="odom_frame" default="$(arg robot_ns)/odom" />
  <arg name="scan_topic" default="/$(arg robot_ns)/scan" />
  <group ns="$(eval arg('robot_ns') or '/')">
    <node pkg="map_server" type="map_server" name="map_server" args="$ ...
      <param name="frame_id" value="$(arg map_frame)" />
    </node>
    <node pkg="amcl" type="amcl" name="amcl" output="screen">
      <param name="global_frame_id" value="$(arg map_frame)" />
      <param name="odom_frame_id" value="$(arg odom_frame)" />
      <param name="base_frame_id" value="$(arg base_frame)" />
      <param name="use_map_topic" value="true" />
      <param name="odom_model_type" value="diff" />
      <param name="laser_model_type" value="likelihood_field" />
      <param name="min_particles" value="500" />
      <param name="max_particles" value="2000" />
      <param name="resample_interval" value="1" />
      <param name="update_min_d" value="0.05" />
      <param name="update_min_a" value="0.05" />
      <param name="transform_tolerance" value="0.3" />
      <remap from="scan" to="$(arg scan_topic)" />
    </node>
    <node pkg="multi_robot_following" type="localization_refresh.py" n ...
    <node pkg="move_base" type="move_base" name="move_base" output="sc ...
      <rosparam command="load" file="$(find multi_robot_following)/con ...
      <rosparam command="load" file="$(find multi_robot_following)/con ...
      <rosparam command="load" file="$(find multi_robot_following)/con ...
      <param name="global_costmap/global_frame" value="$(arg map_frame)" />
      <param name="global_costmap/robot_base_frame" value="$(arg base_ ...
      <param name="global_costmap/static_map" value="true" />
      <param name="global_costmap/update_frequency" value="3.0" />
      <param name="global_costmap/publish_frequency" value="2.0" />
      <param name="global_costmap/laser/topic" value="$(arg scan_topic)" />
      <param name="local_costmap/global_frame" value="$(arg odom_frame)" />
      <param name="local_costmap/robot_base_frame" value="$(arg base_f ...
      <param name="local_costmap/rolling_window" value="true" />
      <param name="local_costmap/static_map" value="false" />
      <param name="local_costmap/width" value="4.0" />
      <param name="local_costmap/height" value="4.0" />
      <param name="local_costmap/resolution" value="0.05" />
```

C.4 跟随器目标源与速度重映射

该入口连接目标适配、地图跟随器与速度门控，目标源只能选 marker 或 pose 之一。motion_allowed 和 raw/safe 速度重映射需检查实际连接，确保导航器不能绕过最

终门控直达底盘。

follow_distance、minimum_distance 和 footprint_radius 自洽仅是必要条件。ALVAR 目标尺寸与 marker_id 要与实物匹配；调用 pose 模式则要共享地图原点和前车有效定位。

源码依据： 地图跟随启动配置。下面列出调试时使用的示例，其中速度门控由对应的 AR 功能包提供，使用时需检查节点包名。

```
<launch>
  <arg name="robot_ns" default="tb2" />
  <arg name="map_frame" default="map" />
  <arg name="base_frame" default="$(arg robot_ns)/base_footprint" />
  <!-- marker follows TonyPi; pose follows a predecessor already local ...
  <arg name="target_source" default="marker" />
  <arg name="marker_id" default="1" />
  <arg name="leader_pose_topic" default="/tb1/amcl_pose" />
  <arg name="marker_topic" default="/$(arg robot_ns)/ar_pose_marker" />
  <arg name="follow_distance" default="0.9" />
  <arg name="scan_topic" default="/$(arg robot_ns)/scan" />
  <arg name="cmd_vel_topic" default="/$(arg robot_ns)/cmd_vel_mux/inpu ...
  <group ns="$(eval arg('robot_ns') or '/')">
    <node if="$(eval arg('target_source') == 'marker')" pkg="multi_rob ...
      <param name="marker_id" value="$(arg marker_id)" />
      <param name="map_frame" value="$(arg map_frame)" />
      <remap from="ar_pose_marker" to="$(arg marker_topic)" />
    </node>
    <node if="$(eval arg('target_source') == 'pose')" pkg="multi_robot ...
      <param name="map_frame" value="$(arg map_frame)" />
      <remap from="leader_pose" to="$(arg leader_pose_topic)" />
    </node>
    <node pkg="multi_robot_following" type="map_follower.py" name="map ...
      <param name="map_frame" value="$(arg map_frame)" />
      <param name="base_frame" value="$(arg base_frame)" />
      <param name="follow_distance" value="$(arg follow_distance)" />
    </node>
    <node pkg="multi_robot_following" type="velocity_gate.py" name="ve ...
      <remap from="scan" to="$(arg scan_topic)" />
      <remap from="cmd_vel_safe" to="$(arg cmd_vel_topic)" />
    </node>
  </group>
</launch>

```

C.5 AR 检测入口与相机参数

ALVAR 检测依赖图像、CameraInfo、optical_frame 与厘米制标签尺寸。图像分辨率改变时内参需同步，不能把普通 QR 图案替代 ALVAR 标签而只改 marker_id。

尺寸误差会直接变为位姿尺度误差。相机采样时刻的 TF 需有效，检测框截图不证明整段目标持续可见。正式现场记录应包含打印尺寸、安装高度和标签朝向。

源码依据： `turtlebot/follow2/multi_robot_following/launch/ar_tracker.launch`；以下为调试配置示例，现场参数需结合实际设备调整。

```
<launch>
  <arg name="robot_ns" default="tb2" />
  <arg name="marker_size_cm" default="9.0" />
  <arg name="image_topic" default="/$(arg robot_ns)/camera/rgb/image_r ...
  <arg name="info_topic" default="/$(arg robot_ns)/camera/rgb/camera_i ...
  <arg name="optical_frame" default="$(arg robot_ns)/camera_rgb_optica ...
  <group ns="$(eval arg('robot_ns') or '/')">
```

```
    <node pkg="ar_track_alvar" type="individualMarkersNoKinect" name=" ...
      <param name="marker_size" value="$(arg marker_size_cm)" type="do ...
      <param name="max_new_marker_error" value="0.08" type="double" />
      <param name="max_track_error" value="0.2" type="double" />
      <param name="output_frame" value="$(arg optical_frame)" />
      <remap from="camera_image" to="$(arg image_topic)" />
      <remap from="camera_info" to="$(arg info_topic)" />
      <remap from="ar_pose_marker" to="ar_pose_marker" />
    </node>
  </group>
</launch>
```

C.6 任务桥的独立运行入口

每个 ROS master 只运行一个任务桥，配置 sender_ip、session 和端口。收到合法心跳才允许跟随任务进入对应状态，旧会话序号和错误来源不刷新有效性。

这份入口提供接口，不自动使最新人形程序具备发送端。适配器须单独实现并验收，测试终端持续发布 carrying 只用于隔离导航测试，不能充当自动协同结果。

源码依据： turtlebot/follow2/multi_robot_following/launch/mission_bridge.launch；以下为调试配置示例，现场参数需结合实际设备调整。

```
<launch>
  <arg name="sender_ip" />
  <arg name="session" default="team1" />
  <arg name="port" default="15000" />
  <node pkg="multi_robot_following" type="mission_bridge.py" name="mis ...
    <param name="sender_ip" value="$(arg sender_ip)" />
    <param name="session" value="$(arg session)" />
    <param name="port" value="$(arg port)" />
  </node>
</launch>
```

C.7 当前抱物动作的数据库统计

以下统计由 SQLite 只读查询生成，帧时长累加不含稳态等待、相机观测与保护暂停。动作文件帧数和锁臂刻度属于客观文件证据，不代表实际关节跟踪误差或物体稳定性。当前动作映射与旧 grab 版本不同，比较时必须固定文件版本。

动作	帧数	名义时长 (s)
grab_final_v2	7	9.5
catch_ball_go	10	0.95
catch_ball_turn_left	6	1
catch_ball_turn_right	6	1

表 22: 最新 maze_route/actions 动作文件统计。

六个手臂通道在抓取尾帧的值按 ID 6、7、8、14、15、16 排列。代码副本生成只替

换指定通道，腿部与时序应保留；首帧桥接又会改变真实执行总时长。文件统计和运行日志应相互核对，不能从名义播放时间估算完整路线耗时。

源码依据： data/source_analysis.json；maze_route/actions/*.d6a。

C.8 证据类型与复现结果的对应

图文材料必须先区分来源，再解释结论。真实动作矩阵来自 d6a 数据库，可复核帧数、时长和命令；模板图来自常量，可复核图案和编码距离；轨迹例子调用无 ROS 核心函数，可复核给定点列的插值；控制门限和通道余量属于解析关系。上述材料均不提供完整实机成功率。

表 23: 实验图表所用数据与用途。

资料类型	可复现输入	结论范围
状态转换图	marker_route 控制流	阶段、守卫、失败出口与模式差异
标记模板/位差	markers.py 常量	字典结构和旋转分离，不是实机混淆矩阵
动作命令热图	当前 SQLite 动作文件	命令刻度和帧结构，不是关节实测轨迹
Trail 返回点	给定折线和真实核心函数	弧长目标行为，不是底盘执行路径
门限/通道曲线	源码表达式与给定假设	参数设计含义，不是最优调参结论
现场照片/日志	本组原图、真实运行文件	对应试验的物理过程，必须独立归档

我们保留了画图程序和参数表，便于检查图中的数值。后续调整动作、标签尺寸或跟随参数时，应同时记录试验条件；不同条件下的数据分开统计，才能看出调整是否有效。

D 人形机器人感知、状态机与动作补充分析

D.1 分层状态机与任务转换

D.1.1 状态机的层次建模与实际控制入口

最新路线程序没有用一个枚举变量表达所有状态，而是由 Route 线程、phase 索引、循环分支及 go/stop_ev 事件共同实现。报告将其抽象为三层：任务层管理 A/B/C 目标次序，单段层根据观测选择对中、直行或等待，执行层串行播放动作帧。图中的状态名称用于解释控制流，不意味着源码存在同名类。状态机描述因此必须同时交代进入条件、动作和退出事件。

任务层的 ROUTE 列表为 A-left、B-right、C-stop。每进入一段，steps、turns、misses、blind、progress、last_seen 等局部记录重置，避免上一段观测影响下一目标。corner 成功返回后才推进 phase；阶段切换不是仅靠固定时间或预设转向次数。抓取播放结束建立锁臂值，但最新入口没有同时检查颜色持物证据，不能把动作结束写成传感器已确认抓住。



图 12: 最新 marker_route 任务主状态转换。到达条件来自距离或近距丢码启发式，转角成功要求下一目标可见。

主状态图表达的是阶段顺序，而每个“走向”状态内部还包含反复观测与动作选择。阶段切换有两个不同守卫：抵近当前目标触发固定方向的转角搜索；确认下一目标可见后才推进路线索引。这样的划分使转角失败可以保留当前阶段，避免一次丢码就跳过目标。

源码依据： humanoid/maze_route/marker_route.py：Route._run、corner、ROUTE。

D.1.2 初始化、启动和锁臂状态

启动先建立日志目录，解析参数及图形模式，再构造 Servos、固定头部、打开 Eyes、启动 Route 线程。真实舵机模式尝试关闭占用串口的厂家程序并停止服务；这是资源准备，不是机器人路线的一部分。交互模式等待 S 设置 go，无头模式在 `start_delay` 后启动。no-grab 跳过初始抱起，但仍从 `grab_final_v2` 尾帧读取锁臂值，因此假定物块和机械姿态已经就位。

六个身体通道固定为抓取末帧，其他通道由步态决定。锁臂约束保护命令中的抱持姿态，不测量夹持力。若末帧来源与实际抓取动作不一致，即使下肢轨迹不变，也可能在起步时改变手臂。最新程序用自己的 actions 副本，不应混入旧 grab 文件的尾帧数值。

$$\mathcal{A} = \{6, 7, 8, 14, 15, 16\}, \qquad p_{k,i} = \begin{cases} p^{\text{grab}}_{\text{last},i}, & i \in \mathcal{A}, \\ p^{\text{gait}}_{k,i}, & i \notin \mathcal{A}. \end{cases} \tag{16}$$

运行开关的含义及其任务前提汇总如下，目的是区分离线检查、已抱物起步和完整搬运流程。

模式	实际含义
dry-run	Route.act 不走转，但仍可能打开串口并设置头部
no-servos	不触碰舵机串口，隐含 dry-run
no-grab	已抱物起步假设，仍读取抓取尾帧用于锁臂
no-release	到 C 停止但不放置，不符合完整放置任务

表 24: 运行模式与任务前提。

这些开关改变的是执行条件和任务完整性，并非同一功能的等价测试方式。特别是禁用舵机的离线检查不能验证抓取可靠性，而跳过放置的运行也不能计为完整搬运成功。

源码依据： `marker_route.py`：main、Servos、Route.act。

D.1.3 三阶段目标与距离守卫

A/B/C 既是模板 ID，也是路线语义。A 是第一段末端，抵近后左转寻找 B；B 是第二段末端，抵近后右转寻找 C；C 是终点，抵近后停止与放置。程序只统计当前 target 的观测，其他码即使同时出现，也不自动跳过当前阶段。距离阈值在前两段使用 `turn_cm`，在末段使用 `stop_cm`，默认都是 40 cm。

这里的距离是相机到码中心的三维斜距，不是机器人几何中心到围挡的地面净距。码的高度、相机偏置、脚步余程和身体外包络都会影响安全停车位置。报告应记录码安装与实际摆位，不将“阈值 40 cm”解释为所有机体部分与障碍都有 40 cm 余量。看到

下一码是转角结束条件，但不能证明精确完成 90 度；后续对中仍会改变朝向。

$$(A, \text{left}) \to (B, \text{right}) \to (C, \text{stop}), \qquad d_{A,B} \le d_{\text{turn}}, \quad d_C \le d_{\text{stop}}. \tag{17}$$

源码依据： `marker_route.py`：ROUTE、limit、phase 与 corner 调用。

D.2 固定标记识别与视觉测距

D.2.1 自定义标记的编码与透视解码

标记由 9 乘 9 格组成，外两格为黑边，中间 5 乘 5 数据。markers.py 保存三张 25 位模板，并在四个旋转方向寻找最少数据位差异；最多允许 1 格数据错误，黑边最多允许 3 格读白。图13直接读取实际模板常量，不是新生成的替代码，也不是标准 ALVAR、ArUco 或 AprilTag 字典。

检测先对灰度图高斯滤波、自适应二值化并寻找轮廓，要求凸四边形和足够的最短边像素；过度拉伸的候选被排除。透视展开到固定网格后，采样每格内部，降低边缘插值和印刷边界影响，再按 Otsu 阈值分类黑白。若内外轮廓重复命中同一码，优先采用数据错误更少、边长更大的候选。识别依赖真实黑边完整、像素大小和成像质量，允许一位误读并不等于对大面积遮挡鲁棒。



图 13: 源码中实际 A/B/C 数据模板及其路线含义，黑色正方形边长设计值为 14 cm。

三种图案把视觉身份绑定为不同路线语义，识别出 A、B、C 后仍需结合当前阶段决定是否采用。黑边与内部数据共同参与解码，打印尺寸则进入后续测距模型。模板图主要用于核对目标设计，识别容错还要通过旋转后的数据位距离分析。

源码依据： `humanoid/maze_route/markers.py`：TEMPLATES、_read、MarkerDetector.detect。

D.2.2 模板旋转分离度与尺寸一致性

设数据位串为 a 和 b，Hamming 距离统计不同格子的数量。四旋转中的最小差异反映有限字典的分离程度；对角线为零，非对角项应足够大，才有识别容错余量。图中

的矩阵由真实模板直接计算，不能当作实机识别混淆矩阵，也不提供误识别概率。

$$d_{\mathrm{H}}(a,b)=\sum_{j=1}^{25}\mathbb{1}(a_j\neq b_j),\qquad d^{*}_{ab}=\min_{r\in\{0,1,2,3\}} d_{\mathrm{H}}(a,\mathrm{rot}_{90r}(b)). \tag{18}$$

模板旋转 rot 与相机解算的目标方位、墙面朝向是不同输出，rot=90 不能解释成机器人转了 90 度。增大打印尺寸能提高远端像素支持，却同时改变近端出画面条件。若实际边长变了而 `SIZE_CM` 仍为 14，PnP 将产生尺度偏差；新增模板也必须重新检查所有旋转的距离，而不仅观察正立图案是否不同。

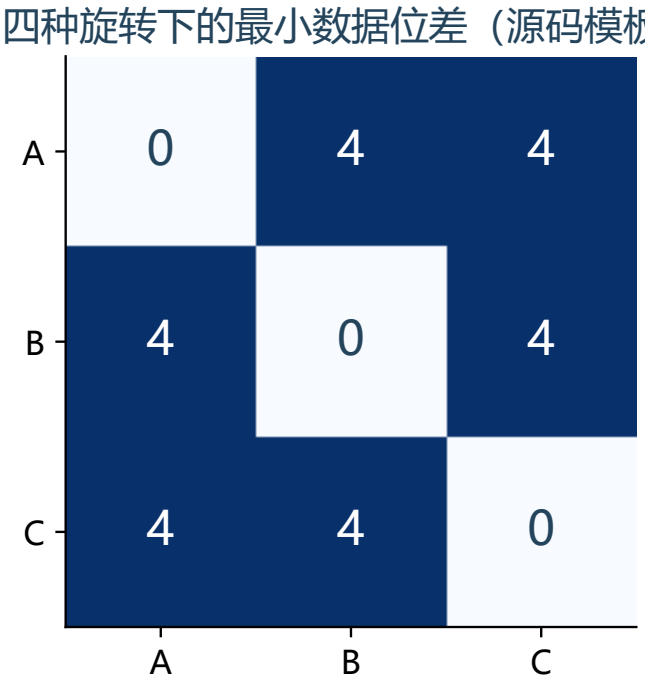


图 14: 实际模板在四旋转下的最小数据位差，不是性能测试。

矩阵应按非对角元素解读：数值越大，两个不同模板在最相似旋转下仍有越多格不同。该结构性质支持设置有限的数据位容错，但未包含模糊、反光和角点提取误差，因此不能直接推算现场识别成功率。后续距离估计仍依赖几何标定的一致性。

源码依据： `markers.py`：SIZE_CM、_ROT、MAX_DATA_ERR。

D.2.3 PnP 距离、方位和墙面姿态

检测器用已知正方形四角、设备标定内参和畸变参数，通过 `SOLVEPNP_IPPE_SQUARE` 求旋转和平移。`dist_cm` 取平移向量范数；`bearing_deg` 由去畸变后的标记中心横坐标求反正切；墙面 yaw 则由标记法向推导。三者量纲和含义不同，路线对中主要使用 bearing，不能因为模块输出 yaw 就宣称采用了完整位姿控制。

$$\lambda\tilde{u}=K[R\mid t]\tilde{X},\quad d=\|t\|_2,\quad \beta=\arctan(u_n),\quad d=\sqrt{d_h^2+\Delta h^2}. \tag{19}$$

码与镜头存在高度差时，斜距大于水平距离。距离误差还受打印尺寸、角点、标定、运动模糊和观察角影响。下图右侧按水平 40 cm 绘制几何关系，不是设备测距精度。正式标定要用同一定义的真实斜距，或同时记录水平距离和高度差，避免把量纲不同的距离直接计算 RMSE。

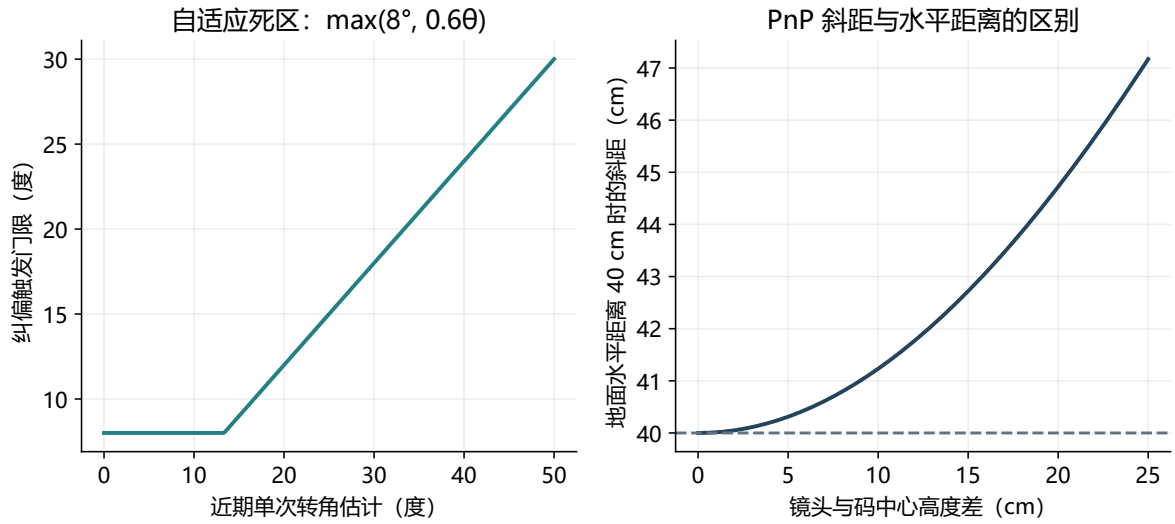


图 15: 纠偏门限解析曲线与斜距几何。两者为公式分析，非实测性能曲线。

左图在近期转角估计较小时保留固定死区，超过交点后门限随估计线性增加；右图则表明，相同水平间距在高度差增大时产生更大的视觉斜距。两项分析分别影响是否纠偏和何时停车，调参时必须保持二者的物理含义独立，不能用调整纠偏门限补偿测距尺度误差。

源码依据： `markers.py`：MarkerDetector._measure；`marker_route.py`：CALIB。

D.3 单段决策、纠偏与转角控制

D.3.1 新帧、多数可见与单段观测状态

Eyes 线程持续更新 frame、found 和 seq。look 开始时记录 seq0，只接受序号增加的新帧，默认观察 3 帧。凑不齐新帧返回 no-camera；取得 3 帧但目标仅出现 0 或 1 帧返回 None；至少 2 帧看见才对距离、方位分别取中位数。seen 字段是可见帧数与新帧数之比，不是神经网络置信度。

中位数减少单帧抖动，却不能校正系统性标定错误。序号增长避免缓存图重复累计，但图像是否真正近期仍依赖 `robust_camera` 的取流契约。动作结束后还有 0.25 s 稳态等待，再开始新的 look，防止直接用大幅身体运动期间画面作下一步判断。

有新帧但无目标与相机断流进入不同分支。相机断流原地等待；目标无效则可能受限盲行和补偿。图16保留这个区别，避免把“所有失锁均停车”写成当前源码已实现行为。

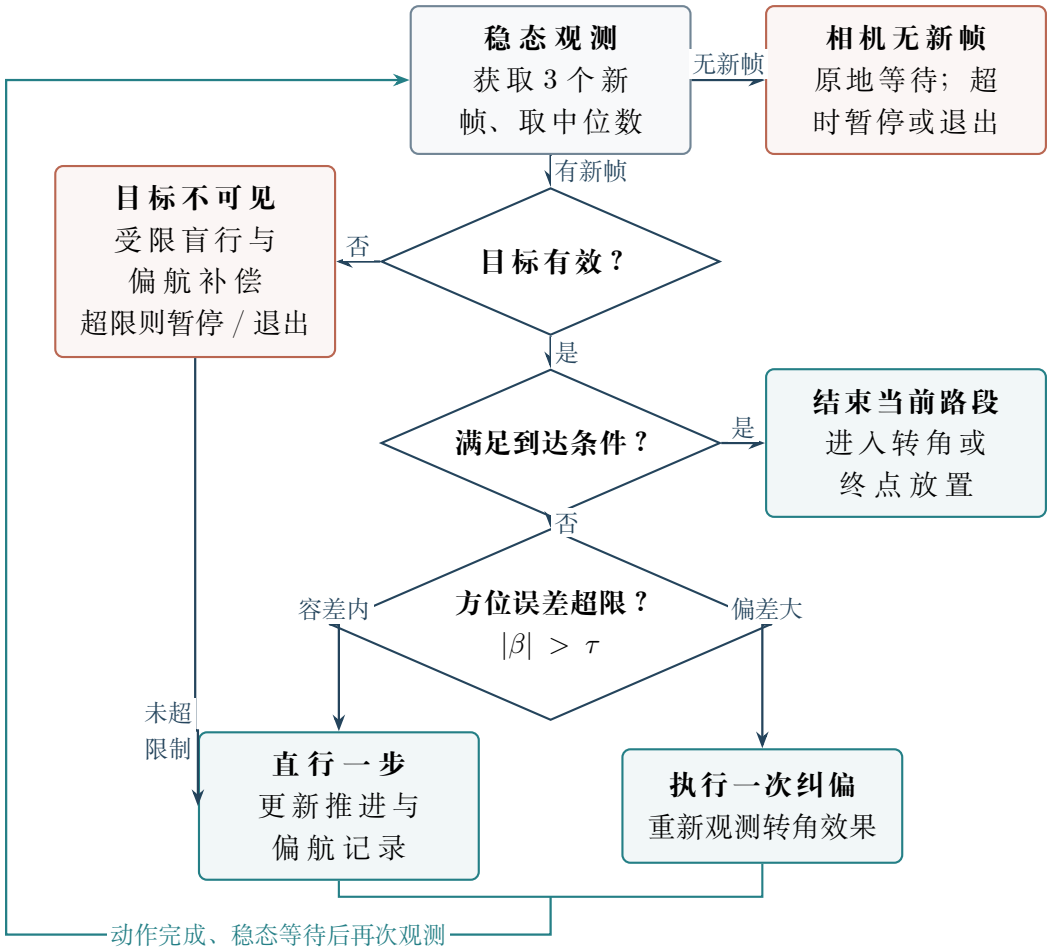


图 16: 单段观测—决策—动作状态。动作完成后回到获取新帧，盲行分支受计数约束。

该图强调判断顺序：先区分相机断流与目标丢失，再检查到达条件，最后决定纠偏或直行。相机无新帧不产生新的前进指令；有画面但无目标则进入受计数限制的分支。底部回路说明动作结束后必须重新观测，上一轮方位不能连续驱动多步。近距丢码的到达推断与盲行补偿细则见后文。

源码依据： `marker_route.py`：Eyes.look、Route._run。

D.3.2 多帧投票如何改变动作入口

图17枚举三个新帧中目标可见的全部八种组合，用于解释`Eyes.look` 的判据。只有至少两帧可见时才采用距离与方位的中位数；在三个新帧尚未收齐时，程序进入相机等待，而不把缺帧填成“未见目标”。

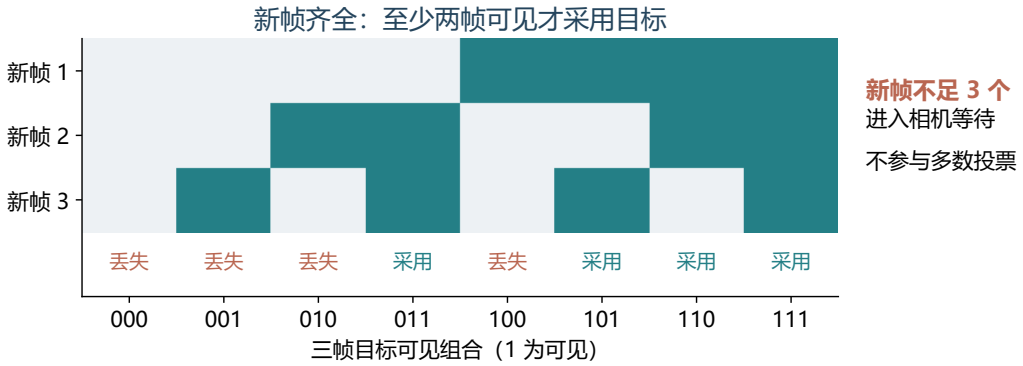


图 17: 三个新帧的多数可见判据。此图为源码条件穷举，不是识别准确率或概率分布。

八种组合并不等概率，连续帧的模糊与遮挡还可能高度相关，因此不能从“八种中四种采用”推出 50% 成功率。中位数能降低偶发异常对一次决策的影响，却不能修正错误标定或持续误识别。现场记录应把“缺帧”“足帧但不可见”“多数可见且采用”分别编码，才能辨别等待来自取流链还是标记检测。

源码依据： `marker_route.py`：Eyes.look。

D.3.3 自适应纠偏、转角学习与离散振荡

双足转向动作一次可能产生较大角度。误差只有几度就转一次，会让目标跨到画面另一侧，导致来回纠偏。程序使用近期五个有效转角效果的中位数，并把门限设为 `centre_deg` 与其 60% 的较大值；默认 `centre_deg` 为 8 度，初始 `turn_step_deg` 为 20 度。

$$\hat{\theta} = \operatorname{median}(\Theta_{t-4:t}), \qquad |\beta| > \max(8^\circ, 0.6\hat{\theta}). \tag{20}$$

转向后重新观测当前码，以方位变化绝对值更新估计，仅接收 3 至 60 度之间的变化。这个估计受到身体平移、码位置和角点噪声影响，不是 IMU 纯航向测量。为把流程图中的判断落实到数值空间，图18假设近期单次转角为 30 度，因此门限 18 度，只在有效观测条件下绘制。容差内继续前进是效率与偏差的折中，连续纠偏次数过多仍会暂停，不应从实验记录中删除这类失败事件。决策区域图给出了前述流程在有效观测条件下的几何切片：斜距不超过 40 cm 时优先判定到达；尚未到达时，以正负 18 度分隔直行与纠偏区域。这里 18 度来自假设转角估计的 60%，不是固定通用参数。该图不覆盖相机断流、计数超限或近距丢码分支，因而需要与流程图结合阅读。

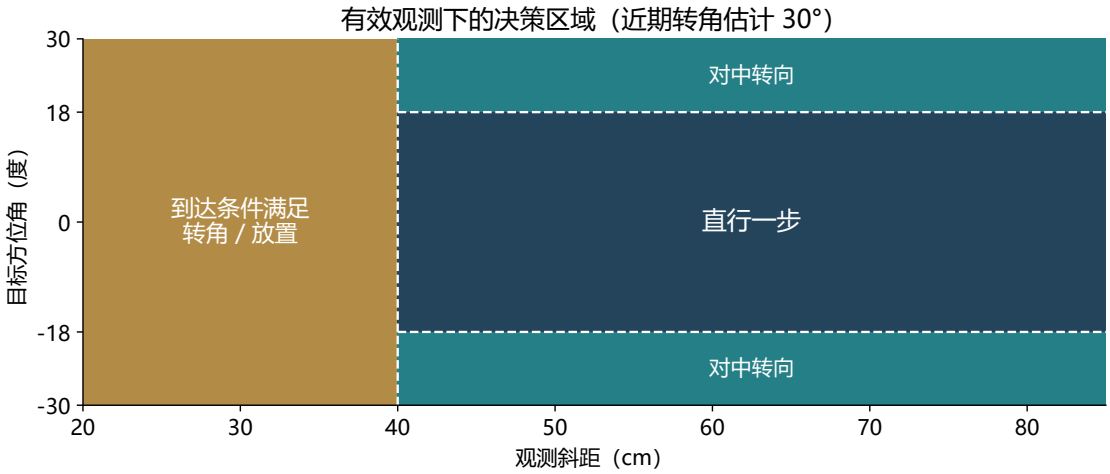


图 18: 有效目标观测下的解析决策域：假设近期转角估计 30 度。

因此，日志中的距离变化与方位变化应先按当前阶段分开，再判断动作是否符合优先级。图中到达区不再按方位分割，正是“先到达、后纠偏”的程序分支顺序所致；不能把它读成到达时方位必然已居中。

源码依据： `marker_route.py`：turn_steps、need、max_centre_turns。

D.3.4 直行偏航学习和受限盲行分支

直行前后目标均可见且方位绝对值小于 15 度时，程序把前后方位差加入 drift。至少三个样本后，可用近期 15 项中位数累计 creep，在无目标时超过单次转角的 60% 则纠偏。该量是图像方位的启发式变化，不是独立身体航向，也不是足端里程计；目标深度和相机晃动仍会影响估计。

首次未见目标和见过后丢失采用不同计数约束，`max_misses` 默认 2，`max_blind_steps` 默认 25，另有每段步数上限。这一策略可能帮助远处 C 暂不可见时推进，却不能保证遮挡空间安全，且与课程失去目标后原地重捕获的严格要求存在差距。报告据实呈现分支；如果现场最终收紧策略，应绑定新的提交和参数再评价，而不是提前写成当前程序能力。

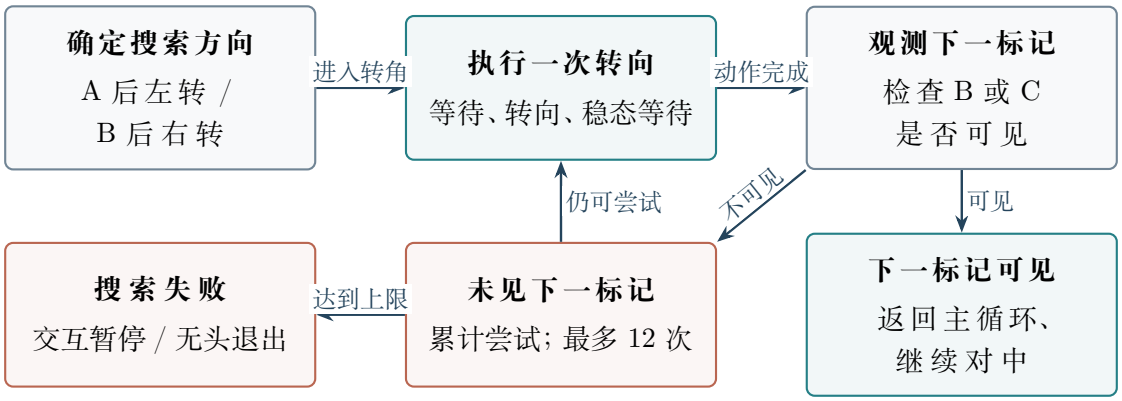
$$\hat{\delta} = \mathrm{median}(\Delta\beta_{t-14:t}), \qquad c_{k+1} = c_k + \hat{\delta}, \quad |c_k| > 0.6\hat{\theta} \Rightarrow \text{反向纠偏}. \tag{21}$$

源码依据： `marker_route.py`：prev_walk_obs、drift、blind、creep。

D.3.5 转角子状态机及最大尝试次数

corner 接收 side 和下一目标 nxt。每次先停 0.3 s，转一次，经 act 的 0.25 s 稳态等待，再停 0.3 s 观察下一码。成功返回只要求下一码有效可见，之后由下一段主循环继续对中；不是在 corner 里完成精准航向闭环。单周期耗时还包括文件动作时长、观测等待和 SDK 调用，不能只累加 d6a 时长估计全程。

若相机无新帧，corner 原地等画面，不继续转动。默认最多 12 次转向，未见下一码则 pause：有窗口等待 S，再重试一轮；无头模式退出。恢复保留任务阶段，但姿态已经变化，应核对目标与场地，不能把 S 恢复描述成回到最初起点重跑。



观测约束：无新帧时原地等待，不盲目继续转向；成功条件是下一标记可见，并非精确完成 90° 转角。

图 19: 转角寻找下一码的状态转换及失败出口。

转角控制将一次步态动作与一次目标检查串行执行，下一标记可见才允许返回主路线。尝试次数上限约束的是搜索过程，不能据此推算固定转角角度。相机断流时等待新帧，避免在没有观测依据的情况下继续累积转向。

源码依据：`marker_route.py`：Route.corner、pause。

D.3.6 近距丢码推断、停滞保护与任务结束

近距离码可能从画面下缘离开。obs 为空但 `last_seen` 距离不大于 limit+12 cm，且方位在 max(`centre_deg`,15 度) 内时，程序构造 `lost_close` 观测并把距离截到 limit。这是启发式到达，不是新的实时测距；相近条件下的遮挡也可能被解释为抵近，因此日志要区分直接观测和推断到达。

推进守卫默认检查 20 步内距离缩短是否超过 5 cm，另有每段 150 步上限。它检测“动作多但接近不足”，不直接判断碰墙、滑步或电机故障。C 到达后解除锁臂并播放 `put_down`，动作完成即返回，最新入口没有接入颜色放置确认链。正式结果需由视频和物体状态核验补足，不能把程序线程结束直接等同于完整比赛成功。

源码依据：`marker_route.py`：arrived_blind、progress、stall_steps、stop 分支。

D.4 动作执行、运行保护与日志

D.4.1 当前动作文件与首帧过渡的实证分析

最新动作副本从 `grab_final_v2` 尾帧锁臂，六个刻度为 566、908、40、435、108、935。下图实际读取最新 actions 目录，横轴帧号，纵轴身体通道，颜色为命令刻度。帧号不是等间隔时间；热图不代表编码器回传、力矩或接地状态，也不能从图中推断持物不掉落。

Servos.play 比较首帧目标与上一命令 pose：无已知姿态或任一身体通道差超过 25 刻度时，首帧至少延长到 300 ms。pose 初始化尝试读实际关节，后续通常是已发目标；并非每帧读反馈的动力学规划。放置动作解除锁臂，生成副本时首帧下蹲保持抱紧，后续松开与站起保持原设计。负载和地面变化仍需要受控实机验证。

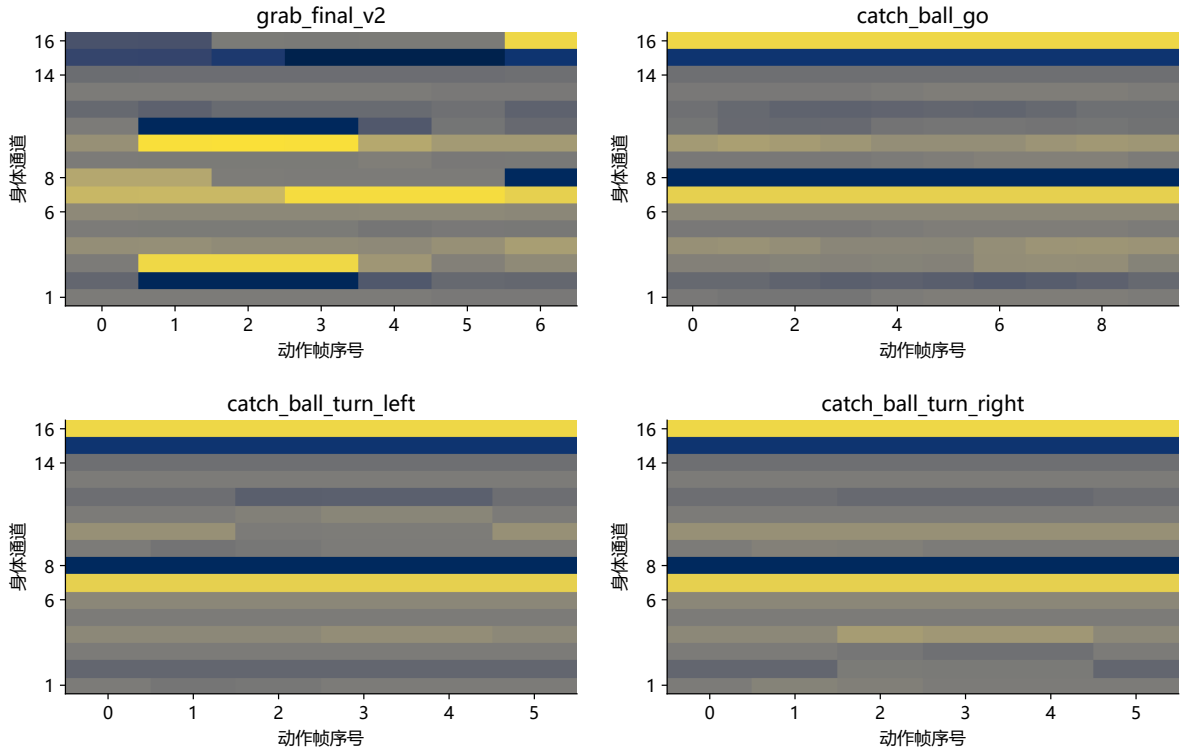


图 20: 最新动作数据库的身体通道命令矩阵。数据源为真实动作文件，非仿真关节轨迹。

热图中保持相同颜色的上肢通道用于核对锁臂约束，而下肢通道随动作帧变化形成步态。该表示可以发现动作副本是否意外改写抱持姿态，但无法证明实际关节已经到位。首帧过渡与负载下的运动连续性仍需结合动作时长和实机观察评价。

源码依据： `maze_route/actions`；`mkcopies.py`；`Servos.play`。

D.4.2 动作时间结构与闭环观测机会

动作数据库提供了可直接计算的时序依据。图21将每个动作的首帧与其余帧时长分开：抱起为 9.50 s，抱物直行为 0.95 s，左右转各 1.00 s，放置为 7.10 s。这里的数值仅为

当前动作副本的帧时长之和，不包含额外等待、首帧延长和接口开销。

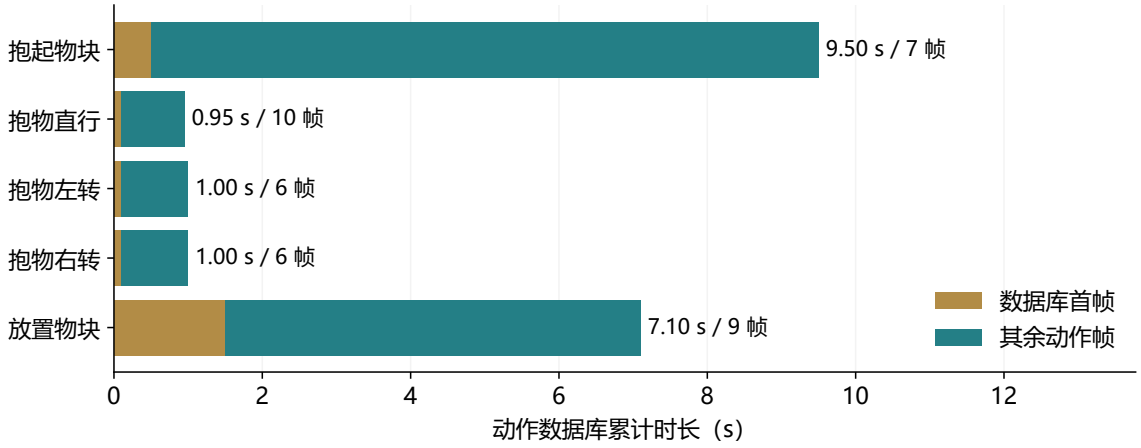


图 21: 实际动作数据库的时长组成和帧数。条形长度表示文件内计划时间，非实测执行耗时。

动作时间结构解释了为什么提高相机帧率不一定同比提高控制更新频率：路线线程通常先等一个完整动作返回，再稳态等待并重新观测。单次直行 0.95 s 已构成明显的动作占用区间；若首帧因姿态差异被延长到至少 0.30 s，实际区间还会增加。对抓放这类较长动作，状态转换应依据播放完成与物理确认分别记录，而非只按墙钟定时跳转。

这一结构也提示优化的取舍。缩短动作可能增加纠偏机会，但会改变双足运动连续性与夹持稳定性；拆分动作以增加观测点，需要重新核对每个片段首尾姿态，而不能简单截掉动作帧。报告中的时间图据真实文件生成，为动作标定与异常日志对齐提供依据，不将计划时长当作机械性能结论。

源码依据： `humanoid/maze_route/actions/*.d6a`；`Servos.play` 与`Route.act`。

D.4.3 交互暂停、无头退出与异常清理

P 清除 go，控制后续动作是否获准执行，不撤回已发舵机帧；S 重新设置 go。Q/Esc 或退出信号设置 `stop_ev` 并置 go，使等待线程得以退出，执行器在下一帧边界停止继续下发。软件停止后续帧与物理立即停止不同，电机当前插值与负载惯性仍存在。

pause 在交互模式等 S，无头模式直接结束。摄像头超时、转角失败、纠偏次数过多都可触发，但恢复点由各循环上下文决定。Route.run 捕获异常写日志、置 crashed 并关闭 CSV；主循环 finally 关闭相机和窗口、等待线程。该生命周期管理不等于倾倒保护或硬件急停。

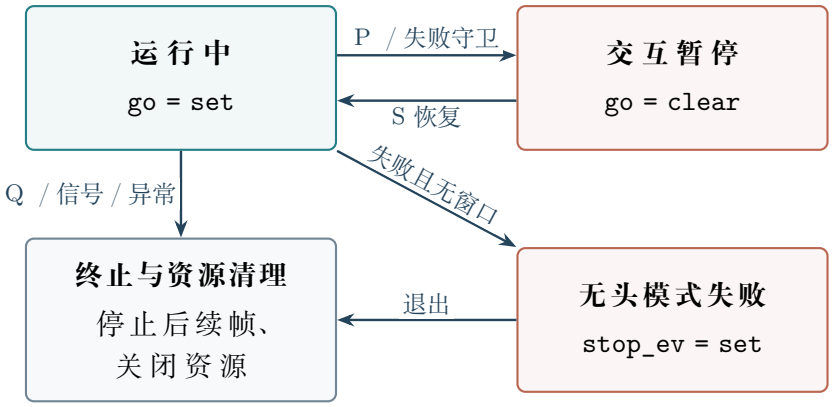


图 22: 运行、暂停、恢复与退出状态；全局故障不等于自动重置路线。

暂停与退出具有不同的生命周期：暂停保留阶段并等待恢复，退出则终止后续动作与资源使用。恢复箭头并不重建机器人位姿，也不会重新执行整条路线；因此恢复前需要确认物块、朝向和当前标记。图中的软件守卫不能替代硬件急停。

源码依据： `marker_route.py`：pause、wait_running、Servos.play、main。

D.4.4 逐次日志形成的状态证据链

route.log 记录参数、阶段、转角学习与暂停退出，decisions.csv 逐次记录 phase、target、seen、斜距、方位、yaw、action、note，snapshots 保存对应标注图。分析应先按 phase 切分，再统计 walk、纠偏、盲行、corner、pause 和 stop，并在转移附近回看图像。这能说明每个状态转换的观测依据，而非只展示最终画面。

本次没有归档完整实机 decisions.csv，因此状态图来自源码，动作图来自真实数据库，参数曲线是解析演示。后续可从真实日志绘制距离和方位时间线、动作事件带、相机断流和近距推断标记。最后一张识别框截图不能证明全程无失锁，日志中动作完成也不能证明无掉落或碰撞；这两类事实要由视频或传感器补充。

源码依据： `marker_route.py`：record、setup_logging、overlay。

D.5 状态契约、参数标定与验证边界

D.5.1 任务层转换契约与可达状态

表中把路线图转换成可核对的前置条件、执行行为和后继状态。状态可达性不仅由视觉决定，也由运行模式决定：no-grab 绕过抓取播放，no-release 绕过终点放置，交互暂停增加等待恢复，无头失败直接结束。分析完整任务时必须固定这四类开关，否则“结束”对应不同物理结果。

表 25: 任务主状态的转换契约。

当前状态	事件/守卫	执行动作	后继状态
等待	S 或无头延时结束	放行 go	可选抓取
抓取	动作播放返回成功	从尾帧读取六个锁臂值	走向 A
走向 A	A 距离到阈值/近距推断	记录 corner，调用 left	找 B
找 B	B 满足新帧多数可见	corner 返回 True	走向 B
走向 B	B 距离到阈值/近距推断	记录 corner，调用 right	找 C
找 C	C 满足新帧多数可见	corner 返回 True	走向 C
走向 C	C 距离到阈值/近距推断	记录 stop，按开关放置	任务返回
任意执行阶段	停止事件/异常	停止后续帧，记录原因	退出/清理

抓取、保持和放置应视为独立物理谓词，但当前入口只对动作序列作控制，不独立验证所有物理谓词。可以定义规划任务成功为 P 抓取且 P 路线且 P 放置，线程正常返回仅是软件条件，不能与该合取等价。正式验收应使用连续视频或新增传感器给这些谓词赋值，附上对应日志时间。

源码依据： `marker_route.py`：Route._run、main 及 ROUTE。

D.5.2 单段决策的优先级及边界输入

分支顺序决定相同观测怎样被解释。无相机新帧先等待；随后判断近距丢码；有效观测先判到达，再判方位；只有未到达且方位在容差内才前进。若距离已经小于阈值但方位很偏，程序仍优先认定到达，这与“必须先严格居中再到达”的控制律不同。报告不改源码，仅把该优先级作为需要现场核对的边界。

输入情形	当前响应	验证重点
不足三个新帧	相机等待	断流不持续走/转
三帧仅一帧有目标	目标不可见分支	盲行上限及先前是否见过
距离达到,方位偏大	到达优先	路口空间与码布置是否合理
距离未到,方位超门限	单次纠偏	防来回摆与最大纠偏次数
距离未到,方位容差内	单步直行	负载下实际偏航和距离推进
近距最后观测后丢码	lost_close 到达推断	遮挡造成误触发的风险

表 26: 单段决策的边界案例，不是新增控制器。

对每个边界案例应使用固定输入或真实日志回放验证动作选择，算法逻辑验证和物理试验分别记录。临界值附近的跳变并不必然是 bug，因为离散动作本身采用阈值切换；重要的是单步幅度和迟滞是否与实际机体相容，以及守卫是否防止状态无限抖动。

源码依据： `marker_route.py`：obs 判断、arrived_blind 与 need 分支。

D.5.3 相机资源重连、USB 故障与视觉时效

`robust_camera` 使用 USB 身份查找摄像头，避免设备号变化后固定 video0 失效，并处理占用相机的网页进程与断线重连。共享摄像头被两个程序分别打开时，能打开节点不代表可以持续取得新图；相机恢复后还要确认帧更新时间与标记检测线程确实推进。报告应区分设备存在、取流成功、新帧连续和识别有效四个层级。

USB 断开可能与供电、线缆或大负载动作有关。当前材料只支持当次恢复和局部取流结果，不能凭重新插接一次就认定完整搬运期间不再掉线。建议把相机错误时间与动作类型、供电、电量、帧序号和等待时长对齐；若故障集中发生在转身，排查电源和线缆拉扯比单纯增大 camera-wait 更直接。故障排查应从最早出现异常的层级向下追踪：有图像不等于有新帧，有新帧不等于目标有效，有效目标也不等于测距正确。按层保存证据可以避免将相机取流故障误判为路线决策错误。

证据	可以支持的结论
设备节点可见	内核当前枚举到相机，不能证明能取图
连续新帧与序号	当前取流持续，不保证标记一定有效
标记多数可见	当前 look 观测成立，不保证后续转角可靠
完整负载录像 + 日志	可评价动作期间断流、等待与恢复效果

表 27: 视觉链故障证据的层级。

当取流已恢复但目标仍无效时，应继续检查标记尺寸、遮挡和检测输出；只有前置观测满足要求，路线状态转换才有可靠依据。按表中层级归档能够保留这一判断顺序。

源码依据： robust_camera.py；Eyes.loop；Eyes.look。

D.5.4 动作时间预算与感知控制带宽

离散双足动作的控制频率由动作长度和观测等待共同决定。直行一次除了动作文件时长，还有 SETTLE_S=0.25 s、WALK_PAUSE_S=0.2 s 和 look 取得三个新帧的时间；转角还包含前后各 0.3 s。机械运动与感知并不是连续毫秒级闭环，因此用相机帧率当机器人纠偏带宽会严重高估响应。

$$T_{\text{walkcycle}} \geq T_{\text{gait}} + 0.25 + 0.20 + T_{\text{look}}, \qquad f_{\text{control}} \leq T_{\text{walkcycle}}^{-1}. \tag{22}$$

首帧桥接可能增加时长，断流和暂停会进一步延长。测量全程时间时应分解有效动作、正常等待、保护等待和人工恢复；较长总时间不必然表明视觉运算慢。优化方向先判断瓶颈：若多数时间用于大角度纠偏，调直抱物步态比提高图像线程帧率更有效；若 look 经常超时，则需要检查相机与计算资源。

源码依据： marker_route.py：WALK_PAUSE_S、CORNER_PAUSE_S、SETTLE_S 与 act。

D.5.5 标定计划：从印刷尺寸到负载步态

标定应分开相机几何、标记布局和身体运动。相机几何检查内参、畸变、实际分辨率以及 14 cm 黑边尺寸；布局记录 A/B/C 安装位置、高度、朝向与观察距离；运动测量抓取尾帧、负载质量、单步长度、左右偏航和转角分布。混合修改所有参数会使效果无法归因，也难以复现。

表 28: 人形标定与数据归档建议。

标定对象	记录量	验证方法
标记打印	黑色方形实际边长	尺量多边，与 SIZE_CM 一致

标定对象	记录量	验证方法
相机	内参、畸变、分辨率	多姿态图，独立距离检查
码布局	墙面坐标、高度与朝向	测绘图与相机视点照片
单步直行	负载、距离、偏航	多次独立起步，报告分布
单次转向	左/右角度与侧移	满载独立重复，外部量角
抓放衔接	首尾姿态、掉落	连续视频和动作文件哈希
相机恢复	断帧时长、恢复状态	人工断流与低速动作试验

文档中的 3.4 cm 步长或 10 至 15 度偏航属于已有描述，不能视为新实验复测结果。报告引用这类数值要保留来源和条件，正式统计须使用同一设备、同一负载和同一动作版本，避免把不同阶段参数拼成一组“最终实测”。

源码依据： `maze_route/README.md`；`marker_route.py` 运行参数。

D.5.6 课程要求与当前人形闭环的对应

当前代码解决了基于固定标记的路线推进、离散纠偏与有界转角搜索；课程则同时要求抓取成功、全程抱持、不触障碍、左右转弯和放置。程序已有的状态守卫与这些要求并非一一充分对应。理解这个差异有助于设计实验，而不是削弱实现价值。

课程要求	当前实现支撑	仍需的证据/闭环
抱起目标物	`grab_final_v2` 播放与锁臂	独立抓取确认或视频
全程保持抱物	命令手臂固定	实际滑落检测及掉落点恢复
左右各一次转向	A-left/B-right 及下一码获取	实测路线、角度、无干预
不触碰围挡	码距离及次数/推进保护	机体包络、测距或实测接触
终点放下	C 到达后 put_down	放置成功和物体位置确认

表 29: 实现机制与任务验收条件的对应。

下一阶段应把真实失败归因到感知、动作或状态接口，再决定是否需要新增传感器。不能用合成路线成功覆盖物理持物与避碰的缺项，也不把软件存在的功能认定为未参与成员的实际工作。组内协作见第2节，技术结果关联具体文件和运行证据。

源码依据： 课程任务手册；`maze_route/marker_route.py` 与已有运行说明。

E 地图跟随定位、规划与保护补充分析

E.1 定位、轨迹与导航算法

E.1.1 已知地图方法的建图与定位分离

方式二在障碍物布置完成后建图，再在固定地图上定位和导航。仓库与课程环境使用 gmapping 建图、`map_server` 加载地图、AMCL 估计本车地图位姿、`move_base` 完成路径与速度规划。建图和比赛运行阶段要分开；gmapping 与 AMCL 不应同时发布相同 map→odom 变换，否则 TF 来源竞争会使导航位姿不一致。

gmapping 的原理背景参见 [7]。gmapping 背后的 Rao-Blackwellized 粒子滤波将轨迹和地图联合后验分解：每个粒子表示机器人轨迹假设，并持有相应地图。激光扫描匹配和里程计先验影响粒子权重，回环处尤其需要足够视角和合理运动。照片里的低纹理板面不妨碍几何扫描，但相似长直通道可能让定位沿通道方向约束较弱；开放端和转角更利于区别位置。

实验地图必须连同 yaml 和 pgm 保存，并记录分辨率、原点和障碍布置版本。仓库 slam/maps 的 room1/room2 属参考资料，不能当作这张照片场地的比赛地图。照片拓扑图也不能转换为虚构 gmapping 输出。比赛现场修改围挡、移动横板或缩小通道后应重新测绘建图，避免规划器在旧空闲区撞上新增实体。

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t}) = p(m \mid x_{1:t}, z_{1:t})\, p(x_{1:t} \mid z_{1:t}, u_{1:t}) \tag{23}$$

场地拓扑见图 1。

核对项	技术含义与参数
准备阶段	驱动 + 传感器 + gmapping + `map_saver`
比赛阶段	`map_server` + AMCL + `move_base`
地图证据	实测 pgm/yaml、分辨率、原点、布置版本

表 30: 已知地图方法的建图与定位分离的实现要点。

建图结果、定位输入与导航调用需要绑定同一张地图及坐标约定。地图文件存在并不保证 AMCL 已收敛，首次运行仍应通过实际扫描与地图的重合关系确认初始位姿。

源码依据： `README.md`；`maps/README.md`；`slam/README.md`。版本以本报告提交号及配置快照为准。

E.1.2 AMCL 粒子定位与协方差有效性

AMCL 在已知地图中递归估计机器人位姿，运动模型预测粒子，观测模型以扫描与地图一致性更新权重，再进行自适应采样。定位不是一次初始位姿输入后永久有效；

若机器人被搬起、里程计异常或扫描缺失，后验可能变宽或跳到错误通道。Follower.on_loc 使用 frame_id 和协方差对定位输入作初步质量检查。

localization_ok 检查 36 元协方差数组中 x、y、yaw 的对角元素，要求有限、非负并低于配置上限。地图 follower 默认 x/y 上限 0.25、yaw 上限 0.35，量纲分别为 m^2 和 rad^2，不是位置/角度误差本身。协方差小也不等于位置正确，多峰分布、错误初值或观测模型过度自信可能产生低协方差误定位。

AMCL 在静止时不一定持续发布新位姿；仓库 localization_refresh 定期请求无运动更新，并要求有效扫描。该机制更新观测证据，不允许凭服务请求制造假的新定位。验证需要检查扫描更新时间、AMCL 新时间戳及 map 到 base 的连通性；定位失效后重新初始化并记录重置，不应让后车依据过期位置持续追踪。

$$w_t^{(i)} \propto p(z_t \mid x_t^{(i)}, m) w_{t-1}^{(i)}, \quad \Sigma_{xx}, \Sigma_{yy} \le 0.25, \quad \Sigma_{\psi\psi} \le 0.35 \tag{24}$$

坐标链由共享 map 与各车独立 odom/base 组成。

核对项	技术含义与参数
协方差检查	数组长度 36；有限；非负；对角阈值
静止更新	request_nomotion_update + 新扫描支持
额外验证	真实位姿误差、重定位稳定性、TF 连通

表 31: AMCL 粒子定位与协方差有效性的实现要点。

协方差门限提供的是定位有效性筛选，并不直接保证真实位姿误差小于某个距离。环境重复或地图失配时仍可能产生错误匹配，因此需结合扫描对齐、TF 连续性和时间戳共同判断。

源码依据： turtlebot/follow2/multi_robot_following/src/competition_core/planning.py；turtlebot/follow2/multi_robot_following/scripts/localization_refresh.py。版本以本报告提交号及配置快照为准。

E.1.3 目标测量、地图变换与轨迹缓存

方式二的领航目标可来自 TonyPi AR 标记或前车共享地图位姿。marker_target 把观测按测量时间转换到地图；pose_relay 校验前车位姿。map_follower 只接受地图坐标系正确、新鲜、严格递增时间戳且跳变可接受的目标。没有目标时不会从 TonyPi 的动作次数生成地图位置，这避免把开环步数误当作准确领航里程计。

Trail 用有界折线保存测得目标位置，默认 spacing=0.08 m、max_length=20 m、jump_limit=0.8 m。相邻目标变化小于采样间距时保留旧点以降低噪声和存储；跳变超过上限被拒绝，目标时间戳不会因此刷新。总弧长超限时删除前端旧点，保证长时间运行不会无限增长。默认参数是设计起始值，其大小必须与真实目标运动速度和更新间隔相容。

对弯道，弧长缓存保留路径转角，而单纯按领航机器人瞬时朝向后退可能把目标放到墙内。必须进一步说明：保留转角的目标点不等于机器人严格复现轨迹，move_base 仍可能在可通行区域平滑或改变局部路径。想证明无切角，应比较后车实际轨迹到领航折线的距离，而不是只画目标点。

$$\ell_k = \|p_k - p_{k-1}\|, \qquad L = \sum_k \ell_k, \qquad p_k\text{接受条件：}\ \ell_k \le \ell_{\text{jump}} \tag{25}$$

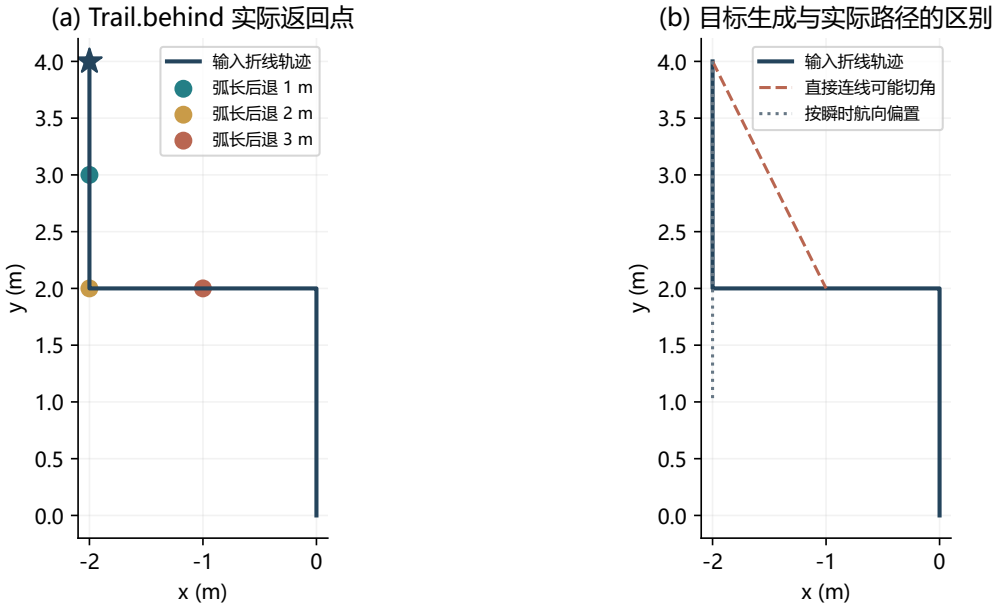


图 23: 核心 Trail.behind 对合成折线的真实返回值。输入点以米为单位，仅用于解释弧长算法。

不同回退距离对应同一折线上的不同位置，结果取决于累计路程和末段插值。输入历史过短时，回退目标不能凭空延伸到未知路径，跟随器必须结合最小轨迹长度与间距条件决定是否发出目标。下面将这些条件与采样参数一并列出，便于复核配置。

核对项	技术含义与参数
有效来源	带时间戳 AR 变换 / 共 map 前车位姿
采样约束	0.08 m 间距；0.8 m 跳变上限；20 m 缓存
导航边界	目标沿折线；实际路径由规划器决定

表 32: 目标测量、地图变换与轨迹缓存的实现要点。

采样时刻决定目标应使用哪一个相机位姿进行变换；弧长参数决定沿缓存轨迹回退多远。将这两类参数混为纯位置偏移会在转弯和运动中引入系统误差，因此日志应同时记录时间戳、地图点和轨迹长度。

源码依据： turtlebot/follow2/multi_robot_following/src/competition_core/planning.py；turtlebot/follow2/multi_robot_following/scripts/map_follower.py。版本以本报告提交号及配置快照为准。

E.1.4 全局路径规划、障碍膨胀与未知区域

地图导航先在栅格代价地图中选择可行目标，再由全局规划器连接本车与目标。实体围挡、扫描障碍和机器人足迹共同确定可行性。围挡照片中的通道较窄，膨胀半径越大，规划越偏向通道中心，但过度膨胀可能让本可通过的通道没有可行目标；减小膨胀必须以实测轮廓和定位误差为依据，不能只为了让路径出现。

`grid_clear` 采用圆形足迹并考虑单元格面积，检查半径加上半个栅格对角线范围内所有单元。未知值小于零或占据值达到阈值均视为阻塞，越界和无效地图也拒绝目标。地图 follower 默认阈值 65，RGB-D Grid.clear 默认阈值 95，二者对 OccupancyGrid 的保守性不同；这些值不能和内部 costmap 的 0—255 代价直接互换。

目标点可行只证明该点周围足迹没有被当前地图占据，不证明从当前位置存在完整路径；规划器仍需处理连通性和局部动态障碍。全局地图应保留固定围挡，动态扫描进入局部及全局障碍层。对于传感器未看到的区域，保守未知处理可能让机器人等待，这是信息不足的结果，不应通过将 NaN 深度标成自由空间解决。

$$r_{\text{check}} = r_{\text{robot}} + \frac{\sqrt{2}}{2}\, s, \qquad c_i < 0 \text{ 或 } c_i \geq c_{\text{th}} \Rightarrow \text{blocked} \tag{26}$$

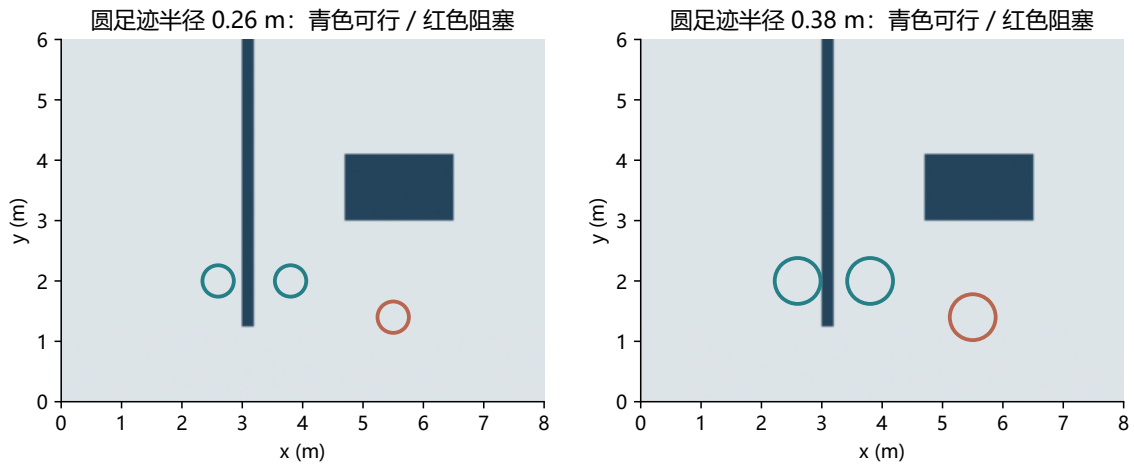


图 24: 圆足迹、栅格离散化和未知区域的安全检查示例，使用实际 `grid_clear` 函数判断颜色。

足迹检查把候选目标由单个点扩展为具有车体半径的区域；中心格可通行不代表整个机器人可放入。栅格边界补偿用于降低离散采样遗漏风险，未知区域则按不可进入处理。表中列出检查时涉及的地图量纲与边界约束。

核对项	技术含义与参数
单元量纲	OccupancyGrid：-1 未知、0—100 占据值
足迹补偿	半径 + 半栅格对角线
可行层级	目标局部足迹可行与整条路径连通分开

表 33: 全局路径规划、障碍膨胀与未知区域的实现要点。

足迹检查负责筛除不可放置的候选目标，规划器负责寻找可连接的路径，二者互不替代。即使目标点通过检查，也可能因通道过窄或地图连通性问题无法规划；这类失败需要保留导航状态与对应栅格。

源码依据： `turtlebot/follow2/multi_robot_following/src/competition_core/planning.py`；`turtlebot/follow2/multi_robot_following/config/costmap_common.yaml`。版本以本报告提交号及配置快照为准。

E.1.5 DWA 速度采样、动力学与弯道通行

DWA 的原始方法参见 [8]，本工程使用 ROS 配置中的实现。局部规划器 DWA 在满足速度及加速度限制的窗口中采样线速度和角速度，对短时轨迹作代价评价。地图配置以 10 个线速度样本、20 个角速度样本和 1 个横向样本工作，最大线速 0.15 m/s、最大角速 0.5 rad/s；RGB-D 配置采用 12×25 样本和更低 0.12 m/s、0.4 rad/s 上限。轨迹展开图展示采样候选的几何差异，不是调用完整 ROS DWA 得到的获胜轨迹。

差速底盘不能横向平移，因此曲线来自 v、ω 的组合。路径距离权重偏向全局路线，目标距离权重鼓励接近目标，障碍代价限制靠近围挡；权重改变通常会影响在狭窄弯道的犹豫、贴墙或局部振荡。`sim_time` 较长可以看见更远的碰撞，却会增加计算并把未来不确定性带入选择。运动采样必须与实际制动能力一致。

仓库关闭恢复旋转和 `clearing_rotation_allowed`，以免失去目标后后台旋转继续运动。导航 ABORTED、REJECTED 或 LOST 会锁定故障，排查后重启 follower。这个配置选择牺牲部分自主逃离能力，换取目标保持和受控恢复；实测比较应记录等待时间、导航失败和人工重置，而不能只计算最终到达率。

$$\dot{x} = v\cos\psi, \quad \dot{y} = v\sin\psi, \quad \dot{\psi} = \omega, \qquad J = w_p J_p + w_g J_g + w_o J_o \tag{27}$$

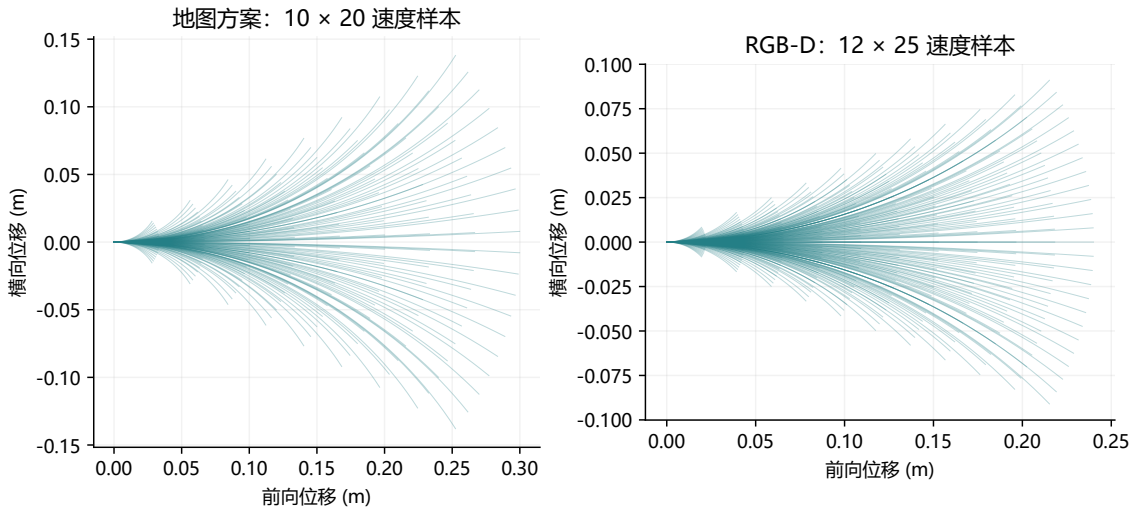


图 25: 按配置速度范围展开的运动学候选轨迹。图未调用 ROS 局部规划器评分，属于采样几何演示。

候选轨迹展示速度与角速度组合如何改变短时运动范围，而实际选择还取决于障碍代价、路径距离和目标距离评分。图中没有运行这些评分项，不能用曲线数量或形状推断避障成功率。配置核对应进一步确认采样密度、预测时域与制动能力是否匹配。

核对项	技术含义与参数
地图方案	max v=0.15；max ω=0.5；10×20 样本
RGB-D 方案	max v=0.12；max ω=0.4；12×25 样本
恢复行为	禁用旋转恢复；失败锁定并请求排故

表 34: DWA 速度采样、动力学与弯道通行的实现要点。

表中两套配置分别约束地图方案与 RGB-D 方案的候选运动，不能把更密的采样直接解释为更高成功率。减小速度上限可降低短时位移，却也改变弯道跟随和超时行为；禁用旋转恢复后，受阻可能需要人工复位。实际调参应同时记录通道宽度、局部障碍和导航失败状态。

源码依据： `turtlebot/follow2/multi_robot_following/config/move_base.yaml`；`turtlebot/follow3/rgbd_following/config/navigation.yaml`。版本以本报告提交号及配置快照为准。

E.1.6 导航目标更新、取消与停止不变量

移动目标持续变化，若每帧向 `move_base` 发送新目标，规划容易不断抢占并产生振荡。`map_follower` 使用 `goal_interval` 和 `goal_shift` 双阈值，默认至少间隔 1 s 且目标移动 0.25 m 才重发；首次目标无条件发送。该滤波位于目标发布层，不能替代感知关联，也不能掩盖真实的突然变向。

RGB-D `need_goal` 使用 0.5 s 间隔、普通位移阈值 0.15 m，并对最终目标或成功状态将位移阈值降到 0.02 m，防止到终点时微小目标更新被忽略。相同“限频”概念在两套代码中实现不同，报告应分别列出参数，不能把最终目标修正能力泛化到地图 follower。地图版本保留了较简单的完成判断，需在终点试验中检查其旧目标冻结风险。

任何不满足许可的 step 调用 block，撤销 `motion_allowed` 并取消已有 goal。导航状态 4、5、9 会锁定 failed，后续循环不再自动恢复。停止不是删除当前状态信息：`follow_status` 保留明确原因，便于区分 `target_lost`、`leader_too_close`、costmap stale 和 localization invalid。这样的可观察停车行为是系统诊断的重要组成部分。

$$\text{Resend} = \big(\Delta t \geq \tau_g\big) \wedge \big(\|g_t - g_{t^-}\| \geq \epsilon_g\big) \tag{28}$$

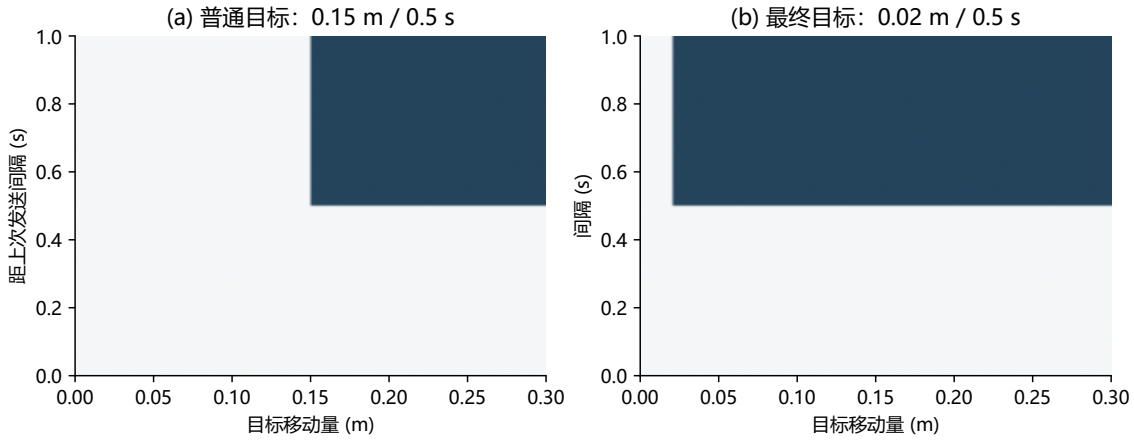


图 26: 实际 `need_goal` 函数的普通/最终目标更新区域；与地图版本的 1 s/0.25 m 阈值区分使用。

目标刷新同时约束发送时间与位置变化，目的是减少频繁抢占导航任务。其边界也意味着小幅移动可能不会产生新目标，因此终点停靠不能只依赖刷新触发。取消目标、任务许可与下游零速门控需要分别承担停止职责。

核对项	技术含义与参数
限频收益	减少重复抢占和目标抖动
终点风险	目标移动小于阈值可能留在旧目标
停止路径	撤销许可 + `cancel_goal` + 状态原因

表 35: 导航目标更新、取消与停止不变量的实现要点。

目标刷新解决导航输入变化，取消处理旧任务，门控零速约束实际底盘出口。三者的作用范围不同，实验记录应区分“停止发送新目标”“取消已有目标”和“速度已经降为零”，才能验证停止链是否完整。

源码依据： `turtlebot/follow2/multi_robot_following/scripts/map_follower.py`；`turtlebot/follow3/rgbd_following/src/rgbd_following_core/motion.py`。版本以本报告提交号及配置快照为准。

E.2 传感器观测与坐标一致性

E.2.1 Kinect 深度到扫描：先验证观测再验证地图

照片场地包含面板、立柱和较低障碍，二维导航所见范围由传感器视场、安装高度、俯仰和深度转扫描采样行共同决定。现场曾报告约 35 cm 障碍未稳定进入地图，应沿原始深度、转换后的 scan、TF、地图四层排查。若原始深度没有有效返回，地图膨胀无法补造障碍；若深度有而 scan 没有，需核对采样高度、有效量程与相机姿态；若 scan 有而地图无，再检查 TF、更新和清除配置。

二维 LaserScan 不是所有三维障碍的完整表示。把一段图像列的深度压缩到平面会丢失高度结构，过低采样可能遇到地面，过高采样可能越过低障碍。实验不能只按 URDF 的理想安装假定覆盖已满足。报告应保存深度与 scan 的同期截图、障碍实测高度、距离与相机安装，并按距离逐点检查有效性。

检查层	观察内容	可定位的问题
原始深度	障碍 ROI 有效深度与缺测	反光、近距盲区、遮挡、驱动
转换扫描	同方向是否有合法 range	采样行、量程、安装俯仰
坐标变换	optical 到 base 与 odom 连通	外参、时间、硬编码 frame
地图/代价层	marking、clearing 与高度范围	更新链、地图时效、清除覆盖

表 36: 35 cm 障碍问题的分层定位顺序。

排查顺序从原始深度到扫描、代价地图，最后到速度出口，能够定位障碍信息在哪一层丢失。若原始传感器未覆盖目标，增大膨胀半径也无法恢复缺失观测；若地图已有障碍而车仍前进，则应检查规划和发布链。

源码依据： `docs/vm_follow2.md`；`latex/sections/30_follow2.tex`；课程传感器链配置。

E.2.2 地图版本、AMCL 初值与长走廊可观测性

赛道面板几何重复，长直通道中扫描对横向位置和朝向约束通常强于沿通道方向约束；转角、横板和不同间距提供更多几何区别。这是定位设计的几何分析，不是对当前 AMCL 误差已测得的结论。初始位姿应在实际地图上给定，随后检查激光与墙面重合、粒子分布和实际运动方向，而不是只看 RViz 中存在一簇粒子。

地图保存需要 yaml 与 pgm 成对，记录分辨率、原点、障碍版本和采集时刻。任何面板移动都可能改变原自由区；比赛前地图不应混用参考 room 地图或照片拓扑图。建图阶段结束要停止 GMapping，再运行 AMCL，保证 map 到 odom 唯一发布者。多台车共享 map 原点，但各车 odom 和 base 保持独立，不能仅在 launch 最外层加 namespace 就假定驱动内部 frame 也已经独立。

源码依据： `navigation.launch`；`localization_refresh.py`；`docs/interfaces.md`。

E.2.3 观测时间的 TF 变换及三类坐标误用

AR 位姿必须按实际采样时间变换到 map，保留原始 stamp。机器人在图像与最新 TF 之间可能已移动，如果用当前变换处理旧像素，目标会产生虚假位移；这种误差还会进入轨迹缓存，影响后续路径切线和间距。`marker_target` 没有有效 TF 时丢弃，不将旧观测重新包装成当前时间。

$$ {}^{m}p_t(t_z) = {}^{m}T_c(t_z)\,{}^{c}p_t(t_z), \qquad \delta p_{\perp} \approx z\,\delta\theta. \tag{29}$$

三类误用尤其要避免：把 optical 的前向 Z 直接当 base 的高度 Z；把前车独立 odom 数值当共享 map 位姿；把 ALVAR 标签姿态当人形行走朝向。前两者破坏空间一致性，后者可能让后退目标落到侧墙。当前方式二使用轨迹切线而非标签 yaw，正确保留这个区分。

源码依据： `marker_target.py`；`pose_relay.py`；`map_follower.py`。

E.2.4 观测时间与变换时间的对应

图27将图像采样时刻和检测结果接收时刻分开。即使目标在世界中静止，本车在两时刻之间的位姿变化也会影响其相机坐标；正确变换需要查询采样时刻的相机位姿。图中仅表达时间先后，不假定具体传输延迟。

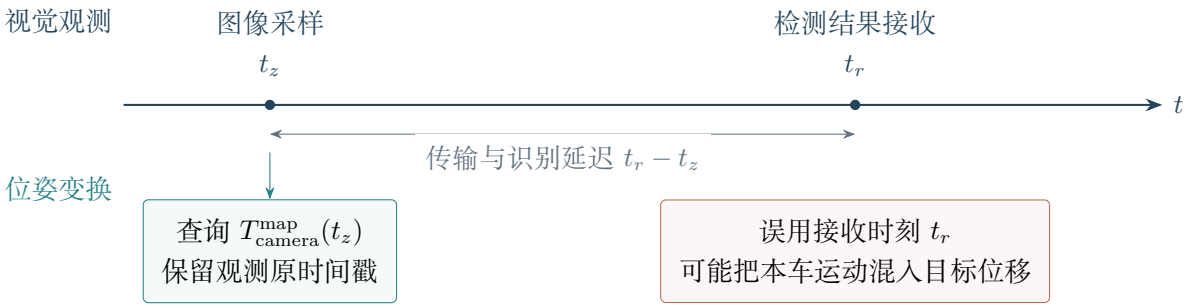


图 27: 带时间戳的目标观测与 TF 查询关系。接收延迟不应改变观测所属时刻。

将旧观测重新盖上当前时间戳会掩盖数据老化，并把机器人运动造成的坐标变化混入目标轨迹。反之，保留采样时间却缺少对应的 TF 历史也无法完成可靠变换，适配器应拒绝该样本。现场联调需同时检查设备时间同步、TF 缓存覆盖和图像消息时间戳，这三项分别约束时间基准、查询可达性与观测归属。

源码依据： `marker_target.py`：按标记时间戳查找 TF 并发布地图坐标目标。

E.3 控制架构与双层安全门控

E.3.1 ROS 节点、话题和唯一速度发布链

方式二的感知入口与导航执行通过 `target_pose` 连接。`marker_target` 只适配目标坐标，不直接控制底盘；`map_follower` 检查任务、定位、地图、TF 和间距后向 `move_`

base 发送目标并周期发布 motion_allowed。导航原始速度再进入独立 velocity_gate。图中分开任务桥、目标变换、规划和门控，有助于定位“看得见但不走”和“导航输出但被拒绝”的原因。从控制责任看，target_pose 只指定追踪对象的位置，导航器输出的是期望速度，motion_allowed 则决定当前任务是否允许速度通过。后两者通过不同话题到达门控，必须同时满足条件，不能只因收到速度就放行。节点图因此将“目标”“速度”“许可”分别画成独立箭头，联调时需要逐条核对，尤其注意跨命名空间的重映射。

键盘遥控或其他直接发布者若仍连接同一底盘输入，会绕过门控；门控零速不能压制所有竞争源。联调要检查实际连接图、话题发布者和 remap，关闭旧导航/遥控节点。多机情况下每车唯一输入、唯一 frame 前缀和共同时间基准应写入参数快照，而不是写死主机地址到共享文稿。

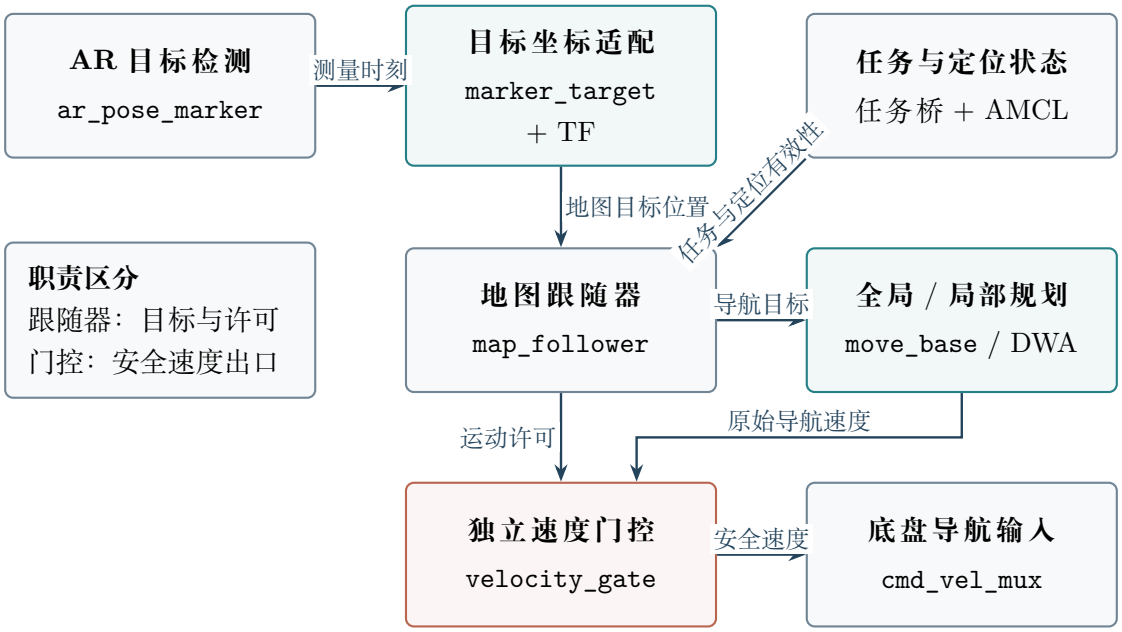


图 28: 方式二节点与数据链。任务、定位与 TF 是导航许可条件，安全速度应只有一个最终出口。

图中的速度门控处于底盘导航输入之前，其安全作用依赖于实际话题连接和控制源唯一性。复现时应把这张逻辑图与真实 ROS 连接图对照，分别确认导航原始速度、运动许可和最终安全速度的发布者，而不是只检查节点是否启动。

源码依据： follow_map.launch；navigation.launch；velocity_gate.py。

E.3.2 地图跟随器的状态优先级与故障锁定

map_follower 的 step 依次检查 finished/failed、任务、目标、定位、地图、TF，然后检查当前最小间距、历史轨迹长度和目标足迹。较前的守卫失败就 block 并返回，所以多个问题同时发生时，follow_status 只显示先遇到的原因。实验应保存原始输入的有效性，不能从单个状态字符串断言其他层都正常。

地图版本将 ABORTED、REJECTED、LOST 锁为 failed，需要排故后重启；目标

暂失或地图过期则撤销许可和取消目标，等待有效输入恢复。finished 则是任务 done 且接近后置点的简化完成条件，不包含 RGB-D 方案的完整终点多边形/停稳保持。报告不能将两套 follower 的结束功能混写，现场需补充进入终点并停稳的独立证据。这组守卫构成逐层收缩的运动许可：上游确认目标与空间条件，下游独立确认命令、许可和扫描是否新鲜。任一环节通过都不意味着其他环节可以跳过，尤其不能用导航器内部避障替代命令看门狗。故障复现时应记录最先拒绝运动的守卫，避免把所有停车归因于规划失败。

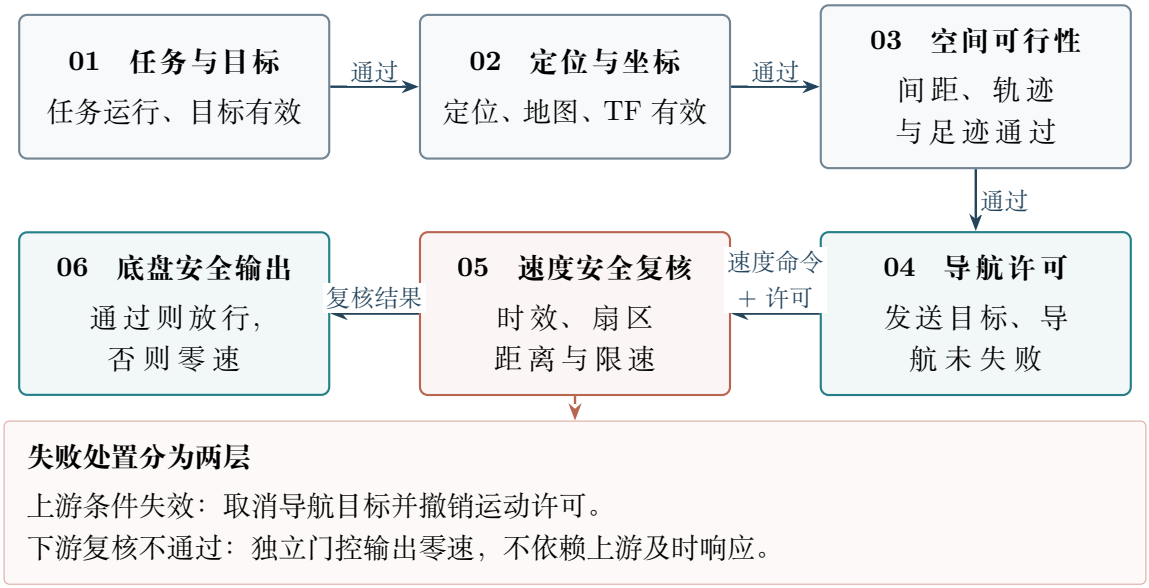


图 29: 方式二的分层守卫。任一上游条件失效取消目标，下游门控独立拒绝不新鲜命令。

图中的安全输出只有在整条许可链成立时才放行；上游取消目标与下游零速是两个独立动作。故障日志应同时保存导航状态和门控原因，才能判断停车发生于哪一层。

源码依据： `map_follower.py`：step、block、stop。

E.3.3 扫描扇区、看门狗与缺测的保守解释

Gate 默认要求命令与许可各在 0.4 s 内刷新，扫描 0.5 s 内刷新。`scan_clear` 只保留量程内的有限样本，前方扇区至少三点且最小 range 大于停车距离才通过；NaN 和 Inf 不能当成无障碍。直行时扇区为正负 45 度，转向角速度超过阈值时扩大到正负 90 度，仍不等于全车外围都有激光覆盖。

输出线程每 0.05 s 按墙钟运行，使 ROS 模拟时钟暂停时仍能归零。该看门狗使用接收时刻与测量时刻的共同约束，防止旧扫描迟到后被当作当前安全。拒绝倒车因为前方扫描不能证明后方空间安全；有限速度夹紧只限制命令，不证明硬件实际制动。地图与足迹检查、传感器覆盖和急停仍需分层验证。左图把每种时效阈值的有效区间分行显示，负时间差与超过阈值的样本均应拒绝；右图给出车体半径对理想横向余量的影响。两者分别约束信息是否可信和空间是否足够，实际窄道通行还需扣除定位误差与障碍更新延迟造成的余量消耗。

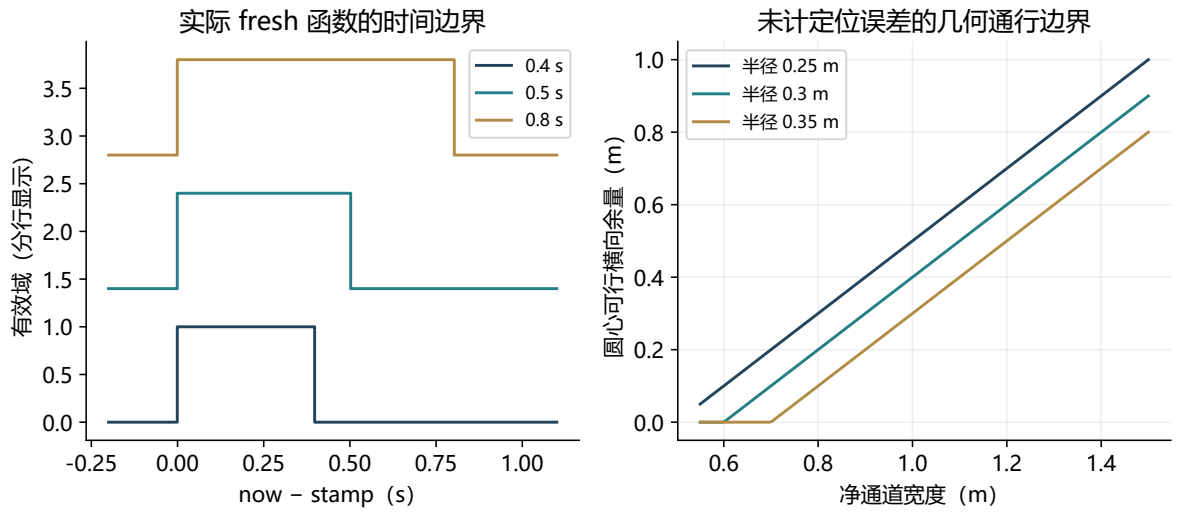


图 30: fresh 时间边界与通道几何余量。右图未计定位误差，是解析设计关系而非实测通行结果。

通道几何中的零余量只是理想圆形模型的边界；实际车体突出部分、定位偏差与动态障碍均会使可用空间进一步缩小。时效曲线和几何曲线应分别用于检查时间契约与足迹配置，不能用其中一条替代另一项验证。

源码依据： `velocity_gate.py`；`competition_core/planning.py`：fresh、scan_clear。

E.3.4 多级时效门限与停车责任

不同信息源承担不同角色，源码没有为所有消息设置同一个超时。图31把各门限并列：速度命令和许可需要更紧的刷新周期，地图和定位允许较长的信息年龄。该排序描述配置约束，不是测得的端到端停止延迟。

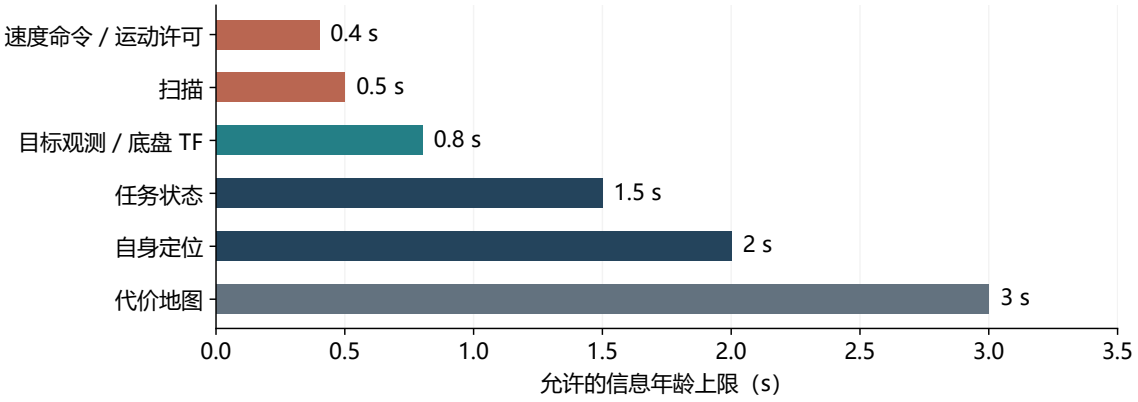


图 31: 方式二关键输入的默认信息年龄上限。门控与跟随器分别采用各自时钟和检查逻辑。

在链路失效时，停车首先由哪个条件触发取决于故障位置。目标消息中断可能由上游 0.8 s 检查发现，许可或速度停止刷新则由下游 0.4 s 看门狗拒绝；扫描缺失有独立 0.5 s 约束。上述门限还需加上调度、传输和物理制动过程才能形成实际停止时间，不能把最短门限直接写为整机安全响应。

年龄检查也依赖时间语义。门控使用接收侧墙钟保持独立性；目标和 TF 则需要可比较的采样时间。若不同设备时钟存在偏移，简单放宽超时可能掩盖同步错误。应先确认时钟与数据更新契约，再结合运动速度和可用安全距离确定阈值。

源码依据： map_follower.py 与velocity_gate.py。

E.3.5 窄道足迹、栅格补偿与参数耦合

圆形足迹半径必须覆盖车身和突出结构，栅格检查额外补偿半格对角线。通道净宽 w、半径 r 的理想横向余量为 w 减 2r，但真实可用余量还需扣除定位、地图边界、传感器安装误差和动态侧摆。理论曲线只能帮助发现参数冲突，不能证明车在实际围挡中通过。

$$\Delta w_{\text{usable}} \le w - 2r - 2\epsilon_{\text{loc}} - 2\epsilon_{\text{map}}, \qquad r_{\text{check}} = r + \frac{\sqrt{2}}{2}s. \tag{30}$$

跟随距离、目标更新阈值、速度上限、DWA 预测时长和膨胀半径互相影响。增大间隔缓解追尾，却可能使目标在转角被遮挡；增大膨胀偏向通道中心，却可能使局部目标不可行；降速减小制动距离，但不修复未知或漏检障碍。调参应基于失败类别逐项调整，不把所有停车归结为“速度太低”。

源码依据： costmap_common.yaml；move_base.yaml；planning.py。

E.4 任务联动与实验评价

E.4.1 TonyPi 最新入口与方式二的任务接口缺口

通用 competition.py 能够发送 UDP 状态，`mission_bridge` 验证发送端、session、`run_id` 和递增 sequence，缺数据超过 1 s 发布 stopped。最新 `marker_route` 是独立入口，尚未实现同一心跳。因此二者分别能运行不能证明自动协同完成，报告应把这一集成点明确画成待对接接口。

建议适配映射为：等待和暂停映射 idle/stopped，实际抱物行走和转角映射 carrying，释放映射 releasing，正常结束映射 done，异常映射 fault。此处是接口设计建议，本次只改报告，不修改机器人控制代码，也不声称这些映射已经执行。映射还需明确持物是否确认、暂停是否立即撤销许可、done 心跳持续多久，以及后车完成何时结束 Pi 进程。

人形阶段	建议任务状态	对接要点
等待/暂停	idle/stopped	不允许后车继续运动
抱物行走/转角	carrying	持续心跳；身份与扫描仍独立检查
放置	releasing	允许后车完成尾部跟随
完成/异常	done/fault	正常收队与故障停车区分

表 37： 拟议适配契约，非当前最新入口已接入的实现声明。

适配完成后还需分别验证正常运行、主动暂停、异常退出及通信中断时的状态传播。只有领航任务状态能稳定触发跟随许可撤销，才能将该接口认定为联动功能，而非仅存在相同名字的话题。

源码依据： mission_bridge.py；marker_route.py；competition.py。

E.4.2 单车导航到移动目标的实验阶梯

方式二验证顺序应是基础驱动、扫描覆盖、地图质量、静态定位、静态目标导航、动态 AR 目标、失效注入、再多车联调。这样可区分感知漏检和规划问题；35 cm 障碍未确认之前直接多车穿行，会使失败原因混杂。各阶段以独立 trial 记录代码、参数、地图和视频索引。

静态目标先检查 AMCL 初值与目标可达性，再让 TonyPi 低速直行。左/右弯分别观察遮挡后停车、重捕获与目标轨迹连续性；移开标签测试目标话题静默，而不只测试检测器发空数组。停止任务、扫描或 TF 分别记录底盘实测停稳延迟。急停命令发出到门控零速，与零速命令到物理停止是两个时间，日志中应分别保留。

源码依据： docs/vm_follow2.md；latex/data/experiment_records.csv。

E.4.3 方式二结果的指标与失效归因

报告同时记录欧氏间距误差和沿路径弧长间隔，拐角处两者不同；目标点沿折线后退并不证明底盘精确复现折线。应对比本车轨迹与领航历史路径的横向距离，结合最小障碍余量、丢失时长和暂停原因。DWA 重规划可能在自由空间切角，此时区分目标生成、规划偏移和定位误差。

$$e_E(t) = \|p_r - p_l\| - d_f, \quad e_\perp(t) = \min_{q \in \Gamma_l} \|p_r - q\|, \quad T_{\text{stop}} = t_{\text{stopped}} - t_{\text{fault}}. \tag{31}$$

停止可能是任务许可暂停、目标丢失、间距过小、地图过期、足迹受阻或导航失败，含义不同。`waiting_for_leader_trail` 属于启动等待，不能与导航失败一概统计。完整路线还需要继续测试。已有 Kinetic 隔离检查主要检查输入失效时能否归零，实际避障和全程跟随效果仍要结合现场运行记录。

源码依据： map_follower.py；velocity_gate.py；实验原始记录。

E.5 参数自洽、失效边界与终点收队

E.5.1 目标源适配契约与输入冲突

方式二可用 marker 或 pose 目标源，但同一 `target_pose` 不能同时由两个互不协调的适配器发布。marker 来自 TonyPi 背部 ALVAR 实际观测，pose 来自前车地图定位；二者目标身份、误差结构与可见条件不同。切换来源必须在实验元数据中登记，不把两种来源得到的结果混为一次独立方法。

目标源	输入前提	主要误差/失效
marker	ID 正确、尺寸标定、测量时间 TF	遮挡、角点误差、内参/外参错误
pose	前车地图原点一致、定位有效	协方差异常、跳变、时间过期
公共输出	frame=map、严格递增 stamp	坐标错用、旧观测刷新、双源竞争

表 38: 目标适配器的共同契约和差异。

`map_follower` 接受的是 map 中的目标点，不会从 TonyPi 动作次数估算位置。输入缺失时取消目标，历史缓存不是无期限继续运动的许可。上层身份有效与下层几何可行均成立，才具有继续推进的必要条件。

源码依据： `marker_target.py`；`pose_relay.py`；`follow_map.launch`。

E.5.2 定位质量门限的统计量纲

`localization_ok` 检查 36 元协方差数组中 x、y、yaw 的对角值，要求有限、非负并小于阈值。0.25 的平移协方差上限量纲是平方米，不是 0.25 米位置误差；0.35 的角协方差是弧度平方。对角值通过只说明 AMCL 自己报告分布未过宽，不能保证全局匹配正确。

$$\sigma_x = \sqrt{\Sigma_{xx}}, \qquad \Sigma_{xx}, \Sigma_{yy} \le 0.25\ \mathrm{m}^2, \quad \Sigma_{\psi\psi} \le 0.35\ \mathrm{rad}^2. \tag{32}$$

其中标准差与协方差的关系应理解为定义上的平方根，不把它当外部真实误差界。重复长走廊可能让错误初值表现得很自信，因而还要观察 scan-map 重合、外部地标和实际运动。静止定位请求只用新扫描更新，不能给没有扫描的前车制造新鲜定位。实验记录要区分内部质量指标和独立真值误差。

源码依据： `planning.py`：localization_ok；`localization_refresh.py`。

E.5.3 路径缓存的时间复杂度与采样取舍

Trail 添加每个新点时计算现有折线总长度，必要时从前端删除旧点，behind 再反向遍历选点。对 N 个缓存点，一次长度扫描和选点都是线性量级；`max_length` 与 spacing 使长期存储有界。默认 20 m 和 0.08 m 对应的理想规模约数百点，但转角密度和噪声会影响实际点数。

$$N \text{ 量级约为 } L_{\max}/s_{\min}, \qquad C_{\mathrm{add}}, C_{\mathrm{behind}} = O(N). \tag{33}$$

采样间距太小会增加抖动与存储，太大则用长边跨过真实弯曲轨迹。跳变阈值应与目标最大速度和观测间隔相容；相同 0.8 m 阈值对高频和低频数据意味着不同允许运动。若未来更新频率提高，可维护累计弧长以降低重复计算，但当前代码没有实现该优化，报告只作为可检验改进方向提出。

源码依据： `competition_core/planning.py`：Trail.add、Trail.behind。

E.5.4 轨迹采样间距与转角几何

图32向实际Trail.add 函数输入同一条人工定义的折线路径，仅改变最小采样间距。这样可以把“轨迹缓存如何表示转角”与实际定位噪声分开，观察函数本身的保留点行为。背景灰线为给定路径，青色点线为函数保留的轨迹。

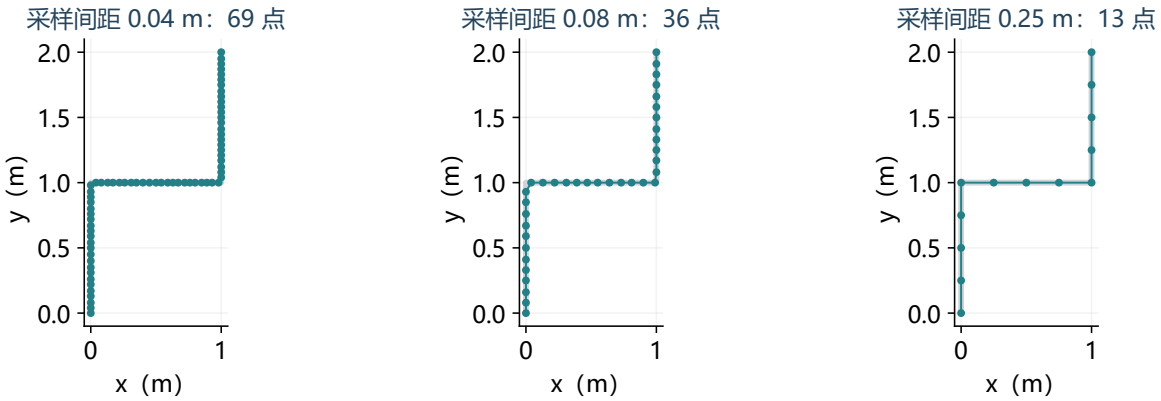


图 32: 实际轨迹缓存函数对不同采样间距的响应；输入为明确给定的合成折线，不是实测轨迹。

较大间距减少保留点数量，也可能使转角附近由一条较长线段连接，从而偏离原始折线。较小间距增加空间分辨率，但不能抑制定位噪声；当噪声本身达到间距量级时，路径弧长还可能被抖动放大。因此 0.08 m 应视为需要结合定位质量和弯道尺度标定的配置，而不是任意场地都最优的常数。

源码依据： planning.py：Trail.add；图源enrich_report_figures.py。

E.5.5 DWA 代价权衡和局部失败解释

当前局部规划在非全向差速限制下采样线速度和角速度，没有横向运动。path_distance_bias、goal_distance_bias 和 occdist_scale 分别影响靠近全局路、追目标和避障。改变权重会改变通道内贴墙、犹豫或切角倾向，但这些是需要实验验证的行为假设，不是调某个权重必然改善的结论。

可调因素	预期影响	应记录的代价
速度/预测时长	反应范围与前视距离	停止余量、计算、目标出视野
路径/目标权重	保持路线与追近目标	横向误差、切角、抢占次数
障碍权重/膨胀	增加围挡余量	窄道不可行、等待时间
目标刷新间隔	降低反复重规划	响应迟滞、终点小位移遗漏

表 39: 导航调参应采用的权衡记录。

失败应结合 actionlib 状态与地图诊断。ABORTED 可能是路径/控制超时，不必然说明感知没看见；REJECTED 可能是目标不可用；LOST 是客户端失去状态。当前程

序将其锁定停车，不能用后台旋转恢复掩盖目标丢失。改进应保留失败样本，比较同场地独立重复，而非仅展示成功的一次。

源码依据： move_base.yaml；map_follower.py 导航状态分支。

E.5.6 终点收队与目标末端冻结

TonyPi 放下物块不代表 TurtleBot 已进入终点。地图 follower 在 mission=done 且距后置点小于 `finish_tolerance` 时置 finished，再取消导航。它未检查整车足迹在终点多边形中，也没有强制连续停稳保持时间；因此需要外部终点区域判定与实测速度证据。

目标限频还有末端问题：最新后置点移动小于 `goal_shift` 时可能不发送新目标，机器人停在旧点附近，而完成判断仍对最新点计算。是否实际导致等待取决于阈值、容差和场景，应通过终点小位移用例验证，不能直接宣称必然失败。当前报告引用 RGB-D 的 `need_goal` 图仅作算法比较，明确它不是地图 follower 已具备的最终目标修正。

$$G_{\text{mapfinish}} = (\text{mission} = \text{done}) \wedge \|p_r - g_{\text{trail}}\| \le \epsilon_f. \tag{34}$$

正式全队完成还要联合四车实际到达与停稳，保留 Pi 状态直到末车完成。终点区域需同时容纳队列的外包络与间隔，这一场地约束不能只按机器人中心点判断。

源码依据： map_follower.py：finished、goal_shift 与 finish_tolerance。

E.5.7 终点刷新与完成容差的边界示例

图33给出一个一维几何示例：机器人已到旧目标，最新回退目标前移 0.24 m。位移小于 0.25 m 重发门限，却大于 0.20 m 完成容差；若其他条件没有触发新目标，就可能出现“没有刷新目标，也未满足完成”的等待。该示例用于解释两个阈值的关系，不表明本次视频已经发生此现象。

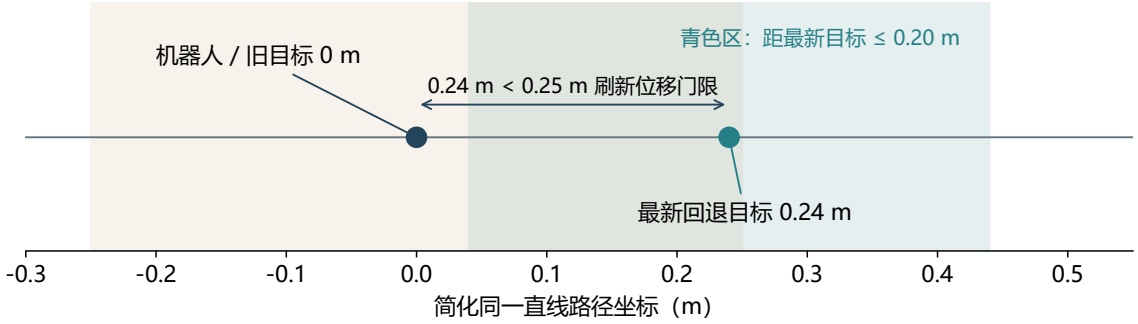


图 33: 终点目标刷新位移与完成容差的简化边界示例。机器人和旧目标重合，青色区为最新目标的完成邻域。

边界问题取决于最新目标、旧目标与实际机器人位置三者的关系，而不是只比较两个配置数值。验证时应保留最后一次发送的导航目标和当前回退点，在进入done 前后

分别检查刷新条件与完成条件。可能的改进包括终点阶段降低位移门限或强制发送最终目标，但这些需要单独实现和验证，本文不将建议写成现有功能。

源码依据： `map_follower.py`：goal_shift 与 finish_tolerance。

E.5.8 从故障命令到物理停稳的时间分解

软件保护有多个时刻：故障发生、上游发现、许可撤销、门控发布零速、底盘实际停稳。日志若只记录零 Twist 发布时间，会低估真实停止延迟；电机控制、电池、地面摩擦和惯性影响最后一段。门控的 0.4/0.5 s 时限只是部分检测预算，不等于最大物理停机时间。

$$T_{\text{stop}} = T_{\text{detect}} + T_{\text{transport}} + T_{\text{gate}} + T_{\text{brake}}, \qquad d_s \approx vT_{\text{delay}} + \frac{v^2}{2a_b}. \tag{35}$$

距离表达是假定匀减速的分析模型，a 应由实测保守下界而不是 DWA 加速度配置替代。方式二当前基于扫描阈值门控，不实现 RGB-D 方案的全方向停止包络。现场建议先低速直道测试断目标、断许可、断扫描，记录独立里程计/外部视频停稳，再进入围挡转角。

源码依据： `velocity_gate.py`；底盘实测制动记录。

F AR 队列跟随的完整启动命令

将功能包放入~/catkin_ws/src/ 后编译：

```
source /opt/ros/kinetic/setup.bash
cd ~/catkin_ws
catkin_make
source devel/setup.bash
```

后车分终端启动底盘、Kinect 和深度扫描转换：

```
# TurtleBot2底盘
source /opt/ros/kinetic/setup.bash
source ~/catkin_ws/devel/setup.bash
roslaunch turtlebot_bringup minimal.launch

# Kinect
source /opt/ros/kinetic/setup.bash
source ~/catkin_ws/devel/setup.bash
roslaunch openni_launch openni.launch

# 深度图转换为 /scan
source /opt/ros/kinetic/setup.bash
rosrun depthimage_to_laserscan depthimage_to_laserscan \
  image:=/camera/depth/image_raw \
  camera_info:=/camera/depth/camera_info
```

确认/scan 持续更新后，二号和三号车分别使用下列参数启动跟随节点：

```
# 二号车跟随一号车（标签ID 0）
roslaunch team_turtlebot_follow_plus follow_ar.launch \
  marker_id:=0 marker_size:=14.0 desired_distance:=0.40 \
  max_linear_speed:=0.35 max_angular_speed:=0.25 \
  avoid_distance:=0.38 stop_distance:=0.20 side_clearance:=0.40 \
  avoid_release_margin:=0.20 obstacle_turn_speed:=0.20 \
  marker_timeout:=0.8 lost_search_timeout:=16 search_angular_speed:=0.20

# 三号车跟随二号车（标签ID 1）
roslaunch team_turtlebot_follow_plus follow_ar.launch \
  marker_id:=1 marker_size:=14.0 desired_distance:=0.40 \
  max_linear_speed:=0.35 max_angular_speed:=0.25 \
  avoid_distance:=0.38 stop_distance:=0.20 side_clearance:=0.40 \
  avoid_release_margin:=0.20 obstacle_turn_speed:=0.20 \
  marker_timeout:=0.8 lost_search_timeout:=16 search_angular_speed:=0.20
```